Table of Contents

AdvancedSceneManager
- SceneManager

AdvancedSceneManager.Callbacks
- CallbackUtility
- CallbackUtility.FluentInvokeAPI<T>
- CallbackUtility.ShowMode
- CombineNull
- CoroutineDiagHelper
- CoroutineDiagHelper.CallerDetails
- CoroutineDiagHelper.SubroutineDetails
- ICollectionClose
- ICollectionExtraData
- ICollectionOpen
- ISceneClose
- ISceneManagerCallbackBase
- ISceneOpen
- LoadingScreen
- ParallelASMCallbacks
- Serializable<T, TValue>
- SerializableDateTime
- SerializableTimeSpan
- SplashScreen

AdvancedSceneManager.Core
- AssetManagement
- Callback
- Callback.When
- CollectionManager
- Editor
- OpenSceneInfo
- Phase
- Runtime
- Runtime.StartProps
- SceneOperation

SceneOperation<ReturnValue>
StandaloneManager
UtilitySceneManager
UtilitySceneManager.ActiveSceneChangedHandler
AdvancedSceneManager.Core.Actions
AggregateAction
CallbackAction
CloseAllUnityScenesAction
OpenAndRunCallbackAction<T>
OpenStartupCollections
OverridableAction<T>
PlaySplashScreenAction
QuickStartupAction
QuitAction
RunCallbackAndCloseAction<T>
SceneAction
SceneCloseAction
SceneCloseCallbackAction
SceneFinishLoadAction
SceneLoadAction
SceneOpenAction
SceneOpenCallbackAction
SceneUnloadAction
StartupAction
AdvancedSceneManager.Defaults
DefaultLoadingScreen
DefaultSplashScreen
IconBounceLoadingScreen
PressAnyButtonLoadingScreen
QuoteLoadingScreen
Quotes
Quotes.Quote
VideoLoadingScreen
AdvancedSceneManager.Editor
EditCollectionPopup
EditorWindow_UIElements<T>
GenericPopup

GenericPopup.Item
IUIToolkitEditor
ObjectField
ObjectField.UxmlFactory
ObjectField.UxmlTraits
OpenInEditorPopup
PickTagPopup
Popup
Popup<T>
SceneAssetEditor
SceneField
SceneField.SceneChangedEvent
SceneField.UxmlFactory
SceneField.UxmlTraits
SceneManagerWindow
SceneManagerWindow.DragAndDropReorder
SceneManagerWindow.DragAndDropReorder.DragElement
SceneManagerWindow.FooterItem
SceneManagerWindow.PostProcess
SceneManagerWindow.Tab
SceneOverviewWindow
ScenePropertyDrawer
ScenesTab
SettingsTab
TagsTab
AdvancedSceneManager.Editor.Utility
AssetDatabaseUtility
AssetsSavedUtility
BlacklistUtility
BuildEventsUtility
BuildEventsUtility.BuildError
BuildSettingsUtility
BuildSettingsUtility.Reason
EditorFolderUtility
EditorGUIUtilityExt
GenericPrompt
GenericPrompt<T, TSelf>

HierarchyGUIUtility
HierarchyGUIUtility.HierarchyGameObjectGUI
HierarchyGUIUtility.HierarchySceneGUI
ListUtility
MenuItemHelper
OnGUIPrompt
PersistentSceneInEditorUtility
PersistentSceneInEditorUtility.OpenInEditorOption
PersistentSceneInEditorUtility.OpenInEditorSetting
PromptInt
PromptKey
PromptName
PromptNameAndMessage
PromptVersion
SettingsTab
SettingsTab._DefaultHeaders
StyleExtensions
TrimUtility
VisualElementExtensions
AdvancedSceneManager.Exceptions
CloseSceneException
OpenSceneException
AdvancedSceneManager.Models
ASMSettings
ASMSettings.Local
ASMSettings.Local.SaveAction
ASMSettings.SettingsProxy
CollectionStartupOption
CollectionThreadPriority
DynamicCollection
ISceneObject
IsOpenReturnValue
LoadingScreenUsage
Profile
Scene
SceneCloseBehavior
SceneCollection

SceneCollectionTemplate
SceneOpenBehavior
SceneTag
TagList
AdvancedSceneManager.Utility
ASM
CanvasGroupExtensions
CanvasSortOrderUtility
DefaultSceneUtility
DictionaryUtility
GuidReference
GuidReference.Editor
GuidReferenceUtility
InGameToolbarUtility
IQueueable
LerpUtility
LoadingScreenUtility
PauseScreenUtility
PersistentUtility
PreloadedSceneHelper
QueueUtility<T>
SceneCollectionUtility
SceneDataUtility
SceneHelper
SceneUtility
ScriptableObjectUtility
SerializableDictionary<TKey, TValue>
SerializableStringBoolDict
TimeSpanUtility

Namespace AdvancedSceneManager

Classes

SceneManager

The core of Advanced Scene Manager, provides access to the following:

assetManagement, provides an overview over all scenes and collections in project and contains functions to manage them.

runtime, manages startup and quit processes of the game.

profile, the current profile, which contains your collections.

settings, settings of the scene manager that isn't stored in the profile.

Scene managers:

collection, contains functions to open or close collections or manage collection scenes.

standalone, contains functions to manage scenes that are not associated with the currently active collection.

utility, contains functions to manage scenes that may be open in either standalone or collection.

editor, a simplified scene manager to manages scenes in editor. Only available in editor.

Class SceneManager

The core of Advanced Scene Manager, provides access to the following:

assetManagement, provides an overview over all scenes and collections in project and contains functions to manage them.

runtime, manages startup and quit processes of the game.

profile, the current profile, which contains your collections.

settings, settings of the scene manager that isn't stored in the profile.

Scene managers:

collection, contains functions to open or close collections or manage collection scenes.

standalone, contains functions to manage scenes that are not associated with the currently active collection.

utility, contains functions to manage scenes that may be open in either standalone or collection.

editor, a simplified scene manager to manages scenes in editor. Only available in editor.

Inheritance

System.Object
SceneManager

Namespace: AdvancedSceneManager

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class SceneManager
```

Properties

assetManagement

Provides an overview over all scenes and collections in project and contains functions to manage them

Declaration

```
public static AssetManagement assetManagement { get; }
```

Property Value

TYPE	DESCRIPTION
AssetManagement	

collection

Provides functions to open or close collections or manage collection scenes

Declaration

```
public static CollectionManager collection { get; }
```

Property Value

TYPE	DESCRIPTION
CollectionManager	

editor

A simplified scene manager to manages scenes in editor.

Declaration

```
public static Editor editor { get; }
```

Property Value

TYPE	DESCRIPTION
Editor	

Remarks

Only available in editor.

profile

The currently active profile.

Declaration

```
public static Profile profile { get; }
```

Property Value

TYPE	DESCRIPTION
Profile	

runtime

Manages startup and quit processes of the game

Declaration

```
public static Runtime runtime { get; }
```

Property Value

TYPE	DESCRIPTION
Runtime	

settings

ASM settings.

Declaration

```
public static ASMSettings.SettingsProxy settings { get; }
```

Property Value

TYPE	DESCRIPTION
ASMSettings.SettingsProxy	

standalone

Provides functions to manage scenes outside that are not associated with the currently active collection

Declaration

```
public static StandaloneManager standalone { get; }
```

Property Value

TYPE	DESCRIPTION
StandaloneManager	

utility

Provides functions to manage scenes that may be open in either standalone or collection

Declaration

```
public static UtilitySceneManager utility { get; }
```

Property Value

TYPE	DESCRIPTION
UtilitySceneManager	

Namespace AdvancedSceneManager.Callbacks

Classes

CallbackUtility

An utility class that invokes callbacks (defined in interfaces based on ISceneManagerCallbackBase), and tracks performance and provides tools for optimizing and diagnosing bottlenecks in these callbacks.

CallbackUtility.FluentInvokeAPI<T>

An helper class to facilitate a fluent api.

Usage: Invoke<T>().

CombineNull

CoroutineDiagHelper

CoroutineDiagHelper.CallerDetails

CoroutineDiagHelper.SubroutineDetails

LoadingScreen

A class that contains callbacks for loading screens.

One instance must exist in a scene that specified as a loading screen.

ParallelASMCallbacks

Specifies whatever the ASM callbacks should be run in parallel for any callbacks defined in this script.

Serializable<T, TValue>

SerializableDateTime

SerializableTimeSpan

SplashScreen

A class that contains callbacks for splash screens.

SplashScreen and LoadingScreen cannot coexist within the same scene.

Interfaces

ICollectionClose

Callback for when a scene in a collection that a UnityEngine.MonoBehaviour is contained within is closed.

Called after loading screen has opened, if one is defined, or else just before collection is closed.

ICollectionExtraData

Callbacks for a UnityEngine.ScriptableObject that has been set as extra data for a collection.

ICollectionOpen

Callback for when a scene in a collection that a UnityEngine.MonoBehaviour is contained within is opened.

Called before loading screen is hidden, if one is defined, or else just when collection has opened.

ISceneClose

Callback for when the scene that a UnityEngine.MonoBehaviour is contained within is closed.

ISceneManagerCallbackBase

Base interface for ISceneOpen, ISceneClose, ICollectionOpen, ICollectionClose. Does nothing on its own, used by CallbackUtility.

ISceneOpen

Callback for when the scene that a UnityEngine.MonoBehaviour is contained within is opened.

Enums

CallbackUtility.ShowMode

Class CallbackUtility

An utility class that invokes callbacks (defined in interfaces based on ISceneManagerCallbackBase), and tracks performance and provides tools for optimizing and diagnosing bottlenecks in these callbacks.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
CallbackUtility

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public class CallbackUtility : EditorWindow
```

Fields

customYieldInstructions

Declaration

```
public static readonly Type[] customYieldInstructions
```

Field Value

TYPE	DESCRIPTION
Type[]	

delayInstructions

Declaration

```
public static readonly Type[] delayInstructions
```

Field Value

TYPE	DESCRIPTION
Type[]	

yieldInstructions

Declaration

```
public static readonly Type[] yieldInstructions
```

Field Value

TYPE	DESCRIPTION
Type[]	

Methods

Invoke<T>()

Declaration

```
public static CallbackUtility.FluentInvokeAPI<T> Invoke<T>()
```

Returns

TYPE	DESCRIPTION
CallbackUtility.FluentInvokeAPI<T>	

Type Parameters

NAME	DESCRIPTION
T	

Open()

Declaration

```
public static void Open()
```

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class CallbackUtility.FluentInvokeAPI<T>

An helper class to facilitate a fluent api.

Usage: Invoke<T>().

Inheritance

System.Object
CallbackUtility.FluentInvokeAPI<T>

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public sealed class FluentInvokeAPI<T>
```

Type Parameters

NAME	DESCRIPTION
T	

Properties

hasDefaultCallback

Gets whatever `T` has a default callback. All callbacks inheriting from ISceneManagerCallbackBase should have one.

Declaration

```
public bool hasDefaultCallback { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

On(OpenSceneInfo[])

Specify the scenes to run this callback on and start execution.

Declaration

```
public IEnumerator On(params OpenSceneInfo[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo[]	scenes	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

On(Scene[])

Specify the open scenes to run this callback on and start execution..

Declaration

```
public IEnumerator On(params Scene[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene[]	scenes	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

On(SceneCollection, Scene[])

Specify the collection scenes to run this callback on and start execution.

Declaration

```
public IEnumerator On(SceneCollection collection, params Scene[] additionalScenes)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
Scene[]	additionalScenes	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

OnAllOpenScenes()

Specify the collection scenes to run this callback on and start execution..

Declaration

```
public IEnumerator OnAllOpenScenes()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

WithCallback(Func<T, IEnumerator>)

Specify a callback, this should point to the interface method which provides a System.Collections.IEnumerator.

This is not needed for callback interfaces inheriting from ISceneManagerCallbackBase.

Declaration

```
public CallbackUtility.FluentInvokeAPI<T> WithCallback(Func<T, IEnumerator> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<T, System.Collections.IEnumerator>	callback	

Returns

TYPE	DESCRIPTION
CallbackUtility.FluentInvokeAPI<T>	

WithParam(Object)

Specify a parameter to use when invoking the callback.

Declaration

```
public CallbackUtility.FluentInvokeAPI<T> WithParam(object param)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	param	

Returns

TYPE	DESCRIPTION
CallbackUtility.FluentInvokeAPI<T>	

Enum CallbackUtility.ShowMode

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum ShowMode
```

Fields

NAME	DESCRIPTION
Frames	
Time	

Class CombineNull

Inheritance

System.Object

CombineNull

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public class CombineNull
```

Fields

value

Declaration

```
public static readonly CombineNull value
```

Field Value

TYPE	DESCRIPTION
CombineNull	

Methods

IsCombineNull(CoroutineDiagHelper.SubroutineDetails)

Declaration

```
public static bool IsCombineNull(CoroutineDiagHelper.SubroutineDetails coroutine)
```

Parameters

TYPE	NAME	DESCRIPTION
CoroutineDiagHelper.SubroutineDetails	coroutine	

Returns

TYPE	DESCRIPTION
System.Boolean	

Class CoroutineDiagHelper

Inheritance

System.Object

CoroutineDiagHelper

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class CoroutineDiagHelper
```

Constructors

CoroutineDiagHelper()

Declaration

```
public CoroutineDiagHelper()
```

CoroutineDiagHelper((MethodBase method, String file, Int32 line), String)

Declaration

```
public CoroutineDiagHelper((MethodBase method, string file, int line) caller, string description)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<System.Reflection.MethodBase, System.String, System.Int32>	caller	
System.String	description	

Fields

caller

Declaration

```
public CoroutineDiagHelper.CallerDetails caller
```

Field Value

TYPE	DESCRIPTION
CoroutineDiagHelper.CallerDetails	

description

Declaration

```
public string description
```

Field Value

TYPE	DESCRIPTION
System.String	

details

Declaration

```
public List<CoroutineDiagHelper.SubroutineDetails> details
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<CoroutineDiagHelper.SubroutineDetails>	

duration

Declaration

```
public SerializableTimeSpan duration
```

Field Value

TYPE	DESCRIPTION
SerializableTimeSpan	

durationFrames

Declaration

```
public int durationFrames
```

Field Value

TYPE	DESCRIPTION
System.Int32	

isComplete

Declaration

```
public bool isComplete
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

isParallel

Declaration

```
public bool isParallel
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

isPaused

Declaration

```
public bool isPaused
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

startFrame

Declaration

```
public int startFrame
```

Field Value

TYPE	DESCRIPTION
System.Int32	

startTime

Declaration

```
public SerializableDateTime startTime
```

Field Value

TYPE	DESCRIPTION
SerializableDateTime	

wasCancelled

Declaration

```
public bool wasCancelled
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

Properties

diagOffset

Declaration

```
public TimeSpan diagOffset { get; }
```

Property Value

TYPE	DESCRIPTION
System.TimeSpan	

Methods

SetParallel(Boolean)

Sets this coroutine as parallel, does nothing beyond provide info for later use, through property isParallel.

Declaration

```
public void SetParallel(bool isParallel)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	isParallel	

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

System.Object.ToString()

ViewCallerInCodeEditor()

View caller in code editor.

Declaration

```
public void ViewCallerInCodeEditor()
```

Class CoroutineDiagHelper.CallerDetails

Inheritance

System.Object

CoroutineDiagHelper.CallerDetails

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class CallerDetails
```

Fields

className

Declaration

```
public string className
```

Field Value

TYPE	DESCRIPTION
System.String	

file

Declaration

```
public string file
```

Field Value

TYPE	DESCRIPTION
System.String	

line

Declaration

```
public int line
```

Field Value

TYPE	DESCRIPTION
System.Int32	

method

Declaration

```
public string method
```

Field Value

TYPE	DESCRIPTION
System.String	

methodParameters

Declaration

```
public string[] methodParameters
```

Field Value

TYPE	DESCRIPTION
System.String[]	

Methods

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

System.Object.ToString()

Class CoroutineDiagHelper.SubroutineDetails

Inheritance

System.Object

CoroutineDiagHelper.SubroutineDetails

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class SubroutineDetails
```

Constructors

SubroutineDetails(Object, Int32, CoroutineDiagHelper, CoroutineDiagHelper.SubroutineDetails)

Declaration

```
public SubroutineDetails(object subroutine, int level, CoroutineDiagHelper helper,
CoroutineDiagHelper.SubroutineDetails parent)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	subroutine	
System.Int32	level	
CoroutineDiagHelper	helper	
CoroutineDiagHelper.SubroutineDetails	parent	

Fields

endFrame

Declaration

```
public int endFrame
```

Field Value

TYPE	DESCRIPTION
System.Int32	

endTime

Declaration

```
public SerializableTimeSpan endTime
```

Field Value

TYPE	DESCRIPTION
SerializableTimeSpan	

isDefaultYieldInstruction

Declaration

```
public bool isDefaultYieldInstruction
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

isDefaultYieldInstructionComponent

Declaration

```
public bool isDefaultYieldInstructionComponent
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

isMethod

Declaration

```
public bool isMethod
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

isValueType

Declaration

```
public bool isValueType
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

level

Declaration

```
public int level
```

Field Value

TYPE	DESCRIPTION
System.Int32	

startFrame

Declaration

```
public int startFrame
```

Field Value

TYPE	DESCRIPTION
System.Int32	

startTime

Declaration

```
public SerializableTimeSpan startTime
```

Field Value

TYPE	DESCRIPTION
SerializableTimeSpan	

subroutine

Declaration

```
public string subroutine
```

Field Value

TYPE	DESCRIPTION
System.String	

type

Declaration

```
public string type
```

Field Value

TYPE	DESCRIPTION
System.String	

Properties

helper

Declaration

```
public CoroutineDiagHelper helper { get; }
```

Property Value

TYPE	DESCRIPTION
CoroutineDiagHelper	

isNull

Declaration

```
public bool isNull { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

End()

Declaration

```
public void End()
```

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

System.Object.ToString()

Interface ICollectionClose

Callback for when a scene in a collection that a UnityEngine.MonoBehaviour is contained within is closed.

Called after loading screen has opened, if one is defined, or else just before collection is closed.

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ICollectionClose : ISceneManagerCallbackBase
```

Methods

OnCollectionClose(SceneCollection)

Callback for when a scene in a collection that a UnityEngine.MonoBehaviour is contained within is closed.

Called after loading screen has opened, if one is defined, or else just before collection is closed.

Declaration

```
IEnumerator OnCollectionClose(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Interface ICollectionExtraData

Callbacks for a UnityEngine.ScriptableObject that has been set as extra data for a collection.

Inherited Members

ICollectionOpen.OnCollectionOpen(SceneCollection)

ICollectionClose.OnCollectionClose(SceneCollection)

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ICollectionExtraData : ICollectionOpen, ICollectionClose, ISceneManagerCallbackBase
```

Interface ICollectionOpen

Callback for when a scene in a collection that a UnityEngine.MonoBehaviour is contained within is opened.

Called before loading screen is hidden, if one is defined, or else just when collection has opened.

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ICollectionOpen : ISceneManagerCallbackBase
```

Methods

OnCollectionOpen(SceneCollection)

Callback for when a scene in a collection that a UnityEngine.MonoBehaviour is contained within is opened.

Called before loading screen is hidden, if one is defined, or else just when collection has opened.

Declaration

```
IEnumerator OnCollectionOpen(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Interface ISceneClose

Callback for when the scene that a UnityEngine.MonoBehaviour is contained within is closed.

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ISceneClose : ISceneManagerCallbackBase
```

Methods

OnSceneClose()

Callback for when the scene that a UnityEngine.MonoBehaviour is contained within is closed.

Declaration

```
IEnumerator OnSceneClose()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Interface ISceneManagerCallbackBase

Base interface for ISceneOpen, ISceneClose, ICollectionOpen, ICollectionClose. Does nothing on its own, used by CallbackUtility.

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ISceneManagerCallbackBase
```

Interface ISceneOpen

Callback for when the scene that a UnityEngine.MonoBehaviour is contained within is opened.

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ISceneOpen : ISceneManagerCallbackBase
```

Methods

OnSceneOpen()

Callback for when the scene that a UnityEngine.MonoBehaviour is contained within is opened.

Declaration

```
IEnumerator OnSceneOpen()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Class LoadingScreen

A class that contains callbacks for loading screens.

One instance must exist in a scene that specified as a loading screen.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
SplashScreen
DefaultLoadingScreen
IconBounceLoadingScreen
PressAnyButtonLoadingScreen
QuoteLoadingScreen
VideoLoadingScreen

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public abstract class LoadingScreen : MonoBehaviour
```

Fields

canvas

The canvas that this loading screen uses.

This will automatically register canvas with CanvasSortOrderUtility, to automatically manage canvas sort order.

You probably want to set this through the inspector.

Declaration

```
[Tooltip("The canvas to automatically manage sort order for, optional.")]
public Canvas canvas
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Canvas	

onDestroy

Occurs when loading screen is destroyed.

Declaration

```
public Action<LoadingScreen> onDestroy
```

Field Value

TYPE	DESCRIPTION
System.Action<LoadingScreen>	

Properties

operation

The current scene operation that this loading screen is associated with. May be null for the first few frames, before loading has actually begun.

Declaration

```
public SceneOperation operation { get; }
```

Property Value

TYPE	DESCRIPTION
SceneOperation	

Methods

OnCancel(SceneOperation)

Called when the associated SceneOperation is cancelled.

Note that OnClose(SceneOperation) is not called.

Declaration

```
public virtual void OnCancel(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

OnClose(SceneOperation)

Called when the associated SceneOperation has ended.

Use this callback to hide your loading screen.

Declaration

```
public abstract IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

OnDestroy()

Declaration

```
protected virtual void OnDestroy()
```

OnOpen(SceneOperation)

Called when the associated SceneOperation is about to start.

Use this callback to show your loading screen, the scene manager will wait until its done.

Declaration

```
public abstract IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

OnScenePhaseChanged(SceneOperation, Phase, Phase)

Called when the associated SceneOperation is moving to a different phase.

Declaration

```
public virtual void OnScenePhaseChanged(SceneOperation operation, Phase previousPhase, Phase nextPhase)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	
Phase	previousPhase	
Phase	nextPhase	

Extension Methods

SceneUtility.Scene(Component)

Class ParallelASMCallbacks

Specifies whatever the ASM callbacks should be run in parallel for any callbacks defined in this script.

Inheritance

System.Object
System.Attribute
ParallelASMCallbacks

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public class ParallelASMCallbacks : Attribute
```

Class Serializable<T, TValue>

Inheritance

System.Object

Serializable<T, TValue>

SerializableDateTime

SerializableTimeSpan

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public abstract class Serializable<T, TValue> : ISerializationCallbackReceiver
```

Type Parameters

NAME	DESCRIPTION
T	
TValue	

Properties

value

Declaration

```
public T value { get; set; }
```

Property Value

TYPE	DESCRIPTION
T	

Methods

Convert(T)

Declaration

```
public abstract TValue Convert(T value)
```

Parameters

TYPE	NAME	DESCRIPTION
T	value	

Returns

TYPE	DESCRIPTION
TValue	

ConvertBack(TValue)

Declaration

```
public abstract T ConvertBack(TValue value)
```

Parameters

TYPE	NAME	DESCRIPTION
TValue	value	

Returns

TYPE	DESCRIPTION
T	

OnAfterDeserialize()

Declaration

```
public void OnAfterDeserialize()
```

OnBeforeSerialize()

Declaration

```
public void OnBeforeSerialize()
```

Class SerializableDateTime

Inheritance

System.Object
Serializable<System.DateTime, System.Int64>
SerializableDateTime

Inherited Members

Serializable<DateTime, Int64>.value
Serializable<DateTime, Int64>.Convert(DateTime)
Serializable<DateTime, Int64>.ConvertBack(Int64)
Serializable<DateTime, Int64>.OnAfterDeserialize()
Serializable<DateTime, Int64>.OnBeforeSerialize()

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class SerializableDateTime : Serializable<DateTime, long>, ISerializationCallbackReceiver
```

Methods

Convert(DateTime)

Declaration

```
public override long Convert(DateTime value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.DateTime	value	

Returns

TYPE	DESCRIPTION
System.Int64	

Overrides

AdvancedSceneManager.Callbacks.Serializable<System.DateTime, System.Int64>.Convert(System.DateTime)

ConvertBack(Int64)

Declaration

```
public override DateTime ConvertBack(long value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Int64	value	

Returns

TYPE	DESCRIPTION
System.DateTime	

Overrides

AdvancedSceneManager.Callbacks.Serializable<System.DateTime, System.Int64>.ConvertBack(System.Int64)

ToString(String)

Declaration

```
public string ToString(string format)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	format	

Returns

TYPE	DESCRIPTION
System.String	

Operators

Implicit(SerializableDateTime to DateTime)

Declaration

```
public static implicit operator DateTime(SerializableDateTime serializable)
```

Parameters

TYPE	NAME	DESCRIPTION
SerializableDateTime	serializable	

Returns

TYPE	DESCRIPTION
System.DateTime	

Implicit(DateTime to SerializableDateTime)

Declaration

```
public static implicit operator SerializableDateTime(DateTime time)
```

Parameters

TYPE	NAME	DESCRIPTION
System.DateTime	time	

Returns

TYPE	DESCRIPTION
SerializableDateTime	

Class SerializableTimeSpan

Inheritance

System.Object
Serializable<System.TimeSpan, System.Int64>
SerializableTimeSpan

Inherited Members

Serializable<TimeSpan, Int64>.value
Serializable<TimeSpan, Int64>.Convert(TimeSpan)
Serializable<TimeSpan, Int64>.ConvertBack(Int64)
Serializable<TimeSpan, Int64>.OnAfterDeserialize()
Serializable<TimeSpan, Int64>.OnBeforeSerialize()

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class SerializableTimeSpan : Serializable<TimeSpan, long>, ISerializationCallbackReceiver
```

Methods

Convert(TimeSpan)

Declaration

```
public override long Convert(TimeSpan value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.TimeSpan	value	

Returns

TYPE	DESCRIPTION
System.Int64	

Overrides

AdvancedSceneManager.Callbacks.Serializable<System.TimeSpan, System.Int64>.Convert(System.TimeSpan)

ConvertBack(Int64)

Declaration

```
public override TimeSpan ConvertBack(long value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Int64	value	

Returns

TYPE	DESCRIPTION
System.TimeSpan	

Overrides

AdvancedSceneManager.Callbacks.Serializable<System.TimeSpan, System.Int64>.ConvertBack(System.Int64)

ToString(String)

Declaration

```
public string ToString(string format)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	format	

Returns

TYPE	DESCRIPTION
System.String	

Operators

Implicit(SerializableTimeSpan to TimeSpan)

Declaration

```
public static implicit operator TimeSpan(SerializableTimeSpan serializable)
```

Parameters

TYPE	NAME	DESCRIPTION
SerializableTimeSpan	serializable	

Returns

TYPE	DESCRIPTION
System.TimeSpan	

Implicit(TimeSpan to SerializableTimeSpan)

Declaration

```
public static implicit operator SerializableTimeSpan(TimeSpan time)
```

Parameters

TYPE	NAME	DESCRIPTION
System.TimeSpan	time	

Returns

TYPE	DESCRIPTION
SerializableTimeSpan	

Class SplashScreen

A class that contains callbacks for splash screens.

SplashScreen and LoadingScreen cannot coexist within the same scene.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
SplashScreen
DefaultSplashScreen

Inherited Members

LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Callbacks

Assembly: AdvancedSceneManager.dll

Syntax

```
public abstract class SplashScreen : LoadingScreen
```

Methods

DisplaySplashScreen()

Called when scene manager is ready to display the splash screen.

Example: yielding new WaitForSeconds(5) will show the splash screen for 5 seconds.

Declaration

```
public abstract IEnumerator DisplaySplashScreen()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

OnClose(SceneOperation)

Declaration

```
public override IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnClose(SceneOperation)

OnOpen(SceneOperation)

Declaration

```
public override IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnOpen(SceneOperation)

Extension Methods

SceneUtility.Scene(Component)

Namespace AdvancedSceneManager.Core

Classes

AssetManagement

Manages assets related to ASM.

Callback

Represents a callback that can be run on Phase change, or right before loading screen hide (or when it would, if it was enabled).

CollectionManager

The manager for collection scenes.

Editor

A simplified scene manager for managing scenes in editor.

OpenSceneInfo

Runtime

Manages the start and quit processes of the game.

SceneOperation

A scene operation is a queueable operation that can open or close scenes. See also: SceneAction.

SceneOperation<ReturnValue>

A scene operation is a queueable operation that can open or close scenes. See also: SceneAction.

StandaloneManager

The manager for stand-alone scenes.

UtilitySceneManager

An utility scene manager that helps with actions that might relate to either collection or standalone managers.

Structs

Runtime.StartProps

Enums

Callback.When

Phase

The phase that a SceneOperation is currently in.

Delegates

UtilitySceneManager.ActiveSceneChangedHandler

Class AssetManagement

Manages assets related to ASM.

Inheritance

System.Object
AssetManagement

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class AssetManagement
```

Properties

allowAutoRefresh

If false, then assets will not be refreshed, this will mean that no Scene ScriptableObject will be created when a SceneAsset added, and a Scene will also not be removed when its associated SceneAsset is removed.

Declaration

```
public bool allowAutoRefresh { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

collections

The collections in this project.

Declaration

```
public ReadOnlyCollection<SceneCollection> collections { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<SceneCollection>	

profiles

The scenes in this project.

Declaration

```
public ReadOnlyCollection<Profile> profiles { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<Profile>	

scenes

The scenes in this project.

Declaration

```
public ReadOnlyCollection<Scene> scenes { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<Scene>	

Methods

Add(SceneAsset, Boolean)

Adds the UnityEditor.SceneAsset to asm. Returns existing Scene if already exist.

Declaration

```
public Scene Add(SceneAsset asset, bool ignoreBlacklist = false)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEditor.SceneAsset	asset	
System.Boolean	ignoreBlacklist	

Returns

TYPE	DESCRIPTION
Scene	

Remarks

Returns null if scene has been added to Ignore(String).

Add<T>(T, Profile, Boolean, Boolean)

Adds the asset.

Declaration

```
public void Add<T>(T obj, Profile profile = null, bool import = true, bool ignoreBlacklist = false)
    where T : ScriptableObject, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
T	obj	
Profile	profile	
System.Boolean	import	
System.Boolean	ignoreBlacklist	

Type Parameters

NAME	DESCRIPTION
T	

AddCollectionToProfile(SceneCollection, Profile)

Adds the collection to the profile.

Declaration

```
public void AddCollectionToProfile(SceneCollection collection, Profile profile)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
Profile	profile	

Remarks

This removes collection from profile, if already associated with one.

Clear()

Clear assets.

Declaration

```
public void Clear()
```

Create<T>(String, Profile, Action<T>, Boolean)

Create and add an asset.

Declaration

```
public T Create<T>(string name, Profile profile = null, Action<T> initializeBeforeSave = null, bool
ignoreBlacklist = false)
    where T : ScriptableObject, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
Profile	profile	
System.Action<T>	initializeBeforeSave	
System.Boolean	ignoreBlacklist	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

Create<T>(String, Action<T>, Boolean)

Declaration

```
public T Create<T>(string name, Action<T> initializeBeforeSave = null, bool ignoreBlacklist = false)
    where T : ScriptableObject, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
System.Action<T>	initializeBeforeSave	
System.Boolean	ignoreBlacklist	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

CreateProfile(String, Boolean)

Create a new profile.

Declaration

```
public Profile CreateProfile(string name = null, bool promptBlacklist = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
System.Boolean	promptBlacklist	

Returns

TYPE	DESCRIPTION
Profile	

CreateProfileAndAssign(Boolean)

Creates a new profile and assigns it as active.

Declaration

```
public void CreateProfileAndAssign(bool promptBlacklist = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	promptBlacklist	

DuplicateProfile()

Duplicates the active profile.

Declaration

```
public Profile DuplicateProfile()
```

Returns

TYPE	DESCRIPTION
Profile	

DuplicateProfileAndAssign()

Duplicates active profile and assigns it as active.

Declaration

```
public void DuplicateProfileAndAssign()
```

FindAssetByID<T>(String)

Find the `T` with the associated asset ID.

Declaration

```
public T FindAssetByID<T>(string assetID)
    where T : Object
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	assetID	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

FindAssetByPath<T>(String)

Find the `T` with the specified path.

Declaration

```
public T FindAssetByPath<T>(string path)
    where T : Object
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

FindSceneByPath(String)

Find the Scene with the associated path (this is the path to the UnityEditor.SceneAsset).

Declaration

```
public Scene FindSceneByPath(string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Returns

TYPE	DESCRIPTION
Scene	

Ignore(String)

Ignores the scene at the specified path.

Declaration

```
public void Ignore(string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Remarks

Only available in editor.

IsIgnored(String)

Gets if the scene should be ignored.

Declaration

```
public bool IsIgnored(string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Returns

TYPE	DESCRIPTION
System.Boolean	

Remarks

Only available in editor.

Remove<T>(T)

Removes the asset.

Declaration

```
public void Remove<T>(T obj)
    where T : ScriptableObject, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
T	obj	

Type Parameters

NAME	DESCRIPTION
T	

Rename<T>(T, String)

Renames the Scene or SceneCollection.

Declaration

```
public void Rename<T>(T obj, string newName)
    where T : ScriptableObject, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
T	obj	
System.String	newName	

Type Parameters

NAME	DESCRIPTION
T	

Events

AssetsChanged

Called when assets changed.

Declaration

```
public event Action AssetsChanged
```

Event Type

TYPE	DESCRIPTION
System.Action	

AssetsCleared

Called when assets are cleared, by either Clear() or from ui.

Declaration

```
public event Action AssetsCleared
```

Event Type

TYPE	DESCRIPTION
System.Action	

Class Callback

Represents a callback that can be run on Phase change, or right before loading screen hide (or when it would, if it was enabled).

Inheritance

System.Object
Callback

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Callback
```

Properties

action

Declaration

```
public Action action { get; }
```

Property Value

TYPE	DESCRIPTION
System.Action	

action2

Declaration

```
public Action<SceneOperation> action2 { get; }
```

Property Value

TYPE	DESCRIPTION
System.Action<SceneOperation>	

coroutine

Declaration

```
public GlobalCoroutine coroutine { get; }
```

Property Value

TYPE	DESCRIPTION
Lazy.Utility.GlobalCoroutine	

enumerator

Declaration

```
public Func<IEnumerator> enumerator { get; }
```

Property Value

TYPE	DESCRIPTION
System.Func<System.Collections.IEnumerator>	

phase

Declaration

```
public Phase? phase { get; }
```

Property Value

TYPE	DESCRIPTION
System.Nullable<Phase>	

scene

Declaration

```
public Scene scene { get; }
```

Property Value

TYPE	DESCRIPTION
Scene	

when

Declaration

```
public Callback.When when { get; }
```

Property Value

TYPE	DESCRIPTION
Callback.When	

Methods

After(Phase)

Runs a callback after the specified phase.

Declaration

```
public static Callback After(Phase phase)
```

Parameters

TYPE	NAME	DESCRIPTION
Phase	phase	

Returns

TYPE	DESCRIPTION
Callback	

After(Phase, Scene)

Runs a callback after the specified phase.

Declaration

```
public static Callback After(Phase phase, Scene on)
```

Parameters

TYPE	NAME	DESCRIPTION
Phase	phase	
Scene	on	

Returns

TYPE	DESCRIPTION
Callback	

AfterLoadingScreenOpen()

Runs a callback after loading screen has opened.

Declaration

```
public static Callback AfterLoadingScreenOpen()
```

Returns

TYPE	DESCRIPTION
Callback	

Before(Phase)

Runs a callback before the specified phase, when processing the specified scene.

Declaration

```
public static Callback Before(Phase phase)
```

Parameters

TYPE	NAME	DESCRIPTION
Phase	phase	

Returns

TYPE	DESCRIPTION
Callback	

Remarks

Phase will still have changed to the next though, but scene actions won't run until after callback.

Before(Phase, Scene)

Runs a callback before the specified phase, when processing the specified scene.

Declaration

```
public static Callback Before(Phase phase, Scene on)
```

Parameters

TYPE	NAME	DESCRIPTION
Phase	phase	
Scene	on	

Returns

TYPE	DESCRIPTION
Callback	

Remarks

Phase will still have changed to the next though, but scene actions won't run until after callback.

BeforeLoadingScreenClose()

Runs a callback before loading screen closes.

Declaration

```
public static Callback BeforeLoadingScreenClose()
```

Returns

TYPE	DESCRIPTION
Callback	

Do(GlobalCoroutine)

Performs the specified callback.

Declaration

```
public Callback Do(GlobalCoroutine coroutine)
```

Parameters

TYPE	NAME	DESCRIPTION
Lazy.Utility.GlobalCoroutine	coroutine	

Returns

TYPE	DESCRIPTION
Callback	

Do(Action)

Performs the specified callback.

Declaration

```
public Callback Do(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

Returns

TYPE	DESCRIPTION
Callback	

Do(Action<SceneOperation>)

Performs the specified callback.

Declaration

```
public Callback Do(Action<SceneOperation> action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<SceneOperation>	action	

Returns

TYPE	DESCRIPTION
Callback	

Do(Func<IEnumerator>)

Performs the specified callback.

Declaration

```
public Callback Do(Func<IEnumerator> enumerator)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<System.Collections.IEnumerator>	enumerator	

Returns

TYPE	DESCRIPTION
Callback	

Do(Single)

Performs the specified callback.

Declaration

```
public Callback Do(float delay)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Single	delay	

Returns

TYPE	DESCRIPTION
Callback	

Operators

Implicit(GlobalCoroutine to Callback)

Declaration

```
public static implicit operator Callback(GlobalCoroutine coroutine)
```

Parameters

TYPE	NAME	DESCRIPTION
Lazy.Utility.GlobalCoroutine	coroutine	

Returns

TYPE	DESCRIPTION
Callback	

Implicit(Action to Callback)

Declaration

```
public static implicit operator Callback(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

Returns

TYPE	DESCRIPTION
Callback	

Implicit(Action<SceneOperation> to Callback)

Declaration

```
public static implicit operator Callback(Action<SceneOperation> action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<SceneOperation>	action	

Returns

TYPE	DESCRIPTION
Callback	

Implicit(Func<IEnumerator> to Callback)

Declaration

```
public static implicit operator Callback(Func<IEnumerator> enumerator)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<System.Collections.IEnumerator>	enumerator	

Returns

TYPE	DESCRIPTION
Callback	

Enum Callback.When

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum When
```

Fields

NAME	DESCRIPTION
After	
Before	

Class CollectionManager

The manager for collection scenes.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
AdvancedSceneManager.Core.SceneManagerBase
CollectionManager

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class CollectionManager : SceneManagerBase, ISerializationCallbackReceiver
```

Remarks

Usage: collection.

Properties

current

The currently open collection.

Declaration

```
public SceneCollection current { get; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

previous

The previously open collection.

Declaration

```
public SceneCollection previous { get; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

Methods

CanOpen(Scene)

Gets whatever the scene can be opened by the current collection.

Declaration

```
public override bool CanOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Overrides

AdvancedSceneManager.Core.SceneManagerBase.CanOpen(AdvancedSceneManager.Models.Scene)

Close()

Closes the current collection.

Declaration

```
public SceneOperation Close()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

Close(OpenSceneInfo)

Closes a scene.

Declaration

```
public override SceneOperation Close(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

Overrides

AdvancedSceneManager.Core.SceneManagerBase.Close(AdvancedSceneManager.Core.OpenSceneInfo)

Remarks

Throws a CloseSceneException if the scene is not a part of the current collection.

CloseMultiple(OpenSceneInfo[])

Closes the scenes.

Declaration

```
public override SceneOperation CloseMultiple(params OpenSceneInfo[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo[]	scenes	

Returns

TYPE	DESCRIPTION
SceneOperation	

Overrides

AdvancedSceneManager.Core.SceneManagerBase.CloseMultiple(AdvancedSceneManager.Core.OpenSceneInfo[])

Remarks

Throws a CloseSceneException if a scene is not a part of the current collection.

IsOpen(SceneCollection)

Gets whatever the collection is currently open.

Declaration

```
public bool IsOpen(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Boolean	

OnAfterDeserialize2()

Declaration

```
public override void OnAfterDeserialize2()
```

Overrides

AdvancedSceneManager.Core.SceneManagerBase.OnAfterDeserialize2()

Open(Scene)

Opens a scene.

Declaration

```
public override SceneOperation<OpenSceneInfo> Open(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

Overrides

AdvancedSceneManager.Core.SceneManagerBase.Open(AdvancedSceneManager.Models.Scene)

Remarks

Throws a OpenSceneException if the scene cannot be opened by the current collection.

Open(SceneCollection, Boolean, Boolean)

Opens the collection.

Declaration

```
public SceneOperation Open(SceneCollection collection, bool ignoreLoadingScreen = false, bool force = false)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Boolean	ignoreLoadingScreen	
System.Boolean	force	Open even if scene is tagged with DoNotOpen.

Returns

TYPE	DESCRIPTION
SceneOperation	

OpenMultiple(Scene[])

Opens the scenes.

Declaration

```
public override SceneOperation<OpenSceneInfo[]> OpenMultiple(params Scene[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene[]	scenes	

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo[]>	

Overrides

AdvancedSceneManager.Core.SceneManagerBase.OpenMultiple(AdvancedSceneManager.Models.Scene[])

Remarks

Throws a OpenSceneException if a scene cannot be opened by the current collection.

Reopen(Boolean)

Reopens the current collection.

Declaration

```
public SceneOperation Reopen(bool reopenPersistent = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	reopenPersistent	

Returns

TYPE	DESCRIPTION
SceneOperation	

Toggle(SceneCollection, Nullable<Boolean>)

Toggles the collection.

Declaration

```
public SceneOperation Toggle(SceneCollection collection, bool? enabled = null)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Nullable<System.Boolean>	enabled	If null, collection will be toggled on or off depending on whatever collection is open or not. Pass a value to ensure that collection either open or closed.

Returns

TYPE	DESCRIPTION
SceneOperation	

Events

closed

Called when a collection is closed.

Declaration

```
public event Action<SceneCollection> closed
```

Event Type

TYPE	DESCRIPTION
System.Action<SceneCollection>	

opened

Called when a collection is opened.

Declaration

```
public event Action<SceneCollection> opened
```

Event Type

TYPE	DESCRIPTION
System.Action<SceneCollection>	

Operators

Implicit(CollectionManager to SceneCollection)

Declaration

```
public static implicit operator SceneCollection(CollectionManager manager)
```

Parameters

TYPE	NAME	DESCRIPTION
CollectionManager	manager	

Returns

TYPE	DESCRIPTION
SceneCollection	

Implicit(CollectionManager to Boolean)

Declaration

```
public static implicit operator bool (CollectionManager manager)
```

Parameters

TYPE	NAME	DESCRIPTION
CollectionManager	manager	

Returns

TYPE	DESCRIPTION
System.Boolean	

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class Editor

A simplified scene manager for managing scenes in editor.

Inheritance

System.Object
Editor

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Editor
```

Remarks

Only available in editor.

Methods

CanClose(SceneCollection)

Declaration

```
public bool CanClose(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Boolean	

Close(Scene, Boolean)

Declaration

```
public void Close(Scene scene, bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	promptSave	

Close(SceneCollection, Boolean)

Declaration

```
public void Close(SceneCollection collection, bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Boolean	promptSave	

Close(Scene, Boolean)

Declaration

```
public void Close(Scene scene, bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	
System.Boolean	promptSave	

CloseAll(Boolean)

Declaration

```
public void CloseAll(bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	promptSave	

IsOpen(Scene)

Declaration

```
public bool IsOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

IsOpen(SceneCollection)

Declaration

```
public bool IsOpen(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Boolean	

Open(Scene, Boolean)

Declaration

```
public Scene? Open(Scene scene, bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	promptSave	

Returns

TYPE	DESCRIPTION
System.Nullable<UnityEngine.SceneManagement.Scene>	

Open(SceneCollection, Boolean, Boolean, Boolean)

Declaration

```
public void Open(SceneCollection collection, bool additive = false, bool promptSave = true, bool
forceOpenAllScenes = false)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Boolean	additive	
System.Boolean	promptSave	
System.Boolean	forceOpenAllScenes	

OpenSingle(Scene, Boolean)

Declaration

```
public void OpenSingle(Scene scene, bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	promptSave	

OpenSingle(SceneAsset, Boolean)

Declaration

```
public void OpenSingle(SceneAsset scene, bool promptSave = true)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEditor.SceneAsset	scene	
System.Boolean	promptSave	

Events

scenesUpdated

Declaration

```
public event Action scenesUpdated
```

Event Type

TYPE	DESCRIPTION
System.Action	

Class OpenSceneInfo

Inheritance

System.Object

OpenSceneInfo

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class OpenSceneInfo : IEquatable<OpenSceneInfo>, IEquatable<Scene>, IEquatable<Scene>,
IEquatable<SceneAsset>
```

Properties

isActive

Gets whatever this scene is the active scene.

Declaration

```
public bool isActive { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isCollection

Gets whatever this scene is a collection scene.

Declaration

```
public bool isCollection { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isOpen

Gets whatever this scene is currently open.

Declaration

```
public bool isOpen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isPersistent

Gets whatever this scene is persistent. See PersistentUtility for more details.

Declaration

```
public bool isPersistent { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isPreloaded

Gets whatever this scene is preloaded.

Declaration

```
public bool isPreloaded { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isSpecial

Gets whatever this scene is a special scene, i.e. splash screen / loading screen.

Declaration

```
public bool isSpecial { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isStandalone

Gets whatever this scene is a collection scene.

Declaration

```
public bool isStandalone { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

scene

The Scene that this OpenSceneInfo is associated with.

Declaration

```
public Scene scene { get; }
```

Property Value

TYPE	DESCRIPTION
Scene	

sceneManager

The scene manager associated with this OpenSceneInfo.

Declaration

```
public SceneManagerBase sceneManager { get; }
```

Property Value

TYPE	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	

unityScene

The UnityEngine.SceneManagement.Scene that this OpenSceneInfo is associated with.

Declaration

```
public Scene? unityScene { get; }
```

Property Value

TYPE	DESCRIPTION
System.Nullable<UnityEngine.SceneManagement.Scene>	

Methods

Equals(OpenSceneInfo)

Declaration

```
public bool Equals(OpenSceneInfo osi)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	osi	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Scene)

Declaration

```
public bool Equals(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Nullable<Scene>)

Declaration

```
public bool Equals(Scene? scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Nullable<UnityEngine.SceneManagement.Scene>	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Object)

Declaration

```
public override bool Equals(object other)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	other	

Returns

TYPE	DESCRIPTION
System.Boolean	

Overrides

System.Object.Equals(System.Object)

Equals(SceneAsset)

Declaration

```
public bool Equals(SceneAsset asset)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEditor.SceneAsset	asset	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Scene)

Declaration

```
public bool Equals(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

GetHashCode()

Declaration

```
public override int GetHashCode()
```

Returns

TYPE	DESCRIPTION
System.Int32	

Overrides

System.Object.GetHashCode()

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

System.Object.ToString()

Enum Phase

The phase that a SceneOperation is currently in.

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum Phase
```

Fields

NAME	DESCRIPTION
CloseCallbacks	The scene operation is currently executing close callbacks on the scenes that are being closed, if any.
CustomActions	The scene operation is currently executing custom actions, added through WithAction(SceneAction[]) or similar methods, if any.
FinishLoad	The scene operation is currently finishing loading / activating the scenes, if any.
LoadScenes	The scene operation is currently loading the scenes, if any.
OpenCallbacks	The scene operation is currently executing open callbacks on the scenes that are being opened, if any.
UnloadScenes	The scene operation is currently unloading the scenes, if any.

Class Runtime

Manages the start and quit processes of the game.

Inheritance

System.Object
Runtime

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Runtime
```

Properties

isBuildMode

Gets whatever we're currently in build mode.

Declaration

```
public bool isBuildMode { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Remarks

This is true when in build or when play button in scene manager window is pressed.

isInitialized

Gets whatever ASM is done with startup process.

Declaration

```
public bool isInitialized { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isQuitting

Gets whatever ASM is currently in the process of quitting.

Declaration

```
public bool isQuitting { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

wasStartedAsBuild

Gets if game was started as a build.

Declaration

```
public bool wasStartedAsBuild { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

CancelQuit()

Cancels a quit in progress.

Only usable during a RegisterQuitCallback() or while isQuitting is true.

Declaration

```
public void CancelQuit()
```

Quit(Boolean, Nullable<Color>, Single)

Quits the game, and calls quitCallbacks, optionally with a fade animation.

Declaration

```
public void Quit(bool fade = true, Color? fadeColor = null, float fadeDuration = 1F)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	fade	
System.Nullable<UnityEngine.Color>	fadeColor	
System.Single	fadeDuration	

RegisterQuitCallback(IEnumerator)

Register a callback to be called before quit.

Declaration

```
public void RegisterQuitCallback(IEnumerator courutine)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.IEnumerator	courutine	

Restart(Boolean)

Restarts game and plays startup sequence again.

Declaration

```
public void Restart(bool playSplashScreen = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	playSplashScreen	

Remarks

Enters playmode if in editor.

Start(SceneCollection, Boolean, Boolean)

Starts startup sequence.

Declaration

```
public void Start(SceneCollection collection = null, bool ignoreDoNotOpen = false, bool playSplashScreen =
true)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	Opens the collection after all other collections and scenes flagged to open has.
System.Boolean	ignoreDoNotOpen	
System.Boolean	playSplashScreen	

Remarks

Enters playmode if in editor.

UnregisterQuitCallback(IEnumerator)

Unregister a callback that was to be called before quit.

Declaration

```
public void UnregisterQuitCallback(IEnumerator courutine)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.IEnumerator	courutine	

Events

afterStart

Occurs after startup process is done, or when Restart(Boolean) is called.

Declaration

```
public event Action afterStart
```

Event Type

TYPE	DESCRIPTION
System.Action	

beforeStart

Occurs before startup process is started, or when Restart(Boolean) is called.

Declaration

```
public event Action beforeStart
```

Event Type

TYPE	DESCRIPTION
System.Action	

Struct Runtime.StartProps

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public struct StartProps
```

Fields

beforeSplashScreenFadeDuration

Declaration

```
public float beforeSplashScreenFadeDuration
```

Field Value

TYPE	DESCRIPTION
System.Single	

fadeColor

Declaration

```
public Color fadeColor
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Color	

ignoreDoNotOpen

Declaration

```
public bool ignoreDoNotOpen
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

initialFadeDuration

Declaration

```
public float initialFadeDuration
```

Field Value

TYPE	DESCRIPTION
System.Single	

skipSplashScreen

Declaration

```
public bool skipSplashScreen
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

Properties

overrideOpenCollection

Declaration

```
public SceneCollection overrideOpenCollection { get; set; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

Methods

GetDefault()

Gets the default Runtime.StartProps.

Declaration

```
public static Runtime.StartProps GetDefault()
```

Returns

TYPE	DESCRIPTION
Runtime.StartProps	

Remarks

Cannot be called during UnityEngine.Object constructor.

Class SceneOperation

A scene operation is a queueable operation that can open or close scenes. See also: SceneAction.

Inheritance

System.Object
UnityEngine.CustomYieldInstruction
SceneOperation
SceneOperation<ReturnValue>

Implements

IQueueable

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneOperation : CustomYieldInstruction, IEnumerator, IQueueable
```

Remarks

See also: SceneOperation<ReturnValue>.

Fields

_extCallbacks

Declaration

```
protected static readonly List<Callback> _extCallbacks
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<Callback>	

isDone

Declaration

```
protected bool isDone
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

Properties

actions

Declaration

```
public ReadOnlyCollection<SceneAction> actions { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<SceneAction>	

callbacks

Declaration

```
public ReadOnlyCollection<Callback> callbacks { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<Callback>	

cancelled

Gets if this scene operation is cancelled.

Declaration

```
public bool cancelled { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

clearUnusedAssets

Declaration

```
public bool? clearUnusedAssets { get; }
```

Property Value

TYPE	DESCRIPTION
System.Nullable<System.Boolean>	

close

Declaration

```
public ReadOnlyCollection<(OpenSceneInfo scene, bool force)> close { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<System.ValueTuple<OpenSceneInfo, System.Boolean>>	

closeBehavior

Declaration

```
public SceneCloseBehavior? closeBehavior { get; }
```

Property Value

TYPE	DESCRIPTION
System.Nullable<SceneCloseBehavior>	

collection

Declaration

```
public SceneCollection collection { get; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

current

The current action that is executing.

Declaration

```
public SceneAction current { get; }
```

Property Value

TYPE	DESCRIPTION
SceneAction	

customActions

Declaration

```
public ReadOnlyCollection<SceneAction> customActions { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<SceneAction>	

doCollectionCallbacks

Declaration

```
public bool doCollectionCallbacks { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

done

Gets a SceneOperation that has already completed.

Declaration

```
public static SceneOperation done { get; }
```

Property Value

TYPE	DESCRIPTION
SceneOperation	

friendlyText

Declaration

```
public string friendlyText { get; protected set; }
```

Property Value

TYPE	DESCRIPTION
System.String	

keepWaiting

Inherited from UnityEngine.CustomYieldInstruction. Tells unity whatever the operation is done or not.

Declaration

```
public override bool keepWaiting { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

UnityEngine.CustomYieldInstruction.keepWaiting

loadingPriority

Declaration

```
public ThreadPriority? loadingPriority { get; }
```

Property Value

TYPE	DESCRIPTION
System.Nullable<UnityEngine.ThreadPriority>	

loadingScreen

Declaration

```
public Scene loadingScreen { get; }
```

Property Value

TYPE	DESCRIPTION
Scene	

loadingScreenCallback

Declaration

```
public Action<LoadingScreen> loadingScreenCallback { get; }
```

Property Value

TYPE	DESCRIPTION
System.Action<LoadingScreen>	

open

Declaration

```
public ReadOnlyCollection<(Scene scene, bool force)> open { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<System.ValueTuple<Scene, System.Boolean>>	

openedLoadingScreen

Gets the loading screen that was opened.

Declaration

```
public SceneOperation<LoadingScreen> openedLoadingScreen { get; }
```

Property Value

TYPE	DESCRIPTION
SceneOperation<LoadingScreen>	

openedScenes

Gets the scenes that was opened in this operation.

Declaration

```
public IEnumerable<OpenSceneInfo> openedScenes { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	

phase

The phase the this scene operation is currently in.

Declaration

```
public Phase phase { get; }
```

Property Value

TYPE	DESCRIPTION
Phase	

reopen

Declaration

```
public ReadOnlyCollection<OpenSceneInfo> reopen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<OpenSceneInfo>	

sceneManager

The scene manager that requested this scene operation.

Declaration

```
public SceneManagerBase sceneManager { get; }
```

Property Value

TYPE	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	

totalProgress

The total progress made by this operation.

Declaration

```
public float totalProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

useLoadingScreen

Declaration

```
public bool useLoadingScreen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

Add(SceneManagerBase, Boolean)

Adds a new scene operation to the queue.

Declaration

```
public static SceneOperation Add(SceneManagerBase sceneManager, bool ignoreQueue = false)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	sceneManager	The scene manager that should manage the scenes when opened.
System.Boolean	ignoreQueue	Sets whatever this operation should ignore the queue, and start immediately.

Returns

TYPE	DESCRIPTION
SceneOperation	

Add<ReturnValue>(SceneManagerBase, Boolean)

Adds a new scene operation to the queue.

Declaration

```
public static SceneOperation<ReturnValue> Add<ReturnValue>(SceneManagerBase sceneManager, bool ignoreQueue =
false)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	sceneManager	The scene manager that should manage the scenes when opened.
System.Boolean	ignoreQueue	Sets whatever this operation should ignore the queue, and start immediately.

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Type Parameters

NAME	DESCRIPTION
ReturnValue	

Add<ReturnValue>(SceneManagerBase, Func<SceneOperation, ReturnValue>, Boolean)

Adds a new scene operation to the queue.

Declaration

```
public static SceneOperation<ReturnValue> Add<ReturnValue>(SceneManagerBase sceneManager, Func<SceneOperation, ReturnValue> return, bool ignoreQueue = false)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	sceneManager	
System.Func<SceneOperation, ReturnValue>	return	
System.Boolean	ignoreQueue	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Type Parameters

NAME	DESCRIPTION
ReturnValue	

AddCallback(Callback)

Adds the callback to every scene operation.

Declaration

```
public static void AddCallback(Callback callback)
```

Parameters

TYPE	NAME	DESCRIPTION
Callback	callback	

AsPersistent(SceneCloseBehavior)

Declaration

```
public SceneOperation AsPersistent(SceneCloseBehavior closeBehavior)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCloseBehavior	closeBehavior	

Returns

TYPE	DESCRIPTION
SceneOperation	

Cancel(Action)

Cancel this operation.

Note that the operation might not be cancelled immediately, if user defined callbacks are currently running (WithAction(), WithCallback()) they will run to completion before operation is cancelled. 'cancelled' property can be used in callbacks to check whatever a operation is cancelled.

Declaration

```
public void Cancel(Action callbackWhenFullyCancelled = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	callbackWhenFullyCancelled	

Close(OpenSceneInfo[])

Declaration

```
public SceneOperation Close(params OpenSceneInfo[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo[]	scenes	

Returns

TYPE	DESCRIPTION
SceneOperation	

Close(Boolean, OpenSceneInfo[])

Declaration

```
public SceneOperation Close(bool force, params OpenSceneInfo[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	force	
OpenSceneInfo[]	scenes	

Returns

TYPE	DESCRIPTION
SceneOperation	

Close(IEnumerable<OpenSceneInfo>, Boolean)

Declaration

```
public SceneOperation Close(IEnumerable<OpenSceneInfo> scenes, bool force = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	scenes	
System.Boolean	force	

Returns

TYPE	DESCRIPTION
SceneOperation	

FindActions<T>()

Finds the actions of a specified type that was used in this operation.

Declaration

```
public IEnumerable<SceneAction> FindActions<T>()
    where T : SceneAction
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<SceneAction>	

Type Parameters

NAME	DESCRIPTION
T	

FindLastAction<T>()

Finds the last action of a specified type that was used in this operation.

Declaration

```
public SceneAction FindLastAction<T>()
    where T : SceneAction
```

Returns

TYPE	DESCRIPTION
SceneAction	

Type Parameters

NAME	DESCRIPTION
T	

Open(Scene[])

Declaration

```
public SceneOperation Open(params Scene[] scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene[]	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

Open(IEnumerable<Scene>, Boolean)

Declaration

```
public SceneOperation Open(IEnumerable<Scene> scene, bool force = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	scene	
System.Boolean	force	

Returns

TYPE	DESCRIPTION
SceneOperation	

RemoveCallback(Callback)

Removes a callback that was added to every scene operation.

Declaration

```
public static void RemoveCallback(Callback callback)
```

Parameters

TYPE	NAME	DESCRIPTION
Callback	callback	

Reopen(OpenSceneInfo[])

Declaration

```
public SceneOperation Reopen(params OpenSceneInfo[] scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo[]	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

Reopen(Scene[])

Declaration

```
public SceneOperation Reopen(params Scene[] scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene[]	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

Reopen(IEnumerable<OpenSceneInfo>)

Declaration

```
public SceneOperation Reopen(IEnumerable<OpenSceneInfo> scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

Reopen(IEnumerable<Scene>)

Declaration

```
public SceneOperation Reopen(IEnumerable<Scene> scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithAction(SceneAction[])

Declaration

```
public SceneOperation WithAction(params SceneAction[] actions)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneAction[]	actions	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithCallback(Callback)

Declaration

```
public SceneOperation WithCallback(Callback actions)
```

Parameters

TYPE	NAME	DESCRIPTION
Callback	actions	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithClearUnusedAssets(Boolean)

Declaration

```
public SceneOperation WithClearUnusedAssets(bool enable = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	enable	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithCollection(SceneCollection, Boolean)

Declaration

```
public SceneOperation WithCollection(SceneCollection collection, bool withCallbacks = true)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Boolean	withCallbacks	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithFriendlyText(String)

Declaration

```
public SceneOperation WithFriendlyText(string text)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	text	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithLoadingPriority(ThreadPriority)

Declaration

```
public SceneOperation WithLoadingPriority(ThreadPriority priority)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.ThreadPriority	priority	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithLoadingScreen(Scene)

Declaration

```
public SceneOperation WithLoadingScreen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithLoadingScreen(Boolean)

Declaration

```
public SceneOperation WithLoadingScreen(bool use)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	use	

Returns

TYPE	DESCRIPTION
SceneOperation	

WithLoadingScreenCallback(Action<LoadingScreen>)

Declaration

```
public SceneOperation WithLoadingScreenCallback(Action<LoadingScreen> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<LoadingScreen>	callback	

Returns

TYPE	DESCRIPTION
SceneOperation	

Explicit Interface Implementations

IQueueable.OnCancel()

Declaration

```
void IQueueable.OnCancel()
```

IQueueable.OnTurn(Action)

Declaration

```
void IQueueable.OnTurn(Action onComplete)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	onComplete	

Implements

IQueueable

Class SceneOperation<ReturnValue>

A scene operation is a queueable operation that can open or close scenes. See also: SceneAction.

Inheritance

System.Object
UnityEngine.CustomYieldInstruction
SceneOperation
SceneOperation<ReturnValue>

Implements

IQueueable

Inherited Members

SceneOperation.Add(SceneManagerBase, Boolean)
SceneOperation.Add<ReturnValue>(SceneManagerBase, Boolean)
SceneOperation.Add<ReturnValue>(SceneManagerBase, Func<SceneOperation, ReturnValue>, Boolean)
SceneOperation.IQueueable.OnTurn(Action)
SceneOperation.IQueueable.OnCancel()
SceneOperation.friendlyText
SceneOperation.open
SceneOperation.close
SceneOperation.reopen
SceneOperation.customActions
SceneOperation.callbacks
SceneOperation.collection
SceneOperation.loadingScreen
SceneOperation.useLoadingScreen
SceneOperation.clearUnusedAssets
SceneOperation.doCollectionCallbacks
SceneOperation.loadingScreenCallback
SceneOperation.loadingPriority
SceneOperation.closeBehavior
SceneOperation._extCallbacks
SceneOperation.AddCallback(Callback)
SceneOperation.RemoveCallback(Callback)
SceneOperation.keepWaiting
SceneOperation.phase
SceneOperation.sceneManager
SceneOperation.current
SceneOperation.cancelled
SceneOperation.openedLoadingScreen
SceneOperation.actions
SceneOperation.totalProgress
SceneOperation.Cancel(Action)
SceneOperation.isDone
SceneOperation.FindActions<T>()
SceneOperation.FindLastAction<T>()
SceneOperation.openedScenes

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneOperation<ReturnValue> : SceneOperation, IEnumerator, IQueueable
```

Type Parameters

NAME	DESCRIPTION
ReturnValue	

Remarks

See also: SceneOperation.

Properties

done

Gets a SceneOperation that has already completed.

Declaration

```
public static SceneOperation<ReturnValue> done { get; }
```

Property Value

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

value

The return value of this SceneOperation<ReturnValue>.

Declaration

```
public ReturnValue value { get; }
```

Property Value

TYPE	DESCRIPTION
ReturnValue	

Methods

AsPersistent(SceneCloseBehavior)

Declaration

```
public SceneOperation<ReturnValue> AsPersistent(SceneCloseBehavior closeBehavior =
SceneCloseBehavior.KeepOpenAlways)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCloseBehavior	closeBehavior	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Close(OpenSceneInfo[])

Declaration

```
public SceneOperation<ReturnValue> Close(params OpenSceneInfo[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo[]	scenes	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Close(Boolean, OpenSceneInfo[])

Declaration

```
public SceneOperation<ReturnValue> Close(bool force, params OpenSceneInfo[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	force	
OpenSceneInfo[]	scenes	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Close(IEnumerable<OpenSceneInfo>, Boolean)

Declaration

```
public SceneOperation<ReturnValue> Close(IEnumerable<OpenSceneInfo> scenes, bool force = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	scenes	
System.Boolean	force	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Open(Scene[])

Declaration

```
public SceneOperation<ReturnValue> Open(params Scene[] scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene[]	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Open(IEnumerable<Scene>, Boolean)

Declaration

```
public SceneOperation<ReturnValue> Open(IEnumerable<Scene> scene, bool force = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	scene	
System.Boolean	force	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Reopen(OpenSceneInfo[])

Declaration

```
public SceneOperation<ReturnValue> Reopen(params OpenSceneInfo[] scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo[]	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Reopen(Scene[])

Declaration

```
public SceneOperation<ReturnValue> Reopen(params Scene[] scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene[]	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Reopen(IEnumerable<OpenSceneInfo>)

Declaration

```
public SceneOperation<ReturnValue> Reopen(IEnumerable<OpenSceneInfo> scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Reopen(IEnumerable<Scene>)

Declaration

```
public SceneOperation<ReturnValue> Reopen(IEnumerable<Scene> scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Return(Func<SceneOperation<ReturnValue>, ReturnValue>)

Callback that is called when SceneOperation is done, that is meant to retrieve the return value of this operation.

Declaration

```
public SceneOperation<ReturnValue> Return(Func<SceneOperation<ReturnValue>, ReturnValue> action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<SceneOperation<ReturnValue>, ReturnValue>	action	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithAction(SceneAction[])

Declaration

```
public SceneOperation<ReturnValue> WithAction(params SceneAction[] actions)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneAction[]	actions	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithCallback(Callback)

Declaration

```
public SceneOperation<ReturnValue> WithCallback(Callback actions)
```

Parameters

TYPE	NAME	DESCRIPTION
Callback	actions	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithClearUnusedAssets(Boolean)

Declaration

```
public SceneOperation<ReturnValue> WithClearUnusedAssets(bool enable = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	enable	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithCollection(SceneCollection, Boolean)

Declaration

```
public SceneOperation<ReturnValue> WithCollection(SceneCollection collection, bool withCallbacks = true)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Boolean	withCallbacks	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithFriendlyText(String)

Declaration

```
public SceneOperation<ReturnValue> WithFriendlyText(string text)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	text	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithLoadingPriority(ThreadPriority)

Declaration

```
public SceneOperation<ReturnValue> WithLoadingPriority(ThreadPriority priority)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.ThreadPriority	priority	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithLoadingScreen(Scene)

Declaration

```
public SceneOperation<ReturnValue> WithLoadingScreen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithLoadingScreen(Boolean)

Declaration

```
public SceneOperation<ReturnValue> WithLoadingScreen(bool use)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	use	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

WithLoadingScreenCallback(Action<LoadingScreen>)

Declaration

```
public SceneOperation<ReturnValue> WithLoadingScreenCallback(Action<LoadingScreen> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<LoadingScreen>	callback	

Returns

TYPE	DESCRIPTION
SceneOperation<ReturnValue>	

Implements

IQueueable

Class StandaloneManager

The manager for stand-alone scenes.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
AdvancedSceneManager.Core.SceneManagerBase
StandaloneManager

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class StandaloneManager : SceneManagerBase, ISerializationCallbackReceiver
```

Remarks

Usage: standalone.

Properties

preloadedScene

Declaration

```
public PreloadedSceneHelper preloadedScene { get; }
```

Property Value

TYPE	DESCRIPTION
PreloadedSceneHelper	

Methods

OpenSingle(Scene)

Close existing scenes and open the specified one.

Declaration

```
public SceneOperation<OpenSceneInfo> OpenSingle(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

Remarks

This will close the current collection.

Preload(Scene, Boolean)

Preloads the scene.

Declaration

```
public SceneOperation<PreloadedSceneHelper> Preload(Scene scene, bool doCallbacks = false)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	doCallbacks	

Returns

TYPE	DESCRIPTION
SceneOperation<PreloadedSceneHelper>	

Remarks

Use FinishLoading() or AdvancedSceneManager.Core.SceneManagerBase.Open(AdvancedSceneManager.Models.Scene) to finish loading scene.

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class UtilitySceneManager

An utility scene manager that helps with actions that might relate to either collection or standalone managers.

Inheritance

System.Object
UtilitySceneManager

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public class UtilitySceneManager
```

Remarks

Usage: utility.

Properties

activeScene

Gets the currently open scene.

Declaration

```
public OpenSceneInfo activeScene { get; }
```

Property Value

TYPE	DESCRIPTION
OpenSceneInfo	

currentOperation

Gets the current active operation in the queue.

Declaration

```
public SceneOperation currentOperation { get; }
```

Property Value

TYPE	DESCRIPTION
SceneOperation	

dontDestroyOnLoad

Represents 'DontDestroyOnLoad' scene.

Declaration

```
public OpenSceneInfo dontDestroyOnLoad { get; }
```

Property Value

TYPE	DESCRIPTION
OpenSceneInfo	

isBusy

Declaration

```
public bool isBusy { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

openScenes

Gets all currently open scenes.

Declaration

```
public IEnumerable<OpenSceneInfo> openScenes { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	

queuedOperations

Gets the current scene operation queue.

Declaration

```
public IEnumerable<SceneOperation> queuedOperations { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<SceneOperation>	

runningOperations

The currently running scene operations.

Declaration

```
public IEnumerable<SceneOperation> runningOperations { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<SceneOperation>	

Methods

Close(OpenSceneInfo)

Closes a scene regardless of whatever it is associated with a collection, or is was opened as stand-alone.

Declaration

```
public SceneOperation Close(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

CloseAll(Boolean)

Closes all scenes.

Declaration

```
public SceneOperation CloseAll(bool closeLoadingScreens = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	closeLoadingScreens	

Returns

TYPE	DESCRIPTION
SceneOperation	

DoSceneCallback<T>(Func<T, IEnumerator>)

Performs a callback on the scripts on all open scenes.

Declaration

```
public IEnumerator DoSceneCallback<T>(Func<T, IEnumerator> action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<T, System.Collections.IEnumerator>	action	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Type Parameters

NAME	DESCRIPTION
T	

FindOpenScene(Scene)

Finds the first open instance of this Scene, if it is open.

Declaration

```
public OpenSceneInfo FindOpenScene(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

FindOpenScene(Scene)

Finds the OpenSceneInfo of this UnityEngine.SceneManagement.Scene.

Declaration

```
public OpenSceneInfo FindOpenScene(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

FindPreloadedScene(Scene)

Find first preloaded instance this scene.

Declaration

```
public OpenSceneInfo FindPreloadedScene(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

FindPreloadedScene(Scene)

Find first preloaded instance this scene.

Declaration

```
public OpenSceneInfo FindPreloadedScene(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

FindPreloadedScenes()

Finds all current preloaded scenes.

Declaration

```
public IEnumerable<(OpenSceneInfo scene, SceneManagerBase sceneManager)> FindPreloadedScenes()
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<System.ValueTuple<OpenSceneInfo, AdvancedSceneManager.Core.SceneManagerBase>>	

IsOpen(Scene)

Gets whatever the scene is open, either as part of a collection, or as stand-alone.

Declaration

```
public IsOpenReturnValue IsOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
IsOpenReturnValue	

IsOpen(Scene)

Gets whatever the scene is open, either as part of a collection, or as stand-alone.

Declaration

```
public IsOpenReturnValue IsOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
IsOpenReturnValue	

OpenOrReopen(Scene, SceneCollection)

Opens the scene if not open, otherwise it will be reopened.

Declaration

```
public SceneOperation<OpenSceneInfo> OpenOrReopen(Scene scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

RegisterOpenCallback<T>(T, Action, Action, Boolean)

Registers a callback for when a scene or collection has opened, or closed, the callback is removed once it has been called, unless persistent is true.

Declaration

```
public void RegisterOpenCallback<T>(T scene, Action onOpen = null, Action onClose = null, bool persistent = false)
    where T : Object, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
T	scene	
System.Action	onOpen	
System.Action	onClose	
System.Boolean	persistent	

Type Parameters

NAME	DESCRIPTION
T	

Reopen(OpenSceneInfo)

Reopen a scene regardless of whatever it is associated with a collection, or is was opened as stand-alone.

Declaration

```
public SceneOperation<OpenSceneInfo> Reopen(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

SetActive(Scene)

Sets a scene as the activate scene.

Declaration

```
public void SetActive(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

SetActive(Scene)

Sets a scene as the activate scene.

Declaration

```
public void SetActive(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Toggle(Scene, Nullable<Boolean>)

Toggles the scene open or closed, if the scene is part of the current collection, then the scene will be toggled within the collection, otherwise, it will be toggled as a stand-alone scene.

Declaration

```
public SceneOperation Toggle(Scene scene, bool? enabled = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Nullable<System.Boolean>	enabled	If null, the scene will be toggled on or off depending on whatever the scene is open or not. Pass a value to ensure that the scene either open or closed.

Returns

TYPE	DESCRIPTION
SceneOperation	

Toggle(Scene, Nullable<Boolean>)

Toggles the scene open or closed, if the scene is part of the current collection, then the scene will be toggled within the collection, otherwise, it will be toggled as a stand-alone scene.

Declaration

```
public SceneOperation Toggle(Scene scene, bool? enabled = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	
System.Nullable<System.Boolean>	enabled	If null, the scene will be toggled on or off depending on whatever the scene is open or not. Pass a value to ensure that the scene either open or closed.

Returns

TYPE	DESCRIPTION
SceneOperation	

UnregisterCallback<T>(T, Action, Action)

Unregisters a callback.

Declaration

```
public void UnregisterCallback<T>(T scene, Action onOpen = null, Action onClose = null)
    where T : Object, ISceneObject
```

Parameters

TYPE	NAME	DESCRIPTION
T	scene	
System.Action	onOpen	

TYPE	NAME	DESCRIPTION
System.Action	onClose	

Type Parameters

NAME	DESCRIPTION
T	

Events

ActiveSceneChanged

Occurs when the active scene changes.

Declaration

```
public event UtilitySceneManager.ActiveSceneChangedHandler ActiveSceneChanged
```

Event Type

TYPE	DESCRIPTION
UtilitySceneManager.ActiveSceneChangedHandler	

LoadingScreenClosed

Occurs when a loading screen has closed.

Declaration

```
public event Action<LoadingScreen> LoadingScreenClosed
```

Event Type

TYPE	DESCRIPTION
System.Action<LoadingScreen>	

LoadingScreenClosing

Occurs when a loading screen is about to close.

Declaration

```
public event Action<LoadingScreen> LoadingScreenClosing
```

Event Type

TYPE	DESCRIPTION
System.Action<LoadingScreen>	

LoadingScreenOpened

Occurs when a loading screen has opened.

Declaration

```
public event Action<LoadingScreen> LoadingScreenOpened
```

Event Type

TYPE	DESCRIPTION
System.Action<LoadingScreen>	

LoadingScreenOpening

Occurs when a loading screen is about to be opened.

Declaration

```
public event Action<LoadingScreen> LoadingScreenOpening
```

Event Type

TYPE	DESCRIPTION
System.Action<LoadingScreen>	

queueEmpty

Declaration

```
public event Action queueEmpty
```

Event Type

TYPE	DESCRIPTION
System.Action	

SceneClosed

Occurs when a scene is closed.

Declaration

```
public event Action<OpenSceneInfo, SceneManagerBase> SceneClosed
```

Event Type

TYPE	DESCRIPTION
System.Action<OpenSceneInfo, AdvancedSceneManager.Core.SceneManagerBase>	

SceneOpened

Occurs when a scene is opened.

Declaration

```
public event Action<OpenSceneInfo, SceneManagerBase> SceneOpened
```

Event Type

TYPE	DESCRIPTION
System.Action<OpenSceneInfo, AdvancedSceneManager.Core.SceneManagerBase>	

Delegate UtilitySceneManager.ActiveSceneChangedHandler

Namespace: AdvancedSceneManager.Core

Assembly: AdvancedSceneManager.dll

Syntax

```
public delegate void ActiveSceneChangedHandler(OpenSceneInfo previousScene, OpenSceneInfo activeScene);
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	previousScene	
OpenSceneInfo	activeScene	

Namespace AdvancedSceneManager.Core.Actions

Classes

AggregateAction

Runs one ore more actions.

CallbackAction

Runs a coroutine.

CloseAllUnityScenesAction

Closes all scenes, except DefaultSceneUtility, regardless of whatever they are tracked or not. This is used in StartupAction, where we cannot be sure scenes are tracked yet.

OpenAndRunCallbackAction<T>

Opens a scene and finds a script of the specified type, and runs a callback, scene is closed if not found.

OpenStartupCollections

Opens all collections and scenes that are set to open at startup.

OverridableAction<T>

An action that can have its functionality overriden, this is needed to provide addressables support. When overriding default ASM actions, use caution since this will obviously lead to undocumented behavior.

PlaySplashScreenAction

Plays splash screen.

QuickStartupAction

Performs startup sequence, but without splash screen and loading screens.

CloseAllUnityScenesAction.

OpenStartupCollections.

QuitAction

RunCallbackAndCloseAction<T>

Runs a callback and closes the scene.

SceneAction

The base class of all scene actions. The scene actions perform an specific action on a Scene when contained within a SceneOperation.

SceneCloseAction

Closes a scene. This is a proxy for running the following actions in order:

SceneCloseCallbackAction, call callbacks.

SceneUnloadAction, unload scenes.

SceneCloseCallbackAction

Calls all OnSceneClose() callbacks in scene.

SceneFinishLoadAction

Finish load of a scene that was loaded, but not activated. See SceneLoadAction.

SceneLoadAction

Loads a scene, but does not activate it. This is the same as preloading. See SceneFinishLoadAction.

SceneOpenAction

Opens a scene. This is a proxy for running the following actions in order:

SceneLoadAction, preload scene.

SceneFinishLoadAction, activate scene.

SceneOpenCallbackAction, call callbacks.

SceneOpenCallbackAction

Calls all OnSceneOpen() callbacks in scene.

SceneUnloadAction

Finish load of a scene that was loaded, but not activated. See SceneLoadAction.

StartupAction

Performs startup sequence, see Start(SceneCollection, Boolean, Boolean):

AdvancedSceneManager.Core.Actions.StartupAction.FadeOut(System.Single,System.Nullable{UnityEngine.Color}).

CloseAllUnityScenesAction.

PlaySplashScreenAction (+ fade in).

AdvancedSceneManager.Core.Actions.StartupAction.ShowStartupLoadingScreen.

OpenStartupCollections.

AdvancedSceneManager.Core.Actions.StartupAction.HideStartupLoadingScreen.

Class AggregateAction

Runs one ore more actions.

Inheritance

System.Object
SceneAction
AggregateAction
SceneCloseAction
SceneOpenAction
StartupAction

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class AggregateAction : SceneAction
```

Constructors

AggregateAction(SceneAction[])

Declaration

```
public AggregateAction(params SceneAction[] actions)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneAction[]	actions	

AggregateAction(Action, SceneAction[])

Declaration

```
public AggregateAction(Action onDone, params SceneAction[] actions)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	onDone	Make sure to set properties such as openScene here, if needed.

TYPE	NAME	DESCRIPTION
SceneAction[]	actions	

Properties

actions

Declaration

```
public SceneAction[] actions { get; }
```

Property Value

TYPE	DESCRIPTION
SceneAction[]	

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

OnDone()

Declaration

```
protected virtual void OnDone()
```

Class CallbackAction

Runs a coroutine.

Inheritance

System.Object
SceneAction
CallbackAction

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class CallbackAction : SceneAction
```

Constructors

CallbackAction(Action)

Declaration

```
public CallbackAction(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

CallbackAction(Func<IEnumerator>)

Declaration

```
public CallbackAction(Func<IEnumerator> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<System.Collections.IEnumerator>	callback	

Properties

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Operators

Implicit(Action to CallbackAction)

Declaration

```
public static implicit operator CallbackAction(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

Returns

TYPE	DESCRIPTION
CallbackAction	

Implicit(Func<IEnumerator> to CallbackAction)

Declaration

```
public static implicit operator CallbackAction(Func<IEnumerator> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<System.Collections.IEnumerator>	callback	

Returns

TYPE	DESCRIPTION
CallbackAction	

Class CloseAllUnityScenesAction

Closes all scenes, except DefaultSceneUtility, regardless of whatever they are tracked or not. This is used in StartupAction, where we cannot be sure scenes are tracked yet.

Inheritance

System.Object
SceneAction
CloseAllUnityScenesAction

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class CloseAllUnityScenesAction : SceneAction
```

Constructors

CloseAllUnityScenesAction()

Declaration

```
public CloseAllUnityScenesAction()
```

CloseAllUnityScenesAction(Func<Scene>)

Declaration

```
public CloseAllUnityScenesAction(Func<Scene> ignore)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<UnityEngine.SceneManagement.Scene>	ignore	

Properties

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Class OpenAndRunCallbackAction<T>

Opens a scene and finds a script of the specified type, and runs a callback, scene is closed if not found.

Inheritance

System.Object
SceneAction
OpenAndRunCallbackAction<T>

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class OpenAndRunCallbackAction<T> : SceneAction
```

Type Parameters

NAME	DESCRIPTION
T	

Constructors

OpenAndRunCallbackAction(Scene, Func<T, IEnumerator>, Nullable<Single>, Boolean, Action)

Declaration

```
public OpenAndRunCallbackAction(Scene scene, Func<T, IEnumerator> runCallback, float? timeout = null, bool
isLoadingScreen = false, Action onMissingCallback = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Func<T, System.Collections.IEnumerator>	runCallback	
System.Nullable<System.Single>	timeout	
System.Boolean	isLoadingScreen	
System.Action	onMissingCallback	

Fields

defaultTimeout

Declaration

```
public const float defaultTimeout = 1F
```

Field Value

TYPE	DESCRIPTION
System.Single	

Properties

callback

Declaration

```
public T callback { get; }
```

Property Value

TYPE	DESCRIPTION
T	

isLoadingScreen

Declaration

```
public bool isLoadingScreen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

onMissingCallback

Declaration

```
public Action onMissingCallback { get; }
```

Property Value

TYPE	DESCRIPTION
System.Action	

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

runCallback

Declaration

```
public Func<T, IEnumerator> runCallback { get; }
```

Property Value

TYPE	DESCRIPTION
System.Func<T, System.Collections.IEnumerator>	

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Class OpenStartupCollections

Opens all collections and scenes that are set to open at startup.

Inheritance

System.Object
SceneAction
OpenStartupCollections

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class OpenStartupCollections : SceneAction
```

Constructors

OpenStartupCollections(SceneCollection, Boolean)

Declaration

```
public OpenStartupCollections(SceneCollection collection = null, bool forceOpenAllScenes = false)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	Opens the specified collection after all other collections and scenes has been opened.
System.Boolean	forceOpenAllScenes	

Properties

pointlessCollectionWarning

Enables warnings during startup when a collection is opened, then immedietly closed, because another collection was opened, and did not contain any persistent scenes.

Declaration

```
public static bool pointlessCollectionWarning { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Remarks

Only available in editor.

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Class OverridableAction<T>

An action that can have its functionality overriden, this is needed to provide addressables support. When overriding default ASM actions, use caution since this will obviously lead to undocumented behavior.

Inheritance

System.Object
SceneAction
OverridableAction<T>
SceneCloseCallbackAction
SceneFinishLoadAction
SceneLoadAction
SceneOpenCallbackAction
SceneUnloadAction

Inherited Members

SceneAction.reportsProgress
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public abstract class OverridableAction<T> : SceneAction where T : OverridableAction<T>
```

Type Parameters

NAME	DESCRIPTION
T	

Properties

lazyScene

Declaration

```
protected Func<OpenSceneInfo> lazyScene { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Func<OpenSceneInfo>	

overrides

All overriden scenes.

Declaration

```
public static ReadOnlyDictionary<string, Func<SceneManagerBase, T, IEnumerator>> overrides { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyDictionary<System.String, System.Func<AdvancedSceneManager.Core.SceneManagerBase, T, System.Collections.IEnumerator>>	

Methods

BeforeDoAction(out Boolean)

Declaration

```
protected virtual void BeforeDoAction(out bool exitOutEarly)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	exitOutEarly	

ClearOverride(String)

Clears overrides for the specified scene.

Declaration

```
public static void ClearOverride(string scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	

ClearOverrides()

Clears all overrides.

Declaration

```
public static void ClearOverrides()
```

DoAction(SceneManagerBase)

This must not be overriden since OverridableAction<T> does not work otherwise

Override DoNonOverridenAction(SceneManagerBase) instead.

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

DoNonOverridenAction(SceneManagerBase)

Declaration

```
public abstract IEnumerator DoNonOverridenAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Override(String, Func<SceneManagerBase, T, IEnumerator>)

Overrides behavior for the specified scene.

Declaration

```
public static void Override(string scene, Func<SceneManagerBase, T, IEnumerator> coroutine)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.Func<AdvancedSceneManager.Core.SceneManagerBase, T, System.Collections.IEnumerator>	coroutine	

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

SceneAction.ToString()

Class PlaySplashScreenAction

Plays splash screen.

Inheritance

System.Object
SceneAction
PlaySplashScreenAction

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PlaySplashScreenAction : SceneAction
```

Constructors

PlaySplashScreenAction(Func<IEnumerator>)

Declaration

```
public PlaySplashScreenAction(Func<IEnumerator> hideInitialLoadingScreen)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<System.Collections.IEnumerator>	hideInitialLoadingScreen	

Properties

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Class QuickStartupAction

Performs startup sequence, but without splash screen and loading screens.

CloseAllUnityScenesAction.

OpenStartupCollections.

Inheritance

System.Object
SceneAction
AggregateAction
StartupAction
QuickStartupAction

Inherited Members

AggregateAction.reportsProgress
AggregateAction.actions
AggregateAction.DoAction(SceneManagerBase)
AggregateAction.OnDone()
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class QuickStartupAction : StartupAction
```

Constructors

QuickStartupAction()

Declaration

```
public QuickStartupAction()
```

Class QuitAction

Inheritance

System.Object
SceneAction
QuitAction

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class QuitAction : SceneAction
```

Constructors

QuitAction(Boolean, Nullable<Color>, Single, Boolean, Boolean)

Declaration

```
public QuitAction(bool fade, Color? fadeColor = null, float fadeDuration = 1F, bool callSceneCloseCallbacks =
true, bool callCollectionCloseCallbacks = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	fade	
System.Nullable<UnityEngine.Color>	fadeColor	
System.Single	fadeDuration	
System.Boolean	callSceneCloseCallbacks	
System.Boolean	callCollectionCloseCallbacks	

Properties

isQuitting

Gets whatever ASM is currently in the process of quitting.

Declaration

```
public static bool isQuitting { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

Methods

CancelQuit()

Cancels a quit in progress.

Only usable during a RegisterQuitCallback() or while isQuitting is true.

Declaration

```
public static void CancelQuit()
```

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Class RunCallbackAndCloseAction<T>

Runs a callback and closes the scene.

Inheritance

System.Object
SceneAction
RunCallbackAndCloseAction<T>

Inherited Members

SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class RunCallbackAndCloseAction<T> : SceneAction where T : MonoBehaviour
```

Type Parameters

NAME	DESCRIPTION
T	

Constructors

RunCallbackAndCloseAction(T, Func<T, IEnumerator>, Boolean, Boolean)

Declaration

```
public RunCallbackAndCloseAction(T callback, Func<T, IEnumerator> runCallback, bool closeScene = true, bool
isLoadingScreen = false)
```

Parameters

TYPE	NAME	DESCRIPTION
T	callback	
System.Func<T, System.Collections.IEnumerator>	runCallback	
System.Boolean	closeScene	
System.Boolean	isLoadingScreen	

Properties

callback

Declaration

```
public T callback { get; }
```

Property Value

TYPE	DESCRIPTION
T	

closeScene

Declaration

```
public bool closeScene { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isLoadingScreen

Declaration

```
public bool isLoadingScreen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

reportsProgress

Declaration

```
public override bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

SceneAction.reportsProgress

runCallback

Declaration

```
public Func<T, IEnumerator> runCallback { get; }
```

Property Value

TYPE	DESCRIPTION

TYPE	DESCRIPTION
System.Func<T, System.Collections.IEnumerator>	

Methods

DoAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SceneAction.DoAction(SceneManagerBase)

Class SceneAction

The base class of all scene actions. The scene actions perform an specific action on a Scene when contained within a SceneOperation.

Inheritance

System.Object
SceneAction
AggregateAction
CallbackAction
CloseAllUnityScenesAction
OpenAndRunCallbackAction<T>
OpenStartupCollections
OverridableAction<T>
PlaySplashScreenAction
QuitAction
RunCallbackAndCloseAction<T>

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public abstract class SceneAction
```

Properties

collection

The collection that is being opened. null if stand-alone.

Declaration

```
public SceneCollection collection { get; protected set; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

isDone

Is this scene action done?

Declaration

```
public bool isDone { get; protected set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

openScene

Declaration

```
public OpenSceneInfo openScene { get; set; }
```

Property Value

TYPE	DESCRIPTION
OpenSceneInfo	

progress

The progress of this scene action.

Declaration

```
public float progress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

reportsProgress

Gets whatever this action reports progress.

Declaration

```
public virtual bool reportsProgress { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

scene

The scene this SceneAction is performing its action on.

Declaration

```
public Scene scene { get; protected set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

Methods

DoAction(SceneManagerBase)

Declaration

```
public abstract IEnumerator DoAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Done()

Declaration

```
protected virtual void Done()
```

Done(OpenSceneInfo)

Declaration

```
protected virtual void Done(OpenSceneInfo openScene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	openScene	

OnProgress(Single)

Declaration

```
protected void OnProgress(float progress)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Single	progress	

OnProgressCallback(Action<Single>)

Declaration

```
public void OnProgressCallback(Action<float> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<System.Single>	callback	

RegisterCallback(Action)

Register a callback when scene action is done.

Declaration

```
public void RegisterCallback(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

System.Object.ToString()

UnregisterCallback(Action)

Remove an registered callback when scene action is done.

Declaration

```
public void UnregisterCallback(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

Class SceneCloseAction

Closes a scene. This is a proxy for running the following actions in order:

SceneCloseCallbackAction, call callbacks.

SceneUnloadAction, unload scenes.

Inheritance

System.Object
SceneAction
AggregateAction
SceneCloseAction

Inherited Members

AggregateAction.reportsProgress
AggregateAction.actions
AggregateAction.DoAction(SceneManagerBase)
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneCloseAction : AggregateAction
```

Constructors

SceneCloseAction(OpenSceneInfo, SceneCollection)

Declaration

```
public SceneCloseAction(OpenSceneInfo scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
SceneCollection	collection	

Methods

OnDone()

Declaration

```
protected override void OnDone()
```

Overrides

AggregateAction.OnDone()

Class SceneCloseCallbackAction

Calls all OnSceneClose() callbacks in scene.

Inheritance

System.Object
SceneAction
OverridableAction<SceneCloseCallbackAction>
SceneCloseCallbackAction

Inherited Members

OverridableAction<SceneCloseCallbackAction>.overrides
OverridableAction<SceneCloseCallbackAction>.Override(String, Func<SceneManagerBase, SceneCloseCallbackAction, IEnumerator>)
OverridableAction<SceneCloseCallbackAction>.ClearOverride(String)
OverridableAction<SceneCloseCallbackAction>.ClearOverrides()
OverridableAction<SceneCloseCallbackAction>.DoAction(SceneManagerBase)
OverridableAction<SceneCloseCallbackAction>.lazyScene
OverridableAction<SceneCloseCallbackAction>.ToString()
SceneAction.reportsProgress
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneCloseCallbackAction : OverridableAction<SceneCloseCallbackAction>
```

Constructors

SceneCloseCallbackAction(OpenSceneInfo, SceneCollection)

Declaration

```
public SceneCloseCallbackAction(OpenSceneInfo scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
SceneCollection	collection	

SceneCloseCallbackAction(Func<OpenSceneInfo>, SceneCollection)

Declaration

```
public SceneCloseCallbackAction(Func<OpenSceneInfo> scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<OpenSceneInfo>	scene	
SceneCollection	collection	

Methods

BeforeDoAction(out Boolean)

Declaration

```
protected override void BeforeDoAction(out bool exitOutEarly)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	exitOutEarly	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneCloseCallbackAction>.BeforeDoAction(System.Boolean)

DoNonOverridenAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoNonOverridenAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneCloseCallbackAction>.DoNonOverridenAction(AdvancedSceneManager.Core.SceneManagerBase)

Class SceneFinishLoadAction

Finish load of a scene that was loaded, but not activated. See SceneLoadAction.

Inheritance

System.Object
SceneAction
OverridableAction<SceneFinishLoadAction>
SceneFinishLoadAction

Inherited Members

OverridableAction<SceneFinishLoadAction>.overrides
OverridableAction<SceneFinishLoadAction>.Override(String, Func<SceneManagerBase, SceneFinishLoadAction, IEnumerator>)
OverridableAction<SceneFinishLoadAction>.ClearOverride(String)
OverridableAction<SceneFinishLoadAction>.ClearOverrides()
OverridableAction<SceneFinishLoadAction>.DoAction(SceneManagerBase)
OverridableAction<SceneFinishLoadAction>.lazyScene
OverridableAction<SceneFinishLoadAction>.ToString()
SceneAction.reportsProgress
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneFinishLoadAction : OverridableAction<SceneFinishLoadAction>
```

Constructors

SceneFinishLoadAction(OpenSceneInfo, SceneCollection)

Declaration

```
public SceneFinishLoadAction(OpenSceneInfo scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
SceneCollection	collection	

SceneFinishLoadAction(Func<OpenSceneInfo>, SceneCollection)

Declaration

```
public SceneFinishLoadAction(Func<OpenSceneInfo> scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<OpenSceneInfo>	scene	
SceneCollection	collection	

Methods

BeforeDoAction(out Boolean)

Declaration

```
protected override void BeforeDoAction(out bool exitOutEarly)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	exitOutEarly	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneFinishLoadAction>.BeforeDoAction(System.Boolean)

DoNonOverridenAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoNonOverridenAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneFinishLoadAction>.DoNonOverridenAction(AdvancedSceneManager.Core.SceneManagerBase)

Class SceneLoadAction

Loads a scene, but does not activate it. This is the same as preloading. See SceneFinishLoadAction.

Inheritance

System.Object
SceneAction
OverridableAction<SceneLoadAction>
SceneLoadAction

Inherited Members

OverridableAction<SceneLoadAction>.overrides
OverridableAction<SceneLoadAction>.Override(String, Func<SceneManagerBase, SceneLoadAction, IEnumerator>)
OverridableAction<SceneLoadAction>.ClearOverride(String)
OverridableAction<SceneLoadAction>.ClearOverrides()
OverridableAction<SceneLoadAction>.DoAction(SceneManagerBase)
OverridableAction<SceneLoadAction>.lazyScene
OverridableAction<SceneLoadAction>.ToString()
SceneAction.reportsProgress
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneLoadAction : OverridableAction<SceneLoadAction>
```

Constructors

SceneLoadAction(Scene, SceneCollection)

Declaration

```
public SceneLoadAction(Scene scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
SceneCollection	collection	

Methods

AddScene(OpenSceneInfo, SceneManagerBase)

Declaration

```
public void AddScene(OpenSceneInfo scene, SceneManagerBase sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
AdvancedSceneManager.Core.SceneManagerBase	sceneManager	

BeforeDoAction(out Boolean)

Declaration

```
protected override void BeforeDoAction(out bool exitOutEarly)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	exitOutEarly	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneLoadAction>.BeforeDoAction(System.Boolean)

DoNonOverridenAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoNonOverridenAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneLoadAction>.DoNonOverridenAction(AdvancedSceneManager.Core.SceneManagerBase)

GetOpenSceneInfo(SceneManagerBase, AsyncOperation)

Declaration

```
protected OpenSceneInfo GetOpenSceneInfo(SceneManagerBase _sceneManager, AsyncOperation asyncOperation)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	
UnityEngine.AsyncOperation	asyncOperation	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

SetPersistentFlag(OpenSceneInfo)

Declaration

```
public void SetPersistentFlag(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Class SceneOpenAction

Opens a scene. This is a proxy for running the following actions in order:

SceneLoadAction, preload scene.

SceneFinishLoadAction, activate scene.

SceneOpenCallbackAction, call callbacks.

Inheritance

System.Object
SceneAction
AggregateAction
SceneOpenAction

Inherited Members

AggregateAction.reportsProgress
AggregateAction.actions
AggregateAction.DoAction(SceneManagerBase)
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneOpenAction : AggregateAction
```

Constructors

SceneOpenAction(Scene, SceneCollection)

Declaration

```
public SceneOpenAction(Scene scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
SceneCollection	collection	

Methods

OnDone()

Declaration

```
protected override void OnDone()
```

Overrides

AggregateAction.OnDone()

Class SceneOpenCallbackAction

Calls all OnSceneOpen() callbacks in scene.

Inheritance

System.Object
SceneAction
OverridableAction<SceneOpenCallbackAction>
SceneOpenCallbackAction

Inherited Members

OverridableAction<SceneOpenCallbackAction>.overrides
OverridableAction<SceneOpenCallbackAction>.Override(String, Func<SceneManagerBase, SceneOpenCallbackAction, IEnumerator>)
OverridableAction<SceneOpenCallbackAction>.ClearOverride(String)
OverridableAction<SceneOpenCallbackAction>.ClearOverrides()
OverridableAction<SceneOpenCallbackAction>.DoAction(SceneManagerBase)
OverridableAction<SceneOpenCallbackAction>.lazyScene
OverridableAction<SceneOpenCallbackAction>.ToString()
SceneAction.reportsProgress
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneOpenCallbackAction : OverridableAction<SceneOpenCallbackAction>
```

Constructors

SceneOpenCallbackAction(OpenSceneInfo, SceneCollection)

Declaration

```
public SceneOpenCallbackAction(OpenSceneInfo scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
SceneCollection	collection	

SceneOpenCallbackAction(Func<OpenSceneInfo>, SceneCollection)

Declaration

```
public SceneOpenCallbackAction(Func<OpenSceneInfo> scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<OpenSceneInfo>	scene	
SceneCollection	collection	

Methods

BeforeDoAction(out Boolean)

Declaration

```
protected override void BeforeDoAction(out bool exitOutEarly)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	exitOutEarly	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneOpenCallbackAction>.BeforeDoAction(System.Boolean)

DoNonOverridenAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoNonOverridenAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneOpenCallbackAction>.DoNonOverridenAction(AdvancedSceneManager.Core.SceneManagerBase)

Class SceneUnloadAction

Finish load of a scene that was loaded, but not activated. See SceneLoadAction.

Inheritance

System.Object
SceneAction
OverridableAction<SceneUnloadAction>
SceneUnloadAction

Inherited Members

OverridableAction<SceneUnloadAction>.overrides
OverridableAction<SceneUnloadAction>.Override(String, Func<SceneManagerBase, SceneUnloadAction, IEnumerator>)
OverridableAction<SceneUnloadAction>.ClearOverride(String)
OverridableAction<SceneUnloadAction>.ClearOverrides()
OverridableAction<SceneUnloadAction>.DoAction(SceneManagerBase)
OverridableAction<SceneUnloadAction>.lazyScene
OverridableAction<SceneUnloadAction>.ToString()
SceneAction.reportsProgress
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneUnloadAction : OverridableAction<SceneUnloadAction>
```

Constructors

SceneUnloadAction(OpenSceneInfo, SceneCollection)

Declaration

```
public SceneUnloadAction(OpenSceneInfo scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
SceneCollection	collection	

SceneUnloadAction(Func<OpenSceneInfo>, SceneCollection)

Declaration

```
public SceneUnloadAction(Func<OpenSceneInfo> scene, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<OpenSceneInfo>	scene	
SceneCollection	collection	

Methods

BeforeDoAction(out Boolean)

Declaration

```
protected override void BeforeDoAction(out bool exitOutEarly)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	exitOutEarly	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneUnloadAction>.BeforeDoAction(System.Boolean)

DoNonOverridenAction(SceneManagerBase)

Declaration

```
public override IEnumerator DoNonOverridenAction(SceneManagerBase _sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
AdvancedSceneManager.Core.SceneManagerBase	_sceneManager	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

AdvancedSceneManager.Core.Actions.OverridableAction<AdvancedSceneManager.Core.Actions.SceneUnloadAction>.DoNonOverridenAction(AdvancedSceneManager.Core.SceneManagerBase)

Remove(OpenSceneInfo, SceneManagerBase)

Declaration

```
public void Remove(OpenSceneInfo scene, SceneManagerBase sceneManager)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
AdvancedSceneManager.Core.SceneManagerBase	sceneManager	

UnsetPersistentFlag(OpenSceneInfo)

Declaration

```
public void UnsetPersistentFlag(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Class StartupAction

Performs startup sequence, see Start(SceneCollection, Boolean, Boolean):

AdvancedSceneManager.Core.Actions.StartupAction.FadeOut(System.Single,System.Nullable{UnityEngine.Color}).

CloseAllUnityScenesAction.

PlaySplashScreenAction (+ fade in).

AdvancedSceneManager.Core.Actions.StartupAction.ShowStartupLoadingScreen.

OpenStartupCollections.

AdvancedSceneManager.Core.Actions.StartupAction.HideStartupLoadingScreen.

Inheritance

System.Object
SceneAction
AggregateAction
StartupAction
QuickStartupAction

Inherited Members

AggregateAction.reportsProgress
AggregateAction.actions
AggregateAction.DoAction(SceneManagerBase)
AggregateAction.OnDone()
SceneAction.openScene
SceneAction.scene
SceneAction.collection
SceneAction.progress
SceneAction.isDone
SceneAction.RegisterCallback(Action)
SceneAction.UnregisterCallback(Action)
SceneAction.OnProgressCallback(Action<Single>)
SceneAction.OnProgress(Single)
SceneAction.Done()
SceneAction.Done(OpenSceneInfo)
SceneAction.ToString()

Namespace: AdvancedSceneManager.Core.Actions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class StartupAction : AggregateAction
```

Constructors

StartupAction(Boolean, Nullable<Color>, Single, Single, SceneCollection, Boolean)

Declaration

```
public StartupAction(bool skipSplashScreen = false, Color? fadeColor = null, float initialFadeDuration = 0F,
float beforeSplashScreenFadeDuration = 0.5F, SceneCollection collection = null, bool forceOpenAllScenes =
false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	skipSplashScreen	
System.Nullable<UnityEngine.Color>	fadeColor	Defaults to unity splash screen color.
System.Single	initialFadeDuration	
System.Single	beforeSplashScreenFadeDuration	
SceneCollection	collection	
System.Boolean	forceOpenAllScenes	

Namespace AdvancedSceneManager.Defaults

Classes

DefaultLoadingScreen

A default loading screen script. Fades the screen out and in. Provides progress if slider is set.

DefaultSplashScreen

A default splash screen script. Fades splash screen in and out.

IconBounceLoadingScreen

A default loading screen script. Displays progress with a bouncing icon.

PressAnyButtonLoadingScreen

A default loading screen script. Requires the user to press any key before loading screen closes.

QuoteLoadingScreen

A default loading screen script. Displays quotes.

Quotes

A list of quotes for QuoteLoadingScreen.

Quotes.Quote

A quote.

VideoLoadingScreen

A default loading screen script. Displays a video.

Class DefaultLoadingScreen

A default loading screen script. Fades the screen out and in. Provides progress if slider is set.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
DefaultLoadingScreen

Inherited Members

LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
[AddComponentMenu("")]
public class DefaultLoadingScreen : LoadingScreen
```

Fields

color

Declaration

```
public Color color
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Color	

duration

Declaration

```
public float duration
```

Field Value

TYPE	DESCRIPTION
System.Single	

group

Declaration

```
public CanvasGroup group
```

Field Value

TYPE	DESCRIPTION
UnityEngine.CanvasGroup	

image

Declaration

```
public Image image
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Image	

slider

Declaration

```
public Slider slider
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Slider	

Methods

OnClose(SceneOperation)

Declaration

```
public override IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnClose(SceneOperation)

OnOpen(SceneOperation)

Declaration

```
public override IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnOpen(SceneOperation)

Extension Methods

SceneUtility.Scene(Component)

Class DefaultSplashScreen

A default splash screen script. Fades splash screen in and out.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
SplashScreen
DefaultSplashScreen

Inherited Members

SplashScreen.OnOpen(SceneOperation)
SplashScreen.OnClose(SceneOperation)
LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
[AddComponentMenu("")]
public class DefaultSplashScreen : SplashScreen
```

Fields

fadeDuration

Declaration

```
public float fadeDuration
```

Field Value

TYPE	DESCRIPTION
System.Single	

group

Declaration

```
public CanvasGroup group
```

Field Value

TYPE	DESCRIPTION
UnityEngine.CanvasGroup	

image

Declaration

```
public Image image
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Image	

waitDuration

Declaration

```
public float waitDuration
```

Field Value

TYPE	DESCRIPTION
System.Single	

Methods

DisplaySplashScreen()

Declaration

```
public override IEnumerator DisplaySplashScreen()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

SplashScreen.DisplaySplashScreen()

Extension Methods

SceneUtility.Scene(Component)

Class IconBounceLoadingScreen

A default loading screen script. Displays progress with a bouncing icon.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
IconBounceLoadingScreen

Inherited Members

LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
public class IconBounceLoadingScreen : LoadingScreen
```

Fields

BackgroundTransform

Declaration

```
public RectTransform BackgroundTransform
```

Field Value

TYPE	DESCRIPTION
UnityEngine.RectTransform	

duration

Declaration

```
public float duration
```

Field Value

TYPE	DESCRIPTION
System.Single	

IconStartRotationZ

Declaration

```
public float IconStartRotationZ
```

Field Value

TYPE	DESCRIPTION
System.Single	

IconStartSize

Declaration

```
public Vector2 IconStartSize
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Vector2	

IconTransform

Declaration

```
public RectTransform IconTransform
```

Field Value

TYPE	DESCRIPTION
UnityEngine.RectTransform	

Methods

OnClose(SceneOperation)

Declaration

```
public override IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnClose(SceneOperation)

OnOpen(SceneOperation)

Declaration

```
public override IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnOpen(SceneOperation)

Extension Methods

SceneUtility.Scene(Component)

Class PressAnyButtonLoadingScreen

A default loading screen script. Requires the user to press any key before loading screen closes.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
PressAnyButtonLoadingScreen

Inherited Members

LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PressAnyButtonLoadingScreen : LoadingScreen
```

Methods

OnClose(SceneOperation)

Declaration

```
public override IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnClose(SceneOperation)

OnOpen(SceneOperation)

Declaration

```
public override IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnOpen(SceneOperation)

Extension Methods

SceneUtility.Scene(Component)

Class QuoteLoadingScreen

A default loading screen script. Displays quotes.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
QuoteLoadingScreen

Inherited Members

LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
public class QuoteLoadingScreen : LoadingScreen
```

Fields

Background

Declaration

```
public Image Background
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Image	

fade

Declaration

```
public Image fade
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Image	

fadeDuration

Declaration

```
public float fadeDuration
```

Field Value

TYPE	DESCRIPTION
System.Single	

Name

Declaration

```
public Text Name
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Text	

Quote

Declaration

```
public Text Quote
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Text	

quotes

Declaration

```
public Quotes quotes
```

Field Value

TYPE	DESCRIPTION
Quotes	

Methods

OnClose(SceneOperation)

Declaration

```
public override IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnClose(SceneOperation)

OnOpen(SceneOperation)

Declaration

```
public override IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnOpen(SceneOperation)

Extension Methods

SceneUtility.Scene(Component)

Class Quotes

A list of quotes for QuoteLoadingScreen.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
Quotes

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Quotes : MonoBehaviour
```

Fields

quoteList

The list of quotes.

Declaration

```
public List<Quotes.Quote> quoteList
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<Quotes.Quote>	

Extension Methods

SceneUtility.Scene(Component)

Class Quotes.Quote

A quote.

Inheritance

System.Object

Quotes.Quote

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class Quote
```

Fields

name

The name of this quote.

Declaration

```
public string name
```

Field Value

TYPE	DESCRIPTION
System.String	

quote

The quote text itself.

Declaration

```
public string quote
```

Field Value

TYPE	DESCRIPTION
System.String	

Class VideoLoadingScreen

A default loading screen script. Displays a video.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
LoadingScreen
VideoLoadingScreen

Inherited Members

LoadingScreen.onDestroy
LoadingScreen.OnDestroy()
LoadingScreen.canvas
LoadingScreen.operation
LoadingScreen.OnScenePhaseChanged(SceneOperation, Phase, Phase)
LoadingScreen.OnCancel(SceneOperation)

Namespace: AdvancedSceneManager.Defaults

Assembly: AdvancedSceneManager.dll

Syntax

```
public class VideoLoadingScreen : LoadingScreen
```

Fields

defaultVideoClip

Declaration

```
public VideoClip defaultVideoClip
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Video.VideoClip	

fadeDuration

Declaration

```
public float fadeDuration
```

Field Value

TYPE	DESCRIPTION
System.Single	

group

Declaration

```
public CanvasGroup group
```

Field Value

TYPE	DESCRIPTION
UnityEngine.CanvasGroup	

videoClip

Declaration

```
public static VideoClip videoClip
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Video.VideoClip	

videoPlayer

Declaration

```
[Header("VideoClip is static, Apply it before loading")]
public VideoPlayer videoPlayer
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Video.VideoPlayer	

VideoRenderer

Declaration

```
public RawImage VideoRenderer
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.RawImage	

Methods

OnClose(SceneOperation)

Declaration

```
public override IEnumerator OnClose(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnClose(SceneOperation)

OnOpen(SceneOperation)

Declaration

```
public override IEnumerator OnOpen(SceneOperation operation)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneOperation	operation	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Overrides

LoadingScreen.OnOpen(SceneOperation)

Extension Methods

SceneUtility.Scene(Component)

Namespace AdvancedSceneManager.Editor

Classes

EditCollectionPopup

EditorWindow_UIElements<T>

GenericPopup

GenericPopup.Item

ObjectField

Object field with type property that can be set from uxml. Also has isReadOnly property to allow selecting or opening value, but not allow changing value.

ObjectField.UxmlFactory

ObjectField.UxmlTraits

OpenInEditorPopup

PickTagPopup

Popup

Popup<T>

SceneAssetEditor

SceneField

An ObjectField that only accepts Scene, with support for UnityEditor.SceneAsset drag drop.

Has support for labelFilter, which filters scenes based on label (i.e. to only show scenes from 'Collection1', for example, use 'ASM:Collection1').

showOpenButtons can be used to toggle open buttons.

When isReadOnly is true, showOpenButtons will still be interactable, but value cannot be changed.

SceneField.SceneChangedEvent

SceneField.UxmlFactory

SceneField.UxmlTraits

SceneManagerWindow

SceneManagerWindow.DragAndDropReorder

An class that manages drag and drop reorder.

SceneManagerWindow.DragAndDropReorder.DragElement

SceneManagerWindow.FooterItem

SceneManagerWindow.PostProcess

SceneOverviewWindow

ScenePropertyDrawer

ScenesTab

SettingsTab

TagsTab

Interfaces

IUIToolkitEditor

An interface used by Popup<T> to allow it to get rootVisualElement and position of the editor it is attached to.

Enums

SceneManagerWindow.Tab

Class EditCollectionPopup

Inheritance

System.Object
Popup
Popup<EditCollectionPopup>
EditCollectionPopup

Inherited Members

Popup<EditCollectionPopup>.Reopen()
Popup<EditCollectionPopup>.Open(VisualElement, IUIToolkitEditor, Boolean, Vector2, Boolean, Boolean)
Popup<EditCollectionPopup>.CloseWithoutAnimation(Boolean)
Popup<EditCollectionPopup>.Close()
Popup<EditCollectionPopup>.Close(Action)
Popup<EditCollectionPopup>.OnClose()
Popup<EditCollectionPopup>.overlay
Popup<EditCollectionPopup>.rootVisualElement
Popup<EditCollectionPopup>.isMainContentLoaded
Popup<EditCollectionPopup>.LoadContent(String, VisualElement)
Popup<EditCollectionPopup>.LoadContent(String, VisualElement, Boolean, Boolean, Boolean)
Popup<EditCollectionPopup>.ReloadContent()
Popup<EditCollectionPopup>.enableBorder
Popup<EditCollectionPopup>.hasBorder
Popup<EditCollectionPopup>.offset
Popup<EditCollectionPopup>.alignRight
Popup<EditCollectionPopup>.placementTarget
Popup<EditCollectionPopup>.parent
Popup<EditCollectionPopup>.SetPosition()
Popup.OnClosed
Popup.RaiseOnClosed()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class EditCollectionPopup : Popup<EditCollectionPopup>
```

Properties

path

Declaration

```
public override string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.EditCollectionPopup>.path

Methods

OnReopen(EditCollectionPopup)

Declaration

```
protected override void OnReopen(EditCollectionPopup newPopup)
```

Parameters

TYPE	NAME	DESCRIPTION
EditCollectionPopup	newPopup	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.EditCollectionPopup>.OnReopen(AdvancedSceneManager.Editor.EditCollectionPopup)

Refresh(SceneCollection, Action<String>, Action)

Declaration

```
public void Refresh(SceneCollection collection, Action<string> onTitlePreview = null, Action onStartChanged = null)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Action<System.String>	onTitlePreview	
System.Action	onStartChanged	

Class EditorWindow_UIElements<T>

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
EditorWindow_UIElements<T>
SceneManagerWindow
SceneOverviewWindow

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public abstract class EditorWindow_UIElements<T> : EditorWindow where T : EditorWindow_UIElements<T>
```

Type Parameters

NAME	DESCRIPTION
T	

Properties

autoReloadOnWindowFocus

Declaration

```
public virtual bool autoReloadOnWindowFocus { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isMainContentLoaded

Declaration

```
public bool isMainContentLoaded { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

path

Declaration

```
public abstract string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

title

Declaration

```
public GUIContent title { get; set; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.GUIContent	

window

Declaration

```
public static T window { get; }
```

Property Value

TYPE	DESCRIPTION
T	

Methods

Close()

Declaration

```
public static void Close()
```

DoShow()

Override this to change the way this window should be opened, by default UnityEditor.EditorWindow.Show() is used.

Declaration

```
protected virtual void DoShow()
```

LoadContent(String, VisualElement)

Loads the UnityEngine.UIElements.VisualTreeAsset and its associated UnityEngine.UIElements.StyleSheet at the same path.

Declaration

```
public void LoadContent(string path, VisualElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	
UnityEngine.UIElements.VisualElement	element	

LoadContent(String, VisualElement, Boolean, Boolean, Boolean)

Loads the UnityEngine.UIElements.VisualTreeAsset and its associated UnityEngine.UIElements.StyleSheet at the same path.

Declaration

```
public void LoadContent(string path, VisualElement element, bool loadTree = false, bool loadStyle = false,
bool clearChildren = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	
UnityEngine.UIElements.VisualElement	element	
System.Boolean	loadTree	
System.Boolean	loadStyle	
System.Boolean	clearChildren	

OnClose()

Declaration

```
public virtual void OnClose()
```

OnEnable()

Declaration

```
public virtual void OnEnable()
```

OnFocus()

Declaration

```
public virtual void OnFocus()
```

OnShow()

Declaration

```
protected virtual void OnShow()
```

Open()

Declaration

```
public static void Open()
```

ReloadContent()

Declaration

```
public void ReloadContent()
```

Reopen()

Declaration

```
public static void Reopen()
```

ShowSuspendMessage()

Declaration

```
protected void ShowSuspendMessage()
```

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class GenericPopup

Inheritance

System.Object
Popup
Popup<GenericPopup>
GenericPopup

Inherited Members

Popup<GenericPopup>.Reopen()
Popup<GenericPopup>.Open(VisualElement, IUIToolkitEditor, Boolean, Vector2, Boolean, Boolean)
Popup<GenericPopup>.CloseWithoutAnimation(Boolean)
Popup<GenericPopup>.Close()
Popup<GenericPopup>.Close(Action)
Popup<GenericPopup>.OnClose()
Popup<GenericPopup>.overlay
Popup<GenericPopup>.rootVisualElement
Popup<GenericPopup>.isMainContentLoaded
Popup<GenericPopup>.LoadContent(String, VisualElement)
Popup<GenericPopup>.LoadContent(String, VisualElement, Boolean, Boolean, Boolean)
Popup<GenericPopup>.ReloadContent()
Popup<GenericPopup>.enableBorder
Popup<GenericPopup>.hasBorder
Popup<GenericPopup>.offset
Popup<GenericPopup>.alignRight
Popup<GenericPopup>.placementTarget
Popup<GenericPopup>.parent
Popup<GenericPopup>.SetPosition()
Popup.OnClosed
Popup.RaiseOnClosed()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class GenericPopup : Popup<GenericPopup>
```

Properties

path

Declaration

```
public override string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.GenericPopup>.path

Separator

Represents an GenericPopup.Item separator. default keyword can also be used.

Declaration

```
public static GenericPopup.Item Separator { get; }
```

Property Value

TYPE	DESCRIPTION
GenericPopup.Item	

Methods

OnReopen(GenericPopup)

Declaration

```
protected override void OnReopen(GenericPopup newPopup)
```

Parameters

TYPE	NAME	DESCRIPTION
GenericPopup	newPopup	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.GenericPopup>.OnReopen(AdvancedSceneManager.Editor.GenericPopup)

Refresh(GenericPopup.Item[])

Declaration

```
public void Refresh(params GenericPopup.Item[] items)
```

Parameters

TYPE	NAME	DESCRIPTION
GenericPopup.Item[]	items	

Class GenericPopup.Item

Inheritance

System.Object

GenericPopup.Item

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class Item
```

Properties

isBold

Declaration

```
public bool isBold { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isCheckable

Declaration

```
public bool isCheckable { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isChecked

Declaration

```
public bool isChecked { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isEnabled

Declaration

```
public bool isEnabled { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isSeparator

Declaration

```
public bool isSeparator { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isVisible

Declaration

```
public bool isVisible { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

name

Declaration

```
public string name { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.String	

onClick

Declaration

```
public Action<bool> onClick { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Action<System.Boolean>	

Separator

Declaration

```
public static GenericPopup.Item Separator { get; }
```

Property Value

TYPE	DESCRIPTION
GenericPopup.Item	

Methods

AsCheckable()

Declaration

```
public GenericPopup.Item AsCheckable()
```

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

AsCheckable(Action<Boolean>)

Declaration

```
public GenericPopup.Item AsCheckable(Action<bool> onCheckedChanged)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<System.Boolean>	onCheckedChanged	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

Create(String)

Declaration

```
public static GenericPopup.Item Create(string name)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

Create(String, Action)

Declaration

```
public static GenericPopup.Item Create(string name, Action onClick)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
System.Action	onClick	

TYPE	NAME	DESCRIPTION

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

WhenClicked(Action)

Declaration

```
public GenericPopup.Item WhenClicked(Action action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

WithBoldState(Boolean)

Declaration

```
public GenericPopup.Item WithBoldState(bool isBold)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	isBold	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

WithCheckedStatus(Boolean)

Declaration

```
public GenericPopup.Item WithCheckedStatus(bool isChecked)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	isChecked	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

WithEnabledState(Boolean)

Declaration

```
public GenericPopup.Item WithEnabledState(bool isEnabled)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	isEnabled	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

WithVisibleState(Boolean)

Declaration

```
public GenericPopup.Item WithVisibleState(bool isVisible)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	isVisible	

Returns

TYPE	DESCRIPTION
GenericPopup.Item	

Interface IUIToolkitEditor

An interface used by Popup<T> to allow it to get rootVisualElement and position of the editor it is attached to.

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public interface IUIToolkitEditor
```

Properties

position

Declaration

```
Rect position { get; set; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.Rect	

rootVisualElement

Declaration

```
VisualElement rootVisualElement { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

Class ObjectField

Object field with type property that can be set from uxml. Also has isReadOnly property to allow selecting or opening value, but not allow changing value.

Inheritance

System.Object
UnityEngine.UIElements.CallbackEventHandler
UnityEngine.UIElements.Focusable
UnityEngine.UIElements.VisualElement
UnityEngine.UIElements.BindableElement
UnityEngine.UIElements.BaseField<UnityEngine.Object>
UnityEditor.UIElements.ObjectField
ObjectField
SceneField

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class ObjectField : ObjectField, IEventHandler, ITransform, ITransitionAnimations,
IExperimentalFeatures, IVisualElementScheduler, IResolvedStyle, IBindable, INotifyValueChanged<Object>
```

Constructors

ObjectField()

Declaration

```
public ObjectField()
```

Properties

isReadOnly

Declaration

```
public bool isReadOnly { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Extension Methods

VisualElementExtensions.SetEnabledExt<TElement>(TElement, Boolean)
VisualElementExtensions.Setup<TElement, T>(TElement, T, String, Action, Boolean, String)
VisualElementExtensions.FindAncestor(VisualElement, String, String, Action<VisualElement>)
VisualElementExtensions.GetRoot(VisualElement)

Class ObjectField.UxmlFactory

Inheritance

System.Object

UnityEngine.UIElements.UxmlFactory<ObjectField, ObjectField.UxmlTraits>

ObjectField.UxmlFactory

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class UxmlFactory : UxmlFactory<ObjectField, ObjectField.UxmlTraits>, IUxmlFactory
```

Class ObjectField.UxmlTraits

Inheritance

System.Object
UnityEngine.UIElements.UxmlTraits
UnityEngine.UIElements.VisualElement.UxmlTraits
ObjectField.UxmlTraits

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class UxmlTraits : VisualElement.UxmlTraits
```

Properties

uxmlChildElementsDescription

Declaration

```
public override IEnumerable<UxmlChildElementDescription> uxmlChildElementsDescription { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<UnityEngine.UIElements.UxmlChildElementDescription>	

Overrides

UnityEngine.UIElements.VisualElement.UxmlTraits.uxmlChildElementsDescription

Methods

Init(VisualElement, IUxmlAttributes, CreationContext)

Declaration

```
public override void Init(VisualElement ve, IUxmlAttributes bag, CreationContext cc)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	ve	
UnityEngine.UIElements.IUxmlAttributes	bag	
UnityEngine.UIElements.CreationContext	cc	

Overrides

UnityEngine.UIElements.VisualElement.UxmlTraits.Init(UnityEngine.UIElements.VisualElement, UnityEngine.UIElements.IUxmlAttributes, UnityEngine.UIElements.CreationContext)

Class OpenInEditorPopup

Inheritance

System.Object
Popup
Popup<OpenInEditorPopup>
OpenInEditorPopup

Inherited Members

Popup<OpenInEditorPopup>.Reopen()
Popup<OpenInEditorPopup>.Open(VisualElement, IUIToolkitEditor, Boolean, Vector2, Boolean, Boolean)
Popup<OpenInEditorPopup>.CloseWithoutAnimation(Boolean)
Popup<OpenInEditorPopup>.Close()
Popup<OpenInEditorPopup>.Close(Action)
Popup<OpenInEditorPopup>.OnClose()
Popup<OpenInEditorPopup>.overlay
Popup<OpenInEditorPopup>.rootVisualElement
Popup<OpenInEditorPopup>.isMainContentLoaded
Popup<OpenInEditorPopup>.LoadContent(String, VisualElement)
Popup<OpenInEditorPopup>.LoadContent(String, VisualElement, Boolean, Boolean, Boolean)
Popup<OpenInEditorPopup>.ReloadContent()
Popup<OpenInEditorPopup>.enableBorder
Popup<OpenInEditorPopup>.hasBorder
Popup<OpenInEditorPopup>.offset
Popup<OpenInEditorPopup>.alignRight
Popup<OpenInEditorPopup>.placementTarget
Popup<OpenInEditorPopup>.parent
Popup<OpenInEditorPopup>.SetPosition()
Popup.OnClosed
Popup.RaiseOnClosed()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class OpenInEditorPopup : Popup<OpenInEditorPopup>
```

Properties

height

Declaration

```
public static float height { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

path

Declaration

```
public override string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.OpenInEditorPopup>.path

Methods

OnReopen(OpenInEditorPopup)

Declaration

```
protected override void OnReopen(OpenInEditorPopup newPopup)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenInEditorPopup	newPopup	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.OpenInEditorPopup>.OnReopen(AdvancedSceneManager.Editor.OpenInEditorPopup)

Refresh(Scene)

Declaration

```
public OpenInEditorPopup Refresh(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
OpenInEditorPopup	

Class PickTagPopup

Inheritance

System.Object
Popup
Popup<PickTagPopup>
PickTagPopup

Inherited Members

Popup<PickTagPopup>.Reopen()
Popup<PickTagPopup>.Open(VisualElement, IUIToolkitEditor, Boolean, Vector2, Boolean, Boolean)
Popup<PickTagPopup>.CloseWithoutAnimation(Boolean)
Popup<PickTagPopup>.Close()
Popup<PickTagPopup>.Close(Action)
Popup<PickTagPopup>.OnClose()
Popup<PickTagPopup>.overlay
Popup<PickTagPopup>.rootVisualElement
Popup<PickTagPopup>.isMainContentLoaded
Popup<PickTagPopup>.LoadContent(String, VisualElement)
Popup<PickTagPopup>.LoadContent(String, VisualElement, Boolean, Boolean, Boolean)
Popup<PickTagPopup>.ReloadContent()
Popup<PickTagPopup>.hasBorder
Popup<PickTagPopup>.offset
Popup<PickTagPopup>.alignRight
Popup<PickTagPopup>.placementTarget
Popup<PickTagPopup>.parent
Popup<PickTagPopup>.SetPosition()
Popup.OnClosed
Popup.RaiseOnClosed()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class PickTagPopup : Popup<PickTagPopup>
```

Properties

enableBorder

Declaration

```
protected override bool enableBorder { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.PickTagPopup>.enableBorder

path

Declaration

```
public override string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.PickTagPopup>.path

Methods

OnReopen(PickTagPopup)

Declaration

```
protected override void OnReopen(PickTagPopup newPopup)
```

Parameters

TYPE	NAME	DESCRIPTION
PickTagPopup	newPopup	

Overrides

AdvancedSceneManager.Editor.Popup<AdvancedSceneManager.Editor.PickTagPopup>.OnReopen(AdvancedSceneManager.Editor.PickTagPopup)

Refresh(SceneTag, Action<SceneTag>)

Declaration

```
public void Refresh(SceneTag selected, Action<SceneTag> onSelected)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneTag	selected	
System.Action<SceneTag>	onSelected	

Class Popup

Inheritance

System.Object

Popup

Popup<T>

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public abstract class Popup
```

Methods

RaiseOnClosed()

Declaration

```
protected void RaiseOnClosed()
```

Reopen()

Declaration

```
public abstract void Reopen()
```

Events

OnClosed

Declaration

```
public static event Action OnClosed
```

Event Type

TYPE	DESCRIPTION
System.Action	

Class Popup<T>

Inheritance

System.Object
Popup
Popup<T>
EditCollectionPopup
GenericPopup
OpenInEditorPopup
PickTagPopup

Inherited Members

Popup.OnClosed
Popup.RaiseOnClosed()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public abstract class Popup<T> : Popup where T : Popup<T>, new()
```

Type Parameters

NAME	DESCRIPTION
T	

Fields

overlay

Declaration

```
public VisualElement overlay
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

rootVisualElement

Declaration

```
public VisualElement rootVisualElement
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

Properties

alignRight

Declaration

```
public bool alignRight { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

enableBorder

Declaration

```
protected virtual bool enableBorder { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

hasBorder

Declaration

```
public bool hasBorder { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isMainContentLoaded

Declaration

```
public bool isMainContentLoaded { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

offset

Declaration

```
public Vector2 offset { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.Vector2	

parent

Declaration

```
public IUIToolkitEditor parent { get; }
```

Property Value

TYPE	DESCRIPTION
IUIToolkitEditor	

path

Declaration

```
public abstract string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

placementTarget

Declaration

```
public VisualElement placementTarget { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

Methods

Close()

Declaration

```
public void Close()
```

Close(Action)

Declaration

```
public void Close(Action onAnimationComplete = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	onAnimationComplete	

CloseWithoutAnimation(Boolean)

Declaration

```
public void CloseWithoutAnimation(bool raiseOnClosed = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	raiseOnClosed	

LoadContent(String, VisualElement)

Loads the UnityEngine.UIElements.VisualTreeAsset and its associated UnityEngine.UIElements.StyleSheet at the same path.

Declaration

```
public void LoadContent(string path, VisualElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	
UnityEngine.UIElements.VisualElement	element	

LoadContent(String, VisualElement, Boolean, Boolean, Boolean)

Loads the UnityEngine.UIElements.VisualTreeAsset and its associated UnityEngine.UIElements.StyleSheet at the same path.

Declaration

```
public void LoadContent(string path, VisualElement element, bool loadTree = false, bool loadStyle = false,
bool clearChildren = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	
UnityEngine.UIElements.VisualElement	element	
System.Boolean	loadTree	
System.Boolean	loadStyle	
System.Boolean	clearChildren	

OnClose()

Declaration

```
protected virtual void OnClose()
```

OnReopen(T)

Declaration

```
protected abstract void OnReopen(T newPopup)
```

Parameters

TYPE	NAME	DESCRIPTION
T	newPopup	

Open(VisualElement, IUIToolkitEditor, Boolean, Vector2, Boolean, Boolean)

Declaration

```
public static T Open(VisualElement placementTarget, IUIToolkitEditor parent, bool alignRight = false, Vector2
offset = default(Vector2), bool stretchToPlacementTargetWidth = false, bool ignoreAnimation = false)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	placementTarget	
IUIToolkitEditor	parent	
System.Boolean	alignRight	
UnityEngine.Vector2	offset	
System.Boolean	stretchToPlacementTargetWidth	
System.Boolean	ignoreAnimation	

Returns

TYPE	DESCRIPTION
T	

ReloadContent()

Declaration

```
public void ReloadContent()
```

Reopen()

Declaration

```
public override void Reopen()
```

Overrides

Popup.Reopen()

SetPosition()

Declaration

```
protected void SetPosition()
```

Class SceneAssetEditor

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.Editor
SceneAssetEditor

Implements

IUIToolkitEditor

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
[CanEditMultipleObjects]
[CustomEditor(typeof(SceneAsset))]
public class SceneAssetEditor : Editor, IUIToolkitEditor
```

Properties

position

Declaration

```
public Rect position { get; set; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.Rect	

rootVisualElement

Declaration

```
public VisualElement rootVisualElement { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

Methods

CreateInspectorGUI()

Declaration

```
public override VisualElement CreateInspectorGUI()
```

Returns

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

Overrides

UnityEditor.Editor.CreateInspectorGUI()

OnHeaderGUI()

Declaration

```
protected override void OnHeaderGUI()
```

Overrides

UnityEditor.Editor.OnHeaderGUI()

Implements

IUIToolkitEditor

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class SceneField

An ObjectField that only accepts Scene, with support for UnityEditor.SceneAsset drag drop.

Has support for labelFilter, which filters scenes based on label (i.e. to only show scenes from 'Collection1', for example, use 'ASM:Collection1').

showOpenButtons can be used to toggle open buttons.

When isReadOnly is true, showOpenButtons will still be interactable, but value cannot be changed.

Inheritance

System.Object
UnityEngine.UIElements.CallbackEventHandler
UnityEngine.UIElements.Focusable
UnityEngine.UIElements.VisualElement
UnityEngine.UIElements.BindableElement
UnityEngine.UIElements.BaseField<UnityEngine.Object>
UnityEditor.UIElements.ObjectField
ObjectField
SceneField

Inherited Members

ObjectField.isReadOnly

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class SceneField : ObjectField, IEventHandler, ITransform, ITransitionAnimations,
IExperimentalFeatures, IVisualElementScheduler, IResolvedStyle, IBindable, INotifyValueChanged<Object>
```

Constructors

SceneField()

Declaration

```
public SceneField()
```

Fields

onValueChanged

Declaration

```
public static EventHandler<(Scene prevScene, Scene newScene)> onValueChanged
```

Field Value

TYPE	DESCRIPTION
System.EventHandler<System.ValueTuple<Scene, Scene>>	

Properties

Collection

This will be passed to Scene.Open().

Declaration

```
public SceneCollection Collection { get; set; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

defaultName

Declaration

```
public string defaultName { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.String	

labelFilter

Declaration

```
public string labelFilter { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.String	

showOpenButtons

Declaration

```
public bool showOpenButtons { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

value

Declaration

```
public Scene value { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

Methods

OpenScene(Scene, Boolean, SceneCollection)

Declaration

```
public static void OpenScene(Scene scene, bool additive, SceneCollection collection = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	additive	
SceneCollection	collection	

RegisterValueChangedCallback(EventCallback<ChangeEvent<Scene>>)

Declaration

```
public void RegisterValueChangedCallback(EventCallback<ChangeEvent<Scene>> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.EventCallback<UnityEngine.UIElements.ChangeEvent<Scene>>	callback	

SetValueWithoutNotify(Scene)

Declaration

```
public SceneField SetValueWithoutNotify(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
SceneField	

Events

OnSceneOpen

Declaration

```
public event Action OnSceneOpen
```

Event Type

TYPE	DESCRIPTION
System.Action	

OnSceneOpenAdditive

Declaration

```
public event Action OnSceneOpenAdditive
```

Event Type

TYPE	DESCRIPTION
System.Action	

Extension Methods

VisualElementExtensions.SetEnabledExt<TElement>(TElement, Boolean)
VisualElementExtensions.Setup<T>(SceneField, String, T, String, Action, Boolean, String)
VisualElementExtensions.Setup<TElement, T>(TElement, T, String, Action, Boolean, String)
VisualElementExtensions.FindAncestor(VisualElement, String, String, Action<VisualElement>)
VisualElementExtensions.GetRoot(VisualElement)

Class SceneField.SceneChangedEvent

Inheritance

System.Object
UnityEngine.UIElements.EventBase
UnityEngine.UIElements.EventBase<UnityEngine.UIElements.ChangeEvent<Scene>>
UnityEngine.UIElements.ChangeEvent<Scene>
SceneField.SceneChangedEvent

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class SceneChangedEvent : ChangeEvent<Scene>, IDisposable, IChangeEvent
```

Constructors

SceneChangedEvent(Scene, Scene)

Declaration

```
public SceneChangedEvent(Scene newValue, Scene oldValue)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	newValue	
Scene	oldValue	

Class SceneField.UxmlFactory

Inheritance

System.Object

UnityEngine.UIElements.UxmlFactory<SceneField, SceneField.UxmlTraits>

SceneField.UxmlFactory

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class UxmlFactory : UxmlFactory<SceneField, SceneField.UxmlTraits>, IUxmlFactory
```

Class SceneField.UxmlTraits

Inheritance

System.Object
UnityEngine.UIElements.UxmlTraits
UnityEngine.UIElements.VisualElement.UxmlTraits
SceneField.UxmlTraits

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class UxmlTraits : VisualElement.UxmlTraits
```

Properties

uxmlChildElementsDescription

Declaration

```
public override IEnumerable<UxmlChildElementDescription> uxmlChildElementsDescription { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<UnityEngine.UIElements.UxmlChildElementDescription>	

Overrides

UnityEngine.UIElements.VisualElement.UxmlTraits.uxmlChildElementsDescription

Methods

Init(VisualElement, IUxmlAttributes, CreationContext)

Declaration

```
public override void Init(VisualElement ve, IUxmlAttributes bag, CreationContext cc)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	ve	
UnityEngine.UIElements.IUxmlAttributes	bag	
UnityEngine.UIElements.CreationContext	cc	

Overrides

UnityEngine.UIElements.VisualElement.UxmlTraits.Init(UnityEngine.UIElements.VisualElement, UnityEngine.UIElements.IUxmlAttributes, UnityEngine.UIElements.CreationContext)

Class SceneManagerWindow

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
EditorWindow_UIElements<SceneManagerWindow>
SceneManagerWindow

Implements

IUIToolkitEditor

Inherited Members

EditorWindow_UIElements<SceneManagerWindow>.title
EditorWindow_UIElements<SceneManagerWindow>.window
EditorWindow_UIElements<SceneManagerWindow>.Open()
EditorWindow_UIElements<SceneManagerWindow>.Close()
EditorWindow_UIElements<SceneManagerWindow>.OnClose()
EditorWindow_UIElements<SceneManagerWindow>.DoShow()
EditorWindow_UIElements<SceneManagerWindow>.OnShow()
EditorWindow_UIElements<SceneManagerWindow>.Reopen()
EditorWindow_UIElements<SceneManagerWindow>.isMainContentLoaded
EditorWindow_UIElements<SceneManagerWindow>.ShowSuspendMessage()
EditorWindow_UIElements<SceneManagerWindow>.LoadContent(String, VisualElement)
EditorWindow_UIElements<SceneManagerWindow>.LoadContent(String, VisualElement, Boolean, Boolean, Boolean)
EditorWindow_UIElements<SceneManagerWindow>.ReloadContent()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
[InitializeOnLoad]
public class SceneManagerWindow : EditorWindow_UIElements<SceneManagerWindow>, IUIToolkitEditor
```

Fields

position

Declaration

```
public static Rect position
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Rect	

Properties

autoReloadOnWindowFocus

Declaration

```
public override bool autoReloadOnWindowFocus { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneManagerWindow>.autoReloadOnWindowFocus

deleteTempBuildMessage

Enables message when temp build is deleted.

Declaration

```
public static bool deleteTempBuildMessage { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

IsDarkMode

Declaration

```
public static bool IsDarkMode { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

path

Declaration

```
public override string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneManagerWindow>.path

Methods

IgnoreProfileChanged()

Declaration

```
public static void IgnoreProfileChanged()
```

OnEnable()

Declaration

```
public override void OnEnable()
```

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneManagerWindow>.OnEnable()

OnFocus()

Declaration

```
public override void OnFocus()
```

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneManagerWindow>.OnFocus()

ReopenTab()

Declaration

```
public static void ReopenTab()
```

Save(ScriptableObject, Boolean)

Declaration

```
public static void Save(ScriptableObject so = null, bool updateBuildSettings = true)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.ScriptableObject	so	
System.Boolean	updateBuildSettings	

Events

MouseUp

Declaration

```
public static event Action MouseUp
```

Event Type

TYPE	DESCRIPTION
System.Action	

OnGUIEvent

Declaration

```
public static event Action OnGUIEvent
```

Event Type

TYPE	DESCRIPTION
System.Action	

Implements

IUIToolkitEditor

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class SceneManagerWindow.DragAndDropReorder

An class that manages drag and drop reorder.

Inheritance

System.Object
SceneManagerWindow.DragAndDropReorder

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public static class DragAndDropReorder
```

Properties

currentDragElement

Declaration

```
public static SceneManagerWindow.DragAndDropReorder.DragElement currentDragElement { get; }
```

Property Value

TYPE	DESCRIPTION
SceneManagerWindow.DragAndDropReorder.DragElement	

newIndex

Declaration

```
public static int newIndex { get; }
```

Property Value

TYPE	DESCRIPTION
System.Int32	

offset

Declaration

```
public static float offset { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

rootVisualElement

Declaration

```
public static VisualElement rootVisualElement { get; set; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

Methods

RegisterList(VisualElement, String, String, String, String)

Declaration

```
public static void RegisterList(VisualElement list, string dragButtonName = null, string dragButtonClass =
null, string itemRootName = null, string itemRootClass = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	list	
System.String	dragButtonName	
System.String	dragButtonClass	
System.String	itemRootName	
System.String	itemRootClass	

UnregisterList(VisualElement)

Declaration

```
public static void UnregisterList(VisualElement list)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	list	

UnregisterListAll()

Declaration

```
public static void UnregisterListAll()
```

Events

OnDragCancel

Declaration

```
public static event Action<SceneManagerWindow.DragAndDropReorder.DragElement> OnDragCancel
```

Event Type

TYPE	DESCRIPTION
System.Action<SceneManagerWindow.DragAndDropReorder.DragElement>	

OnDragEnded

Declaration

```
public static event Action<SceneManagerWindow.DragAndDropReorder.DragElement, int> OnDragEnded
```

Event Type

TYPE	DESCRIPTION
System.Action<SceneManagerWindow.DragAndDropReorder.DragElement, System.Int32>	

OnDragStarted

Declaration

```
public static event Action<SceneManagerWindow.DragAndDropReorder.DragElement> OnDragStarted
```

Event Type

TYPE	DESCRIPTION
System.Action<SceneManagerWindow.DragAndDropReorder.DragElement>	

Class SceneManagerWindow.DragAndDropReorder.DragElement

Inheritance

System.Object

SceneManagerWindow.DragAndDropReorder.DragElement

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class DragElement
```

Fields

button

Declaration

```
public VisualElement button
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

index

Declaration

```
public int index
```

Field Value

TYPE	DESCRIPTION
System.Int32	

item

Declaration

```
public VisualElement item
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

itemRootClass

Declaration

```
public string itemRootClass
```

Field Value

TYPE	DESCRIPTION
System.String	

itemRootName

Declaration

```
public string itemRootName
```

Field Value

TYPE	DESCRIPTION
System.String	

list

Declaration

```
public VisualElement list
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

mouseDown

Declaration

```
public EventCallback<MouseDownEvent> mouseDown
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.EventCallback<UnityEngine.UIElements.MouseDownEvent>	

mouseMove

Declaration

```
public EventCallback<MouseMoveEvent> mouseMove
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.EventCallback<UnityEngine.UIElements.MouseMoveEvent>	

mouseUp

Declaration

```
public EventCallback<MouseUpEvent> mouseUp
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UIElements.EventCallback<UnityEngine.UIElements.MouseUpEvent>	

Class SceneManagerWindow.FooterItem

Inheritance

System.Object

SceneManagerWindow.FooterItem

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class FooterItem
```

Properties

element

Declaration

```
public VisualElement element { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

left

Declaration

```
public bool left { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

Button(String, Action, String, Action<Button>)

Declaration

```
public SceneManagerWindow.FooterItem Button(string text, Action click, string tooltip = "", Action<Button>
setup = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	text	
System.Action	click	
System.String	tooltip	
System.Action<UnityEngine.UIElements.Button>	setup	

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

Create()

Declaration

```
public static SceneManagerWindow.FooterItem Create()
```

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

Element<T>(String, Action<T>, String)

Declaration

```
public SceneManagerWindow.FooterItem Element<T>(string text = "", Action<T> setup = null, string tooltip = "")
    where T : VisualElement, new()
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	text	
System.Action<T>	setup	
System.String	tooltip	

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

Type Parameters

NAME	DESCRIPTION
T	

Hidden()

Declaration

```
public SceneManagerWindow.FooterItem Hidden()
```

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

OnLeft()

Declaration

```
public SceneManagerWindow.FooterItem OnLeft()
```

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

OnRight()

Declaration

```
public SceneManagerWindow.FooterItem OnRight()
```

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

Visible(Boolean)

Declaration

```
public SceneManagerWindow.FooterItem Visible(bool visible = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	visible	

Returns

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem	

Class SceneManagerWindow.PostProcess

Inheritance

System.Object

UnityEditor.AssetPostprocessor

SceneManagerWindow.PostProcess

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class PostProcess : AssetPostprocessor
```

Enum SceneManagerWindow.Tab

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public enum Tab
```

Fields

NAME	DESCRIPTION
Scenes	
Settings	
Tags	

Class SceneOverviewWindow

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
EditorWindow_UIElements<SceneOverviewWindow>
SceneOverviewWindow

Implements

IUIToolkitEditor

Inherited Members

EditorWindow_UIElements<SceneOverviewWindow>.title
EditorWindow_UIElements<SceneOverviewWindow>.window
EditorWindow_UIElements<SceneOverviewWindow>.Open()
EditorWindow_UIElements<SceneOverviewWindow>.Close()
EditorWindow_UIElements<SceneOverviewWindow>.OnClose()
EditorWindow_UIElements<SceneOverviewWindow>.DoShow()
EditorWindow_UIElements<SceneOverviewWindow>.OnShow()
EditorWindow_UIElements<SceneOverviewWindow>.Reopen()
EditorWindow_UIElements<SceneOverviewWindow>.isMainContentLoaded
EditorWindow_UIElements<SceneOverviewWindow>.ShowSuspendMessage()
EditorWindow_UIElements<SceneOverviewWindow>.LoadContent(String, VisualElement)
EditorWindow_UIElements<SceneOverviewWindow>.LoadContent(String, VisualElement, Boolean, Boolean, Boolean)
EditorWindow_UIElements<SceneOverviewWindow>.ReloadContent()
EditorWindow_UIElements<SceneOverviewWindow>.OnFocus()

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public class SceneOverviewWindow : EditorWindow_UIElements<SceneOverviewWindow>, IUIToolkitEditor
```

Properties

autoReloadOnWindowFocus

Declaration

```
public override bool autoReloadOnWindowFocus { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneOverviewWindow>.autoReloadOnWindowFocus

path

Declaration

```
public override string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneOverviewWindow>.path

Methods

OnEnable()

Declaration

```
public override void OnEnable()
```

Overrides

AdvancedSceneManager.Editor.EditorWindow_UIElements<AdvancedSceneManager.Editor.SceneOverviewWindow>.OnEnable()

Implements

IUIToolkitEditor

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class ScenePropertyDrawer

Inheritance

System.Object
UnityEditor.GUIDrawer
UnityEditor.PropertyDrawer
ScenePropertyDrawer

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
[CustomPropertyDrawer(typeof(Scene))]
public class ScenePropertyDrawer : PropertyDrawer
```

Methods

ObjectField<T>(Rect, T, Func<Object, (T obj, Boolean didConvert)>, Type[])

Makes a object field. You can assign objects either by drag and drop objects or by selecting a object using the Object Picker, allows other types to be dragged onto the field and be converted to the target object..

Declaration

```
public static T ObjectField<T>(Rect position, T obj, Func<Object, (T obj, bool didConvert)> convertAllowedType
= null, params Type[] extraAllowedTypes)
    where T : Object
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Rect	position	
T	obj	
System.Func<UnityEngine.Object, System.ValueTuple<T, System.Boolean>>	convertAllowedType	
Type[]	extraAllowedTypes	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

OnGUI(Rect, SerializedProperty, GUIContent)

Declaration

```
public override void OnGUI(Rect position, SerializedProperty property, GUIContent label)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Rect	position	
UnityEditor.SerializedProperty	property	
UnityEngine.GUIContent	label	

Overrides

UnityEditor.PropertyDrawer.OnGUI(UnityEngine.Rect, UnityEditor.SerializedProperty, UnityEngine.GUIContent)

Class ScenesTab

Inheritance

System.Object

ScenesTab

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public static class ScenesTab
```

Properties

FooterButtons

Declaration

```
public static SceneManagerWindow.FooterItem[] FooterButtons { get; }
```

Property Value

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem[]	

Methods

OnDisable()

Declaration

```
public static void OnDisable()
```

OnEnable(VisualElement)

Declaration

```
public static void OnEnable(VisualElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	element	

OnReorderEnd(SceneManagerWindow.DragAndDropReorder.DragElement, Int32)

Declaration

```
public static void OnReorderEnd(SceneManagerWindow.DragAndDropReorder.DragElement element, int newIndex)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneManagerWindow.DragAndDropReorder.DragElement	element	
System.Int32	newIndex	

OnReorderStart(SceneManagerWindow.DragAndDropReorder.DragElement)

Declaration

```
public static void OnReorderStart(SceneManagerWindow.DragAndDropReorder.DragElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneManagerWindow.DragAndDropReorder.DragElement	element	

Open((SceneCollection collection, Int32 i), Boolean)

Declaration

```
public static void Open(this (SceneCollection collection, int i) scene, bool additive)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<SceneCollection, System.Int32>	scene	
System.Boolean	additive	

Path((SceneCollection collection, Int32 i))

Declaration

```
public static string Path(this (SceneCollection collection, int i) scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<SceneCollection, System.Int32>	scene	

Returns

TYPE	DESCRIPTION
System.String	

Class SettingsTab

Inheritance

System.Object

SettingsTab

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public static class SettingsTab
```

Properties

Settings

Declaration

```
public static SettingsTab Settings { get; }
```

Property Value

TYPE	DESCRIPTION
SettingsTab	

Methods

InitializeProfile(VisualElement, Action)

Declaration

```
public static void InitializeProfile(VisualElement element, Action changedCallback)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	element	
System.Action	changedCallback	

OnEnable(VisualElement)

Declaration

```
public static void OnEnable(VisualElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	element	

Class TagsTab

Inheritance

System.Object

TagsTab

Namespace: AdvancedSceneManager.Editor

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public static class TagsTab
```

Properties

FooterButtons

Declaration

```
public static SceneManagerWindow.FooterItem[] FooterButtons { get; }
```

Property Value

TYPE	DESCRIPTION
SceneManagerWindow.FooterItem[]	

Methods

OnEnable(VisualElement)

Declaration

```
public static void OnEnable(VisualElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	element	

OnReorderEnd(SceneManagerWindow.DragAndDropReorder.DragElement, Int32)

Declaration

```
public static void OnReorderEnd(SceneManagerWindow.DragAndDropReorder.DragElement element, int newIndex)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneManagerWindow.DragAndDropReorder.DragElement	element	
System.Int32	newIndex	

Namespace AdvancedSceneManager.Editor.Utility

Classes

AssetDatabaseUtility

Provides methods to make using UnityEditor.AssetDatabase.AllowAutoRefresh() easier.

AssetsSavedUtility

Provides an event that is called when UnityEditor.AssetModificationProcessor.OnWillSaveAssets(string[] paths) is called.

BlacklistUtility

Provides methods for working with the blacklist.

BuildEventsUtility

An utility for registering to build events.

Only available in editor.

BuildSettingsUtility

An utility class for managing build settings scene order.

EditorFolderUtility

Contains functions for folders in editor.

EditorGUIUtilityExt

Contains proxy functions for internal UnityEditor.EditorGUIUtility functions that should have a public counterpart.

GenericPrompt

GenericPrompt<T, TSelf>

HierarchyGUIUtility

An utility for adding extra icons to scene fields in the heirarchy window. Only available in editor.

ListUtility

Provides methods for working with lists.

MenuItemHelper

An helper class for creating UnityEditor.MenuItem.

OnGUIPrompt

PersistentSceneInEditorUtility

An utility class to automatically open persistent scenes in editor.

PersistentSceneInEditorUtility.OpenInEditorSetting

PromptInt

PromptKey

PromptName

PromptNameAndMessage

PromptVersion

SettingsTab

Provides the ability to add settings to advanced scene manager window settings.

SettingsTab._DefaultHeaders

StyleExtensions

TrimUtility

Applies text-overflow uss property in 2020+, but applies manual trim in earlier versions.

VisualElementExtensions

Enums

BuildSettingsUtility.Reason

PersistentSceneInEditorUtility.OpenInEditorOption

Delegates

BuildEventsUtility.BuildError

HierarchyGUIUtility.HierarchyGameObjectGUI

Called after reserving a rect in hierarchy game object field. Return true to indicate that something was drawn, false means that the rect will be re-used for next OnGUI callback.

HierarchyGUIUtility.HierarchySceneGUI

Called after reserving a rect in hierarchy scene field. Return true to indicate that something was drawn, false means that the rect will be re-used for next OnGUI callback.

Class AssetDatabaseUtility

Provides methods to make using UnityEditor.AssetDatabase.AllowAutoRefresh() easier.

Inheritance

System.Object
AssetDatabaseUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class AssetDatabaseUtility
```

Methods

AllowAutoRefresh(Object)

Declaration

```
public static void AllowAutoRefresh(object key)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	key	

DisallowAutoRefresh(Object)

Declaration

```
public static void DisallowAutoRefresh(object key)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	key	

Class AssetsSavedUtility

Provides an event that is called when UnityEditor.AssetModificationProcessor.OnWillSaveAssets(string[] paths) is called.

Inheritance

System.Object
UnityEditor.AssetModificationProcessor
AssetsSavedUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class AssetsSavedUtility : AssetModificationProcessor
```

Remarks

Only available in editor.

Events

onAssetsSaved

Declaration

```
public static event Action<string[]> onAssetsSaved
```

Event Type

TYPE	DESCRIPTION
System.Action<System.String[]>	

onAssetsSaved2

Declaration

```
public static event Action onAssetsSaved2
```

Event Type

TYPE	DESCRIPTION
System.Action	

Class BlacklistUtility

Provides methods for working with the blacklist.

Inheritance

System.Object
BlacklistUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class BlacklistUtility
```

Methods

IsBlocked(String)

Gets if asset is blocked for importing.

Declaration

```
public static bool IsBlocked(string assetPath)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	assetPath	

Returns

TYPE	DESCRIPTION
System.Boolean	

Class BuildEventsUtility

An utility for registering to build events.

Only available in editor.

Inheritance

System.Object
BuildEventsUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class BuildEventsUtility
```

Events

onError

Occurs when an error occurs during build.

Declaration

```
public static event Action onError
```

Event Type

TYPE	DESCRIPTION
System.Action	

onErrorWithArgs

Occurs when an error occurs during build.

Declaration

```
public static event BuildEventsUtility.BuildError onErrorWithArgs
```

Event Type

TYPE	DESCRIPTION
BuildEventsUtility.BuildError	

onWarningWithArgs

Occurs when an error occurs during build.

Declaration

```
public static event BuildEventsUtility.BuildError onWarningWithArgs
```

Event Type

TYPE	DESCRIPTION
BuildEventsUtility.BuildError	

postBuild

Occurs after build.

Declaration

```
public static event Action<BuildReport> postBuild
```

Event Type

TYPE	DESCRIPTION
System.Action<UnityEditor.Build.Reporting.BuildReport>	

preBuild

Occurs before build.

Declaration

```
public static event Action<BuildReport> preBuild
```

Event Type

TYPE	DESCRIPTION
System.Action<UnityEditor.Build.Reporting.BuildReport>	

Remarks

Note that changes may not be included in build.

Delegate BuildEventsUtility.BuildError

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public delegate void BuildError(string condition, string stacktrace, LogType type);
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	condition	
System.String	stacktrace	
UnityEngine.LogType	type	

Class BuildSettingsUtility

An utility class for managing build settings scene order.

Inheritance

System.Object
BuildSettingsUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class BuildSettingsUtility
```

Remarks

Only available in editor.

Methods

GetEnabledState(String)

Gets the enabled state of a scene.

Declaration

```
public static (bool enabled, BuildSettingsUtility.Reason reason) GetEnabledState(string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Returns

TYPE	DESCRIPTION
System.ValueTuple<System.Boolean, BuildSettingsUtility.Reason>	

GetOrderedList()

Get an ordered list of all scenes that would be set as scene build settings.

Declaration

```
public static IEnumerable<(string scene, EditorBuildSettingsScene buildScene, BuildSettingsUtility.Reason
reason)> GetOrderedList()
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<System.ValueTuple<System.String, UnityEditor.EditorBuildSettingsScene, BuildSettingsUtility.Reason>>	

IsIncluded(Scene)

Get if scene is included in build.

Declaration

```
public static bool IsIncluded(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

UpdateBuildSettings()

Updates the scene build settings.

Declaration

```
public static void UpdateBuildSettings()
```

Enum BuildSettingsUtility.Reason

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum Reason
```

Fields

NAME	DESCRIPTION
IncludedInProfile	
IsAddressable	
None	
NotIncludedInProfile	
Overriden	

Class EditorFolderUtility

Contains functions for folders in editor.

Inheritance

System.Object
EditorFolderUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class EditorFolderUtility
```

Methods

EnsureFolderExists(String)

Declaration

```
public static void EnsureFolderExists(string folder)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	folder	

Class EditorGUIUtilityExt

Contains proxy functions for internal UnityEditor.EditorGUIUtility functions that should have a public counterpart.

Inheritance

System.Object
EditorGUIUtilityExt

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class EditorGUIUtilityExt
```

Methods

GetDefaultBackgroundColor()

Declaration

```
public static Color GetDefaultBackgroundColor()
```

Returns

TYPE	DESCRIPTION
UnityEngine.Color	

PingOrOpenAsset(Object, Int32)

Declaration

```
public static void PingOrOpenAsset(Object targetObject, int clickCount)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Object	targetObject	
System.Int32	clickCount	

Class GenericPrompt

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<System.Object, GenericPrompt>
GenericPrompt

Inherited Members

GenericPrompt<Object, GenericPrompt>.validate
GenericPrompt<Object, GenericPrompt>.Validate(Object)
GenericPrompt<Object, GenericPrompt>.Prompt(Object, Func<Object, (T1, T2)<Boolean, String>>[])
GenericPrompt<Object, GenericPrompt>.OnContentGUI(Object)
GenericPrompt<Object, GenericPrompt>.title
GenericPrompt<Object, GenericPrompt>.updateSizeAutomatically
GenericPrompt<Object, GenericPrompt>.extraHeight
GenericPrompt<Object, GenericPrompt>.width
GenericPrompt<Object, GenericPrompt>.okButton
GenericPrompt<Object, GenericPrompt>.cancelButton

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class GenericPrompt : GenericPrompt<object, GenericPrompt>
```

Properties

cancelButton

Declaration

```
public override string cancelButton { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.Object, AdvancedSceneManager.Editor.Utility.GenericPrompt>.cancelButton

extraHeight

Declaration

```
public override float extraHeight { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.Object, AdvancedSceneManager.Editor.Utility.GenericPrompt>.extraHeight

okButton

Declaration

```
public override string okButton { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.Object, AdvancedSceneManager.Editor.Utility.GenericPrompt>.okButton

title

Declaration

```
public override string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.Object, AdvancedSceneManager.Editor.Utility.GenericPrompt>.title

width

Declaration

```
public override float width { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.Object, AdvancedSceneManager.Editor.Utility.GenericPrompt>.width

Methods

OnContentGUI(ref Object)

Declaration

```
public override void OnContentGUI(ref object value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	value	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.Object, AdvancedSceneManager.Editor.Utility.GenericPrompt>.OnContentGUI(System.Object)

Prompt(String, String, String, String, Single)

Declaration

```
public static bool Prompt(string title, string message, string OkButton = "Ok", string cancelButton =
"Cancel", float maxContentHeight = 400F)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	title	
System.String	message	
System.String	OkButton	
System.String	cancelButton	
System.Single	maxContentHeight	

Returns

TYPE	DESCRIPTION
System.Boolean	

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class GenericPrompt<T, TSelf>

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<T, TSelf>
GenericPrompt
PromptInt
PromptKey
PromptName
PromptNameAndMessage
PromptVersion

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public abstract class GenericPrompt<T, TSelf> : EditorWindow where TSelf : GenericPrompt<T, TSelf>
```

Type Parameters

NAME	DESCRIPTION
T	
TSelf	

Fields

validate

Declaration

```
protected readonly List<Func<T, (bool isValid, string message)>> validate
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<System.Func<T, System.ValueTuple<System.Boolean, System.String>>>	

Properties

cancelButton

Declaration

```
public virtual string cancelButton { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

extraHeight

Declaration

```
public virtual float extraHeight { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

okButton

Declaration

```
public virtual string okButton { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

title

Declaration

```
public abstract string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

updateSizeAutomatically

Declaration

```
public virtual bool updateSizeAutomatically { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

width

Declaration

```
public virtual float width { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

Methods

OnContentGUI(ref T)

Declaration

```
public abstract void OnContentGUI(ref T value)
```

Parameters

TYPE	NAME	DESCRIPTION
T	value	

Prompt(T, Func<T, (Boolean isValid, String message)>[])

Declaration

```
public static (T value, bool successful) Prompt(T defaultValue = null, params Func<T, (bool isValid, string message)>[] validate)
```

Parameters

TYPE	NAME	DESCRIPTION
T	defaultValue	
System.Func<T, System.ValueTuple<System.Boolean, System.String>>[]	validate	

Returns

TYPE	DESCRIPTION
System.ValueTuple<T, System.Boolean>	

Validate(T)

Declaration

```
protected virtual (bool isValid, string message) Validate(T value)
```

Parameters

TYPE	NAME	DESCRIPTION
T	value	

Returns

TYPE	DESCRIPTION
System.ValueTuple<System.Boolean, System.String>	

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class HierarchyGUIUtility

An utility for adding extra icons to scene fields in the heirarchy window. Only available in editor.

Inheritance

System.Object
HierarchyGUIUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class HierarchyGUIUtility
```

Properties

defaultStyle

The default style for text in hierarchy.

Declaration

```
public static GUIStyle defaultStyle { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.GUIStyle	

Methods

AddGameObjectGUI(HierarchyGUIUtility.HierarchyGameObjectGUI, Int32)

Adds a onGUI call for UnityEngine.GameObject fields.

Declaration

```
public static void AddGameObjectGUI(HierarchyGUIUtility.HierarchyGameObjectGUI onGUI, int index = 0)
```

Parameters

TYPE	NAME	DESCRIPTION
HierarchyGUIUtility.HierarchyGameObjectGUI	onGUI	
System.Int32	index	

AddSceneGUI(HierarchyGUIUtility.HierarchySceneGUI, Int32)

Adds a onGUI call for Scene fields.

Declaration

```
public static void AddSceneGUI(HierarchyGUIUtility.HierarchySceneGUI onGUI, int index = 0)
```

Parameters

TYPE	NAME	DESCRIPTION

TYPE	NAME	DESCRIPTION
HierarchyGUIUtility.HierarchySceneGUI	onGUI	
System.Int32	index	

RemoveGameObjectGUI(HierarchyGUIUtility.HierarchyGameObjectGUI)

Remove a onGUI call for a UnityEngine.GameObject.

Declaration

```
public static void RemoveGameObjectGUI(HierarchyGUIUtility.HierarchyGameObjectGUI onGUI)
```

Parameters

TYPE	NAME	DESCRIPTION
HierarchyGUIUtility.HierarchyGameObjectGUI	onGUI	

RemoveSceneGUI(HierarchyGUIUtility.HierarchySceneGUI)

Remove a onGUI call for a Scene.

Declaration

```
public static void RemoveSceneGUI(HierarchyGUIUtility.HierarchySceneGUI onGUI)
```

Parameters

TYPE	NAME	DESCRIPTION
HierarchyGUIUtility.HierarchySceneGUI	onGUI	

Repaint()

Declaration

```
public static void Repaint()
```

Delegate HierarchyGUIUtility.HierarchyGameObjectGUI

Called after reserving a rect in hierarchy game object field. Return true to indicate that something was drawn, false means that the rect will be re-used for next OnGUI callback.

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public delegate bool HierarchyGameObjectGUI(GameObject gameObject);
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	gameObject	

Returns

TYPE	DESCRIPTION
System.Boolean	

Delegate HierarchyGUIUtility.HierarchySceneGUI

Called after reserving a rect in hierarchy scene field. Return true to indicate that something was drawn, false means that the rect will be re-used for next OnGUI callback.

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public delegate bool HierarchySceneGUI(Scene scene);
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Class ListUtility

Provides methods for working with lists.

Inheritance

System.Object
ListUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class ListUtility
```

Remarks

Only available in editor.

Methods

Except<T>(IEnumerable<T>, T)

Excludes the items from the list.

Declaration

```
public static IEnumerable<T> Except<T>(this IEnumerable<T> list, T item)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	list	
T	item	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	

Type Parameters

NAME	DESCRIPTION
T	

Flatten<T>(IEnumerable<T>, Func<T, IEnumerable<T>>)

Flattens a multidimensional list.

Usage: list.Flatten(item => item.subItems);

Declaration

```
public static IEnumerable<T> Flatten<T>(this IEnumerable<T> list, Func<T, IEnumerable<T>> getSubList)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	list	
System.Func<T, System.Collections.Generic.IEnumerable<T>>	getSubList	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	

Type Parameters

NAME	DESCRIPTION
T	

ForEach<T>(T[], Action<T, Int32>)

Runs `action` on each item in `list`.

Declaration

```
public static void ForEach<T>(this T[] list, Action<T, int> action)
```

Parameters

TYPE	NAME	DESCRIPTION
T[]	list	
System.Action<T, System.Int32>	action	

Type Parameters

NAME	DESCRIPTION
T	

ForEach<T>(IEnumerable<T>, Action<T, Int32>)

Runs `action` on each item in `list`.

Declaration

```
public static void ForEach<T>(this IEnumerable<T> list, Action<T, int> action)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	list	
System.Action<T, System.Int32>	action	

Type Parameters

NAME	DESCRIPTION
T	

GroupConsecutive<T>(IEnumerable<T>, Func<T, T, Boolean>)

Groups consecutive items together.

Declaration

```
public static IEnumerable<IEnumerable<T>> GroupConsecutive<T>(this IEnumerable<T> list, Func<T, T, bool>
compare)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	list	
System.Func<T, T, System.Boolean>	compare	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<System.Collections.Generic.IEnumerable<T>>	

Type Parameters

NAME	DESCRIPTION
T	

MoveDown<T>(ref T[], T)

Moves the `item` up in the `array`. Returns false if index is last or `item` does not exist in `array`.

Declaration

```
public static bool MoveDown<T>(ref T[] array, T item)
```

Parameters

TYPE	NAME	DESCRIPTION
T[]	array	
T	item	

Returns

TYPE	DESCRIPTION
System.Boolean	

Type Parameters

NAME	DESCRIPTION
T	

MoveUp<T>(ref T[], T)

Moves the `item` up in the `array`. Returns false if index is 0 or `item` does not exist in `array`.

Declaration

```
public static bool MoveUp<T>(ref T[] array, T item)
```

Parameters

TYPE	NAME	DESCRIPTION
T[]	array	
T	item	

Returns

TYPE	DESCRIPTION
System.Boolean	

Type Parameters

NAME	DESCRIPTION
T	

Class MenuItemHelper

An helper class for creating UnityEditor.MenuItem.

Inheritance

System.Object
MenuItemHelper

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class MenuItemHelper
```

Methods

InvokeThisMenuItem()

Invokes a menu item, only usable when called from a method decorated with UnityEditor.MenuItem that has been Setup() using MenuItemHelper.

Declaration

```
public static void InvokeThisMenuItem()
```

Refresh()

Declaration

```
[InitializeOnLoadMethod]
public static void Refresh()
```

Setup(MethodBase, Action)

Sets up an clickable menu item.

Declaration

```
public static void Setup(MethodBase method, Action onClick = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Reflection.MethodBase	method	
System.Action	onClick	

Setup(MethodBase, Func<Boolean>, Action<Boolean>)

Sets up an toggleable menu item.

Declaration

```
public static void Setup(MethodBase method, Func<bool> get, Action<bool> set = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Reflection.MethodBase	method	

TYPE	NAME	DESCRIPTION
System.Func<System.Boolean>	get	
System.Action<System.Boolean>	set	

Setup(MethodBase, PropertyInfo, Action<Boolean>, Object)

Sets up an toggleable menu item, using property as backing value.

Declaration

```
public static void Setup(MethodBase method, PropertyInfo property, Action<bool> onChecked = null, object
propertyTarget = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Reflection.MethodBase	method	
System.Reflection.PropertyInfo	property	
System.Action<System.Boolean>	onChecked	
System.Object	propertyTarget	

Class OnGUIPrompt

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
OnGUIPrompt

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class OnGUIPrompt : EditorWindow
```

Methods

Prompt(GUIContent, Action, Action, Action, Action, Action, Boolean, Boolean, String, RectOffset, Nullable<Vector2>)

Declaration

```
public static bool Prompt(GUIContent title, Action onGUI, Action onEnable = null, Action onDisable = null,
Action onFocus = null, Action onLostFocus = null, bool drawDefaultFooter = true, bool show = true, string
acceptButton = "Continue", RectOffset margin = null, Vector2? size = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GUIContent	title	
System.Action	onGUI	
System.Action	onEnable	
System.Action	onDisable	
System.Action	onFocus	
System.Action	onLostFocus	
System.Boolean	drawDefaultFooter	
System.Boolean	show	
System.String	acceptButton	
UnityEngine.RectOffset	margin	
System.Nullable<UnityEngine.Vector2>	size	

Returns

TYPE	DESCRIPTION
System.Boolean	

Prompt(GUIContent, Action, out Action<Boolean>, out Action<Boolean>, Action, Action, Action, Action, Boolean, Boolean, String, RectOffset, Nullable<Vector2>)

Shows an editor window with the specified OnGUI callback.

Declaration

```
public static bool Prompt(GUIContent title, Action onGUI, out Action<bool> setResult, out Action<bool>
setCanContinue, Action onEnable = null, Action onDisable = null, Action onFocus = null, Action onLostFocus =
null, bool drawDefaultFooter = true, bool show = true, string acceptButton = "Continue", RectOffset margin =
null, Vector2? size = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GUIContent	title	
System.Action	onGUI	
System.Action<System.Boolean>	setResult	Sets the result and closes this prompt. Use in combination with `drawDefaultFooter`: false. You probably want to declare a variable before calling this and set that as out variable.
System.Action<System.Boolean>	setCanContinue	Sets whatever `acceptButton` is enabled .You probably want to declare a variable before calling this and set that as out variable.
System.Action	onEnable	
System.Action	onDisable	
System.Action	onFocus	
System.Action	onLostFocus	
System.Boolean	drawDefaultFooter	
System.Boolean	show	Determines whatever we should show window. Set to false to show manually.
System.String	acceptButton	
UnityEngine.RectOffset	margin	
System.Nullable<UnityEngine.Vector2>	size	

Returns

TYPE	DESCRIPTION
System.Boolean	true if user pressed `acceptButton`. No effect if `drawDefaultFooter` is false.

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class PersistentSceneInEditorUtility

An utility class to automatically open persistent scenes in editor.

Inheritance

System.Object
PersistentSceneInEditorUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class PersistentSceneInEditorUtility
```

Remarks

Only available in editor.

Methods

GetAssociatedScenes(Scene)

Gets the scenes flagged to open when the specified scene is opened.

Declaration

```
public static Scene[] GetAssociatedScenes(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
Scene[]	

GetPersistentOption(Scene)

Gets the persistent option of a scene.

Declaration

```
public static PersistentSceneInEditorUtility.OpenInEditorSetting GetPersistentOption(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
PersistentSceneInEditorUtility.OpenInEditorSetting	

IsPersistent(Scene)

Declaration

```
public static bool IsPersistent(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

OpenAssociatedPersistentScenes(Scene, Boolean)

Open all scenes that are flagged to open when the specified scene is opened.

Declaration

```
public static void OpenAssociatedPersistentScenes(Scene scene, bool promptSave = false)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	promptSave	If true, then the user will be prompted to save any unsaved scenes before opening the scenes.

Enum PersistentSceneInEditorUtility.OpenInEditorOption

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum OpenInEditorOption
```

Fields

NAME	DESCRIPTION
AnySceneOpens	
Never	
WhenAnyOfTheFollowingScenesOpen	
WhenAnySceneOpensExcept	

Class PersistentSceneInEditorUtility.OpenInEditorSetting

Inheritance

System.Object

PersistentSceneInEditorUtility.OpenInEditorSetting

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public sealed class OpenInEditorSetting
```

Fields

list

Declaration

```
public string[] list
```

Field Value

TYPE	DESCRIPTION
System.String[]	

name

Declaration

```
public string name
```

Field Value

TYPE	DESCRIPTION
System.String	

option

Declaration

```
public PersistentSceneInEditorUtility.OpenInEditorOption option
```

Field Value

TYPE	DESCRIPTION
PersistentSceneInEditorUtility.OpenInEditorOption	

scene

Declaration

```
public string scene
```

Field Value

TYPE	DESCRIPTION
System.String	

Methods

Save()

Saves this persistent option.

Declaration

```
public void Save()
```

Class PromptInt

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<System.String, PromptInt>
PromptInt

Inherited Members

GenericPrompt<String, PromptInt>.validate
GenericPrompt<String, PromptInt>.Validate(String)
GenericPrompt<String, PromptInt>.Prompt(String, Func<String, (T1, T2)<Boolean, String>>[])
GenericPrompt<String, PromptInt>.OnContentGUI(String)
GenericPrompt<String, PromptInt>.title
GenericPrompt<String, PromptInt>.updateSizeAutomatically
GenericPrompt<String, PromptInt>.extraHeight
GenericPrompt<String, PromptInt>.width
GenericPrompt<String, PromptInt>.okButton
GenericPrompt<String, PromptInt>.cancelButton

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PromptInt : GenericPrompt<string, PromptInt>
```

Properties

title

Declaration

```
public override string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptInt>.title

Methods

OnContentGUI(ref String)

Declaration

```
public override void OnContentGUI(ref string value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	value	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptInt>.OnContentGUI(System.String)

Validate(String)

Declaration

```
protected override (bool isValid, string message) Validate(string value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	value	

Returns

TYPE	DESCRIPTION
System.ValueTuple<System.Boolean, System.String>	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptInt>.Validate(System.String)

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class PromptKey

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<System.ValueTuple<EventModifiers, KeyCode>, PromptKey>
PromptKey

Inherited Members

GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.validate
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.Validate((T1, T2)<EventModifiers, KeyCode>)
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.Prompt((T1, T2)<EventModifiers, KeyCode>, Func<(T1, T2)<EventModifiers, KeyCode>, (T1, T2)<Boolean, String>>[])
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.OnContentGUI((T1, T2)<EventModifiers, KeyCode>)
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.title
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.updateSizeAutomatically
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.extraHeight
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.width
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.okButton
GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>.cancelButton

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PromptKey : GenericPrompt<(EventModifiers modifiers, KeyCode key), PromptKey>
```

Properties

title

Declaration

```
public override string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.ValueTuple<UnityEngine.EventModifiers, UnityEngine.KeyCode>, AdvancedSceneManager.Editor.Utility.PromptKey>.title

Methods

OnContentGUI(ref (EventModifiers modifiers, KeyCode key))

Declaration

```
public override void OnContentGUI(ref (EventModifiers modifiers, KeyCode key) value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<EventModifiers, KeyCode>	value	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.ValueTuple<UnityEngine.EventModifiers, UnityEngine.KeyCode>, AdvancedSceneManager.Editor.Utility.PromptKey>.OnContentGUI(System.ValueTuple<UnityEngine.EventModifiers, UnityEngine.KeyCode>)

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class PromptName

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<System.String, PromptName>
PromptName

Inherited Members

GenericPrompt<String, PromptName>.validate
GenericPrompt<String, PromptName>.Validate(String)
GenericPrompt<String, PromptName>.Prompt(String, Func<String, (T1, T2)<Boolean, String>>[])
GenericPrompt<String, PromptName>.OnContentGUI(String)
GenericPrompt<String, PromptName>.title
GenericPrompt<String, PromptName>.updateSizeAutomatically
GenericPrompt<String, PromptName>.extraHeight
GenericPrompt<String, PromptName>.width
GenericPrompt<String, PromptName>.okButton
GenericPrompt<String, PromptName>.cancelButton

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PromptName : GenericPrompt<string, PromptName>
```

Properties

title

Declaration

```
public override string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptName>.title

Methods

OnContentGUI(ref String)

Declaration

```
public override void OnContentGUI(ref string value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	value	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptName>.OnContentGUI(System.String)

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class PromptNameAndMessage

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<System.ValueTuple<System.String, System.String>, PromptNameAndMessage>
PromptNameAndMessage

Inherited Members

GenericPrompt<(String name, String message), PromptNameAndMessage>.validate
GenericPrompt<(String name, String message), PromptNameAndMessage>.Validate((T1, T2)<String, String>)
GenericPrompt<(String name, String message), PromptNameAndMessage>.Prompt((T1, T2)<String, String>, Func<(T1, T2)<String, String>, (T1, T2)<Boolean, String>>[])
GenericPrompt<(String name, String message), PromptNameAndMessage>.OnContentGUI((T1, T2)<String, String>)
GenericPrompt<(String name, String message), PromptNameAndMessage>.title
GenericPrompt<(String name, String message), PromptNameAndMessage>.updateSizeAutomatically
GenericPrompt<(String name, String message), PromptNameAndMessage>.extraHeight
GenericPrompt<(String name, String message), PromptNameAndMessage>.width
GenericPrompt<(String name, String message), PromptNameAndMessage>.okButton
GenericPrompt<(String name, String message), PromptNameAndMessage>.cancelButton

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PromptNameAndMessage : GenericPrompt<(string name, string message), PromptNameAndMessage>
```

Properties

extraHeight

Declaration

```
public override float extraHeight { get; }
```

Property Value

TYPE	DESCRIPTION
System.Single	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.ValueTuple<System.String, System.String>, AdvancedSceneManager.Editor.Utility.PromptNameAndMessage>.extraHeight

title

Declaration

```
public override string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.ValueTuple<System.String, System.String>, AdvancedSceneManager.Editor.Utility.PromptNameAndMessage>.title

Methods

OnContentGUI(ref (String name, String message))

Declaration

```
public override void OnContentGUI(ref (string name, string message) value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<System.String, System.String>	value	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.ValueTuple<System.String, System.String>, AdvancedSceneManager.Editor.Utility.PromptNameAndMessage>.OnContentGUI(System.ValueTuple<System.String, System.String>)

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class PromptVersion

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.EditorWindow
GenericPrompt<System.String, PromptVersion>
PromptVersion

Inherited Members

GenericPrompt<String, PromptVersion>.validate
GenericPrompt<String, PromptVersion>.Validate(String)
GenericPrompt<String, PromptVersion>.Prompt(String, Func<String, (T1, T2)<Boolean, String>>[])
GenericPrompt<String, PromptVersion>.OnContentGUI(String)
GenericPrompt<String, PromptVersion>.title
GenericPrompt<String, PromptVersion>.updateSizeAutomatically
GenericPrompt<String, PromptVersion>.extraHeight
GenericPrompt<String, PromptVersion>.width
GenericPrompt<String, PromptVersion>.okButton
GenericPrompt<String, PromptVersion>.cancelButton

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PromptVersion : GenericPrompt<string, PromptVersion>
```

Properties

title

Declaration

```
public override string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptVersion>.title

Methods

OnContentGUI(ref String)

Declaration

```
public override void OnContentGUI(ref string value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	value	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptVersion>.OnContentGUI(System.String)

Validate(String)

Declaration

```
protected override (bool isValid, string message) Validate(string value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	value	

Returns

TYPE	DESCRIPTION
System.ValueTuple<System.Boolean, System.String>	

Overrides

AdvancedSceneManager.Editor.Utility.GenericPrompt<System.String, AdvancedSceneManager.Editor.Utility.PromptVersion>.Validate(System.String)

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class SettingsTab

Provides the ability to add settings to advanced scene manager window settings.

Inheritance

System.Object
SettingsTab

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SettingsTab
```

Remarks

Only available in editor.

Constructors

SettingsTab()

Declaration

```
public SettingsTab()
```

Properties

DefaultHeaders

Declaration

```
public SettingsTab._DefaultHeaders DefaultHeaders { get; }
```

Property Value

TYPE	DESCRIPTION
SettingsTab._DefaultHeaders	

instance

Declaration

```
public static SettingsTab instance { get; }
```

Property Value

TYPE	DESCRIPTION
SettingsTab	

Methods

Add(VisualElement, String)

Add field to settings tab.

Declaration

```
public void Add(VisualElement callback, string header)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	callback	The callback that is called when your field is to be constructed. An element of type UnityEngine.UIElements.VisualElement is expected to be returned.
System.String	header	The header to place the field under, see also DefaultHeaders, for the default headers.

AddHeaderContent(VisualElement, String)

Declaration

```
public void AddHeaderContent(VisualElement callback, string header)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	callback	
System.String	header	

GetHeaderOrder(String)

Declaration

```
public int GetHeaderOrder(string header)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	header	

Returns

TYPE	DESCRIPTION
System.Int32	

Remove(VisualElement)

Removes the setting field.

Declaration

```
public void Remove(VisualElement callback)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	callback	

RemoveHeaderContent(VisualElement)

Declaration

```
public void RemoveHeaderContent(VisualElement callback)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	callback	

SetHeaderOrder(String, Int32)

Declaration

```
public void SetHeaderOrder(string header, int order)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	header	
System.Int32	order	

Spacer(String)

Declaration

```
public void Spacer(string header)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	header	

Class SettingsTab._DefaultHeaders

Inheritance

System.Object

SettingsTab._DefaultHeaders

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class _DefaultHeaders
```

Properties

Appearance

Declaration

```
public string Appearance { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Appearance_CollectionHeader

Declaration

```
public string Appearance_CollectionHeader { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Appearance_Hierarchy

Declaration

```
public string Appearance_Hierarchy { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Appearance_ScenesTab

Declaration

```
public string Appearance_ScenesTab { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Appearance_WindowHeader

Declaration

```
public string Appearance_WindowHeader { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Blacklist

Declaration

```
public string Blacklist { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

DynamicCollections

Declaration

```
public string DynamicCollections { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Options

Declaration

```
public string Options { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Options_Local

Declaration

```
public string Options_Local { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Options_Log

Declaration

```
public string Options_Log { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Options_Profile

Declaration

```
public string Options_Profile { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Options_Project

Declaration

```
public string Options_Project { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

PluginsAndExamples

Declaration

```
public string PluginsAndExamples { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

PluginsAndExamples_Examples

Declaration

```
public string PluginsAndExamples_Examples { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

PluginsAndExamples_Experiments

Declaration

```
public string PluginsAndExamples_Experiments { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

PluginsAndExamples_Plugins

Declaration

```
public string PluginsAndExamples_Plugins { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Class StyleExtensions

Inheritance

System.Object

StyleExtensions

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public static class StyleExtensions
```

Methods

SetBorderColor(IStyle, Nullable<Color>, Nullable<Color>, Nullable<Color>, Nullable<Color>, Nullable<Color>, Nullable<Color>, Nullable<Color>)

Declaration

```
public static void SetBorderColor(this IStyle style, Color? all = null, Color? vertical = null, Color?
horizontal = null, Color? left = null, Color? right = null, Color? top = null, Color? bottom = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.IStyle	style	
System.Nullable<UnityEngine.Color>	all	
System.Nullable<UnityEngine.Color>	vertical	
System.Nullable<UnityEngine.Color>	horizontal	
System.Nullable<UnityEngine.Color>	left	
System.Nullable<UnityEngine.Color>	right	
System.Nullable<UnityEngine.Color>	top	
System.Nullable<UnityEngine.Color>	bottom	

SetBorderWidth(IStyle, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>)

Declaration

```
public static void SetBorderWidth(this IStyle style, float? all = null, float? vertical = null, float?
horizontal = null, float? left = null, float? right = null, float? top = null, float? bottom = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.IStyle	style	
System.Nullable<System.Single>	all	

TYPE	NAME	DESCRIPTION
System.Nullable<System.Single>	vertical	
System.Nullable<System.Single>	horizontal	
System.Nullable<System.Single>	left	
System.Nullable<System.Single>	right	
System.Nullable<System.Single>	top	
System.Nullable<System.Single>	bottom	

SetMargin(IStyle, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>)

Declaration

```
public static void SetMargin(this IStyle style, float? all = null, float? vertical = null, float? horizontal = null, float? left = null, float? right = null, float? top = null, float? bottom = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.IStyle	style	
System.Nullable<System.Single>	all	
System.Nullable<System.Single>	vertical	
System.Nullable<System.Single>	horizontal	
System.Nullable<System.Single>	left	
System.Nullable<System.Single>	right	
System.Nullable<System.Single>	top	
System.Nullable<System.Single>	bottom	

SetPadding(IStyle, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>, Nullable<Single>)

Declaration

```
public static void SetPadding(this IStyle style, float? all = null, float? vertical = null, float? horizontal = null, float? left = null, float? right = null, float? top = null, float? bottom = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.IStyle	style	

TYPE	NAME	DESCRIPTION
System.Nullable<System.Single>	all	
System.Nullable<System.Single>	vertical	
System.Nullable<System.Single>	horizontal	
System.Nullable<System.Single>	left	
System.Nullable<System.Single>	right	
System.Nullable<System.Single>	top	
System.Nullable<System.Single>	bottom	

Class TrimUtility

Applies text-overflow uss property in 2020+, but applies manual trim in earlier versions.

Inheritance

System.Object
TrimUtility

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class TrimUtility
```

Methods

GetLength(Char, TextElement)

Declaration

```
public static float GetLength(char c, TextElement label)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Char	c	
UnityEngine.UIElements.TextElement	label	

Returns

TYPE	DESCRIPTION
System.Single	

GetLength(String, TextElement)

Declaration

```
public static float GetLength(string str, TextElement label)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	str	
UnityEngine.UIElements.TextElement	label	

Returns

TYPE	DESCRIPTION
System.Single	

TrimLabel(TextElement, String, Func<Single>, Boolean)

Trim the label and show ellipsis if too long.

Declaration

```
public static void TrimLabel(this TextElement label, string text, Func<float> maxWidth, bool enableAuto)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.TextElement	label	
System.String	text	
System.Func<System.Single>	maxWidth	
System.Boolean	enableAuto	

Class VisualElementExtensions

Inheritance

System.Object

VisualElementExtensions

Namespace: AdvancedSceneManager.Editor.Utility

Assembly: AdvancedSceneManager.Editor.dll

Syntax

```
public static class VisualElementExtensions
```

Properties

DefaultBackgroundColor

Declaration

```
public static Color DefaultBackgroundColor { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.Color	

Methods

FindAncestor(VisualElement, String, String, Action<VisualElement>)

Declaration

```
public static VisualElement FindAncestor(this VisualElement element, string name = "", string className = "",
Action<VisualElement> actionToPerformOnEachParent = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	element	
System.String	name	
System.String	className	
System.Action<UnityEngine.UIElements.VisualElement>	actionToPerformOnEachParent	

Returns

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

GetRoot(VisualElement)

Declaration

```
public static VisualElement GetRoot(this VisualElement element)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.VisualElement	element	

Returns

TYPE	DESCRIPTION
UnityEngine.UIElements.VisualElement	

SetEnabledExt<TElement>(TElement, Boolean)

Declaration

```
public static TElement SetEnabledExt<TElement>(this TElement element, bool enabled)
    where TElement : VisualElement
```

Parameters

TYPE	NAME	DESCRIPTION
TElement	element	
System.Boolean	enabled	

Returns

TYPE	DESCRIPTION
TElement	

Type Parameters

NAME	DESCRIPTION
TElement	

Setup(Button, Action)

Declaration

```
public static Button Setup(this Button element, Action onClick)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.Button	element	
System.Action	onClick	

Returns

TYPE	DESCRIPTION
UnityEngine.UIElements.Button	

Setup(Toggle, EventCallback<ChangeEvent<Boolean>>, Boolean, String)

Declaration

```
public static Toggle Setup(this Toggle element, EventCallback<ChangeEvent<bool>> valueChanged, bool
defaultValue = false, string tooltip = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.UIElements.Toggle	element	
UnityEngine.UIElements.EventCallback<UnityEngine.UIElements.ChangeEvent<System.Boolean>>	valueChanged	
System.Boolean	defaultValue	
System.String	tooltip	

Returns

TYPE	DESCRIPTION
UnityEngine.UIElements.Toggle	

Setup<T>(SceneField, String, T, String, Action, Boolean, String)

Declaration

```
public static SceneField Setup<T>(this SceneField element, string label, T target, string field, Action
onChanged = null, bool saveInSceneManagerWindow = true, string tooltip = null)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneField	element	
System.String	label	
T	target	
System.String	field	
System.Action	onChanged	
System.Boolean	saveInSceneManagerWindow	
System.String	tooltip	

Returns

TYPE	DESCRIPTION
SceneField	

Type Parameters

NAME	DESCRIPTION
T	

Setup<TElement, T>(TElement, T, String, Action, Boolean, String)

Sets up an ui toolkit element, by setting current value and registers value changed callback and automatically sets the new value to target.

Declaration

```
public static TElement Setup<TElement, T>(this TElement element, T target, string field, Action onChanged =
null, bool saveInSceneManagerWindow = true, string tooltip = null)
    where TElement : VisualElement
```

Parameters

TYPE	NAME	DESCRIPTION
TElement	element	
T	target	
System.String	field	
System.Action	onChanged	
System.Boolean	saveInSceneManagerWindow	
System.String	tooltip	

Returns

TYPE	DESCRIPTION
TElement	

Type Parameters

NAME	DESCRIPTION
TElement	
T	

SetValueWithoutNotifyExt(EnumField, Enum)

Declaration

```
public static EnumField SetValueWithoutNotifyExt(this EnumField element, Enum value)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEditor.UIElements.EnumField	element	

TYPE	NAME	DESCRIPTION
System.Enum	value	

Returns

TYPE	DESCRIPTION
UnityEditor.UIElements.EnumField	

Namespace AdvancedSceneManager.Exceptions

Classes

CloseSceneException

Thrown when a scene could not be closed.

OpenSceneException

Thrown when a scene could not be opened.

Class CloseSceneException

Thrown when a scene could not be closed.

Inheritance

System.Object
System.Exception
CloseSceneException

Namespace: AdvancedSceneManager.Exceptions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class CloseSceneException : Exception, ISerializable
```

Constructors

CloseSceneException(Scene, Scene, SceneCollection, String)

Declaration

```
public CloseSceneException(Scene scene, Scene unityScene, SceneCollection collection = null, string message =
null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
UnityEngine.SceneManagement.Scene	unityScene	
SceneCollection	collection	
System.String	message	

Fields

collection

Declaration

```
public SceneCollection collection
```

Field Value

TYPE	DESCRIPTION
SceneCollection	

scene

Declaration

```
public Scene scene
```

Field Value

TYPE	DESCRIPTION
Scene	

unityScene

Declaration

```
public Scene unityScene
```

Field Value

TYPE	DESCRIPTION
UnityEngine.SceneManagement.Scene	

Class OpenSceneException

Thrown when a scene could not be opened.

Inheritance

System.Object
System.Exception
OpenSceneException

Namespace: AdvancedSceneManager.Exceptions

Assembly: AdvancedSceneManager.dll

Syntax

```
public class OpenSceneException : Exception, ISerializable
```

Constructors

OpenSceneException(Scene, SceneCollection, String)

Declaration

```
public OpenSceneException(Scene scene, SceneCollection collection = null, string message = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
SceneCollection	collection	
System.String	message	

Properties

collection

The collection that the scene was associated with.

Null if scene was opened as stand-alone.

Declaration

```
public SceneCollection collection { get; }
```

Property Value

TYPE	DESCRIPTION
SceneCollection	

scene

The scene that was attempted to be opened.

Declaration

```
public Scene scene { get; }
```

Property Value

TYPE	DESCRIPTION
Scene	

Namespace AdvancedSceneManager.Models

Classes

ASMSettings

Settings relating to ASM.

ASMSettings.Local

ASMSettings.SettingsProxy

Used to make settings a bit more intuitive.

DynamicCollection

Represents a dynamic collection.

Profile

A profile, contains settings, collections and scenes.

Scene

A Scene is a UnityEngine.ScriptableObject that represents a scene in Unity, and are automatically generated or updated when a scene is added, renamed, moved or removed.

The advantage of doing it this way is that we can actually create variables in script that refers to a scene rather than an arbitrary int or magic string. This also allows us to open scenes directly from an UnityEngine.Events.UnityEvent and not have to use a proxy script.

SceneCollection

A SceneCollection contains a list of Scene, all of which are opened when the SceneCollection is opened (except for scenes tagged DoNotOpen).

SceneCollectionTemplate

Represents a template for a SceneCollection.

SceneTag

A layer that makes it easier to identify certain scenes.

TagList

Structs

IsOpenReturnValue

A struct to make working with the return value of the IsOpen() function a bit easier.

Implicitly casts to bool, which can be used to check if scene is open as either a collection or as standalone.

Interfaces

ISceneObject

Identifies either SceneCollection or Scene.

Enums

ASMSettings.Local.SaveAction

CollectionStartupOption

CollectionThreadPriority

Wrapper for UnityEngine.ThreadPriority, adds Auto.

UnityEngine.ThreadPriority:

LoadingScreenUsage

SceneCloseBehavior

SceneOpenBehavior

Class ASMSettings

Settings relating to ASM.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
ASMSettings

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class ASMSettings : ScriptableObject
```

Remarks

Usage: settings.

Properties

buildProfile

The profile to use during build, this is set to current before building.

Declaration

```
public Profile buildProfile { get; }
```

Property Value

TYPE	DESCRIPTION
Profile	

buildUnitySplashScreenColor

This is the color of the unity splash screen, used to make fade from splash screen to asm smooth, this is set before building. UnityEngine.Color.black is used when the unity splash screen is disabled.

Declaration

```
public Color buildUnitySplashScreenColor { get; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.Color	

inGameToolbarEnabled

Enables or disables InGameToolbarUtility in builds.

Declaration

```
public bool inGameToolbarEnabled { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

inGameToolbarExpandedByDefault

Gets or sets whatever InGameToolbarUtility in should be expanded by default.

Declaration

```
public bool inGameToolbarExpandedByDefault { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

name

Declaration

```
public string name { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

profile

The currently selected profile.

Declaration

```
public Profile profile { get; }
```

Property Value

TYPE	DESCRIPTION
Profile	

Remarks

This is the same as buildProfile in builds.

Methods

MarkAsDirty()

Mark scriptable object as dirty after modifying.

Declaration

```
public void MarkAsDirty()
```

Remarks

Only available in editor.

Save()

Saves the scriptable object after modifying.

Declaration

```
public void Save()
```

Remarks

Only available in editor.

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class ASMSettings.Local

Inheritance

System.Object

ASMSettings.Local

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Local
```

Properties

allowEditingOfBuildSettings

Declaration

```
public bool allowEditingOfBuildSettings { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

allowExcludingCollectionsFromBuild

Declaration

```
public bool allowExcludingCollectionsFromBuild { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

autoOpenScenesWhenCreated

Declaration

```
public bool autoOpenScenesWhenCreated { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayCollectionAdditiveButton

Declaration

```
public bool displayCollectionAdditiveButton { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayCollectionOpenButton

Declaration

```
public bool displayCollectionOpenButton { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayCollectionPlayButton

Declaration

```
public bool displayCollectionPlayButton { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayCollectionTitleOnScenesInHierarchy

Declaration

```
public bool displayCollectionTitleOnScenesInHierarchy { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayDynamicCollections

Declaration

```
public bool displayDynamicCollections { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayExtraAddCollectionButton

Declaration

```
public bool displayExtraAddCollectionButton { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displayPersistentIndicatorInHierarchy

Declaration

```
public bool displayPersistentIndicatorInHierarchy { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

displaySceneHelperDragButton

Declaration

```
public bool displaySceneHelperDragButton { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

openAssociatedCollectionOnSceneAssetOpen

When true: opens the first found collection that a scene is contained in when opening an SceneAsset in editor.

Declaration

```
public bool openAssociatedCollectionOnSceneAssetOpen { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

saveActionWhenUsingASMPlayButton

Declaration

```
public ASMSettings.Local.SaveAction saveActionWhenUsingASMPlayButton { get; set; }
```

Property Value

TYPE	DESCRIPTION
ASMSettings.Local.SaveAction	

useSaveDialogWhenCreatingScenesFromSceneField

Declaration

```
public bool useSaveDialogWhenCreatingScenesFromSceneField { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

Get<T>(T, String)

Gets a value from local storage.

Declaration

```
public T Get<T>(T defaultValue = null, string key = "")
```

Parameters

TYPE	NAME	DESCRIPTION
T	defaultValue	
System.String	key	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

Set<T>(T, String)

Sets a value from local storage.

Declaration

```
public void Set<T>(T value, string key = "")
```

Parameters

TYPE	NAME	DESCRIPTION
T	value	
System.String	key	

Type Parameters

NAME	DESCRIPTION
T	

Enum ASMSettings.Local.SaveAction

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum SaveAction
```

Fields

NAME	DESCRIPTION
DoNothing	
Prompt	
Save	

Class ASMSettings.SettingsProxy

Used to make settings a bit more intuitive.

Inheritance

System.Object
ASMSettings.SettingsProxy

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SettingsProxy
```

Properties

local

The local asm settings, not synced to source.

Declaration

```
public ASMSettings.Local local { get; }
```

Property Value

TYPE	DESCRIPTION
ASMSettings.Local	

profile

The profile-wide asm settings.

Declaration

```
public Profile profile { get; }
```

Property Value

TYPE	DESCRIPTION
Profile	

project

The project-wide asm settings.

Declaration

```
public ASMSettings project { get; }
```

Property Value

TYPE	DESCRIPTION
ASMSettings	

Enum CollectionStartupOption

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum CollectionStartupOption
```

Fields

NAME	DESCRIPTION
Auto	
DoNotOpen	
Open	
OpenAsPersistent	

Enum CollectionThreadPriority

Wrapper for UnityEngine.ThreadPriority, adds Auto.

UnityEngine.ThreadPriority:

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum CollectionThreadPriority
```

Fields

NAME	DESCRIPTION
Auto	Automatically decide UnityEngine.ThreadPriority based on if loading screen is open.
BelowNormal	
High	
Low	
Normal	

Class DynamicCollection

Represents a dynamic collection.

Inheritance

System.Object
DynamicCollection

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class DynamicCollection
```

Fields

isAuto

Declaration

```
public bool isAuto
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

scenes

Declaration

```
public List<string> scenes
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<System.String>	

title

Declaration

```
public string title
```

Field Value

TYPE	DESCRIPTION
System.String	

Interface ISceneObject

Identifies either SceneCollection or Scene.

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface ISceneObject
```

Methods

OnPropertyChanged()

Declaration

```
void OnPropertyChanged()
```

Struct IsOpenReturnValue

A struct to make working with the return value of the IsOpen() function a bit easier.

Implicitly casts to bool, which can be used to check if scene is open as either a collection or as standalone.

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public struct IsOpenReturnValue
```

Fields

asStandalone

The scene was opened as standalone.

Declaration

```
public bool asStandalone
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

isPreloaded

The scene is currently preloaded.

Declaration

```
public bool isPreloaded
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

withCollection

The scene was opened as part of a collection.

Declaration

```
public bool withCollection
```

Field Value

TYPE	DESCRIPTION
System.Boolean	

Operators

Implicit(IsOpenReturnValue to Boolean)

Declaration

```
public static implicit operator bool (IsOpenReturnValue value)
```

Parameters

TYPE	NAME	DESCRIPTION
IsOpenReturnValue	value	

Returns

TYPE	DESCRIPTION
System.Boolean	

Implicit((Boolean withCollection, Boolean asStandalone, Boolean isPreloaded) to IsOpenReturnValue)

Declaration

```
public static implicit operator IsOpenReturnValue((bool withCollection, bool asStandalone, bool isPreloaded)
value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<System.Boolean, System.Boolean, System.Boolean>	value	

Returns

TYPE	DESCRIPTION
IsOpenReturnValue	

Implicit((Boolean withCollection, Boolean asStandalone) to IsOpenReturnValue)

Declaration

```
public static implicit operator IsOpenReturnValue((bool withCollection, bool asStandalone) value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.ValueTuple<System.Boolean, System.Boolean>	value	

Returns

TYPE	DESCRIPTION
IsOpenReturnValue	

Enum LoadingScreenUsage

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum LoadingScreenUsage
```

Fields

NAME	DESCRIPTION
DoNotUse	
Override	
UseDefault	

Class Profile

A profile, contains settings, collections and scenes.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
Profile

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Profile : ScriptableObject, INotifyPropertyChanged
```

Fields

tagDefinitions

The layers defined in the tags tab in the scene manager window.

Declaration

```
public SceneTag[] tagDefinitions
```

Field Value

TYPE	DESCRIPTION
SceneTag[]	

Properties

backgroundLoadingPriority

UnityEngine.Application.backgroundLoadingPriority setting is not saved, and must be manually set every time build or editor starts, this property persists the value and automatically sets it during startup.

Declaration

```
public ThreadPriority backgroundLoadingPriority { get; set; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.ThreadPriority	

blacklist

The blacklist settings.

Declaration

```
public SettingsModule blacklist { get; }
```

Property Value

TYPE	DESCRIPTION
plugin.asm.dependency_manager.Blacklist.SettingsModule	

Remarks

Only available in editor.

collections

Gets the collections contained within this profile.

Declaration

```
public ReadOnlyCollection<SceneCollection> collections { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<SceneCollection>	

createCameraDuringStartup

Enable or disable ASM automatically creating a camera during startup.

Declaration

```
public bool createCameraDuringStartup { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

current

Gets the currently active profile.

Declaration

```
public static Profile current { get; set; }
```

Property Value

TYPE	DESCRIPTION
Profile	

dynamicCollectionPaths

The paths of which all scenes should be included in build, as a dynamic collection.

Declaration

```
public string[] dynamicCollectionPaths { get; }
```

Property Value

TYPE	DESCRIPTION
System.String[]	

Remarks

These are only evaluated and used in BuildSettingsUtility.

dynamicCollections

Gets the dynamic collections specified on this profile.

Declaration

```
public DynamicCollection[] dynamicCollections { get; }
```

Property Value

TYPE	DESCRIPTION
DynamicCollection[]	

enableChangingBackgroundLoadingPriority

Enable or disable ASM automatically changing UnityEngine.Application.backgroundLoadingPriority.

Declaration

```
public bool enableChangingBackgroundLoadingPriority { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

includeFadeLoadingScene

Enables the fade loading scene.

Declaration

```
public bool includeFadeLoadingScene { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Remarks

Has no effect while in play mode.

loadingScreen

The default loading screen.

Declaration

```
public Scene loadingScreen { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

name

Declaration

```
public string name { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

scenePaths

Gets the scenes managed by this profile.

Declaration

```
public IEnumerable<string> scenePaths { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<System.String>	

Remarks

Includes both collection and standalone scenes.

scenes

Gets the scenes managed by this profile.

Declaration

```
public IEnumerable<Scene> scenes { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	

Remarks

Includes both collection and standalone scenes.

splashScreen

The splash screen.

Declaration

```
public Scene splashScreen { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

startupLoadingScreen

The loading screen to use during startup.

Declaration

```
public Scene startupLoadingScreen { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

startupScene

The startup scene.

Declaration

```
public Scene startupScene { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

useDefaultPauseScreen

Enables the default pause screen.

Declaration

```
public bool useDefaultPauseScreen { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Remarks

Has no effect while in play mode.

Methods

Add(Scene, String, Boolean)

Adds a scene to a dynamic collection on this profile.

Declaration

```
public void Add(Scene scene, string collectionTitle = "", bool updateBuildSettings = true)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	collectionTitle	
System.Boolean	updateBuildSettings	

Remarks

Only available in editor.

Add(SceneCollection)

Adds a collection to this profile.

Declaration

```
public void Add(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Remarks

Only available in editor.

Clear(String, Boolean, Boolean)

Clears a dynamic collection.

Declaration

```
public void Clear(string collectionTitle, bool save = true, bool updateBuildSettings = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	collectionTitle	
System.Boolean	save	
System.Boolean	updateBuildSettings	

Remarks

Only available in editor.

CreateCollection(String, Action<SceneCollection>)

Create a collection and add it to this profile.

Declaration

```
public SceneCollection CreateCollection(string name, Action<SceneCollection> initializeBeforeSave = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
System.Action<SceneCollection>	initializeBeforeSave	

Returns

TYPE	DESCRIPTION
SceneCollection	

Remarks

Only available in editor.

Delete()

Declaration

```
public void Delete()
```

Find(Func<Profile, Boolean>)

Declaration

```
public static Profile Find(Func<Profile, bool> predicate)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<Profile, System.Boolean>	predicate	

Returns

TYPE	DESCRIPTION
Profile	

Find(String)

Declaration

```
public static Profile Find(string name)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	

Returns

TYPE	DESCRIPTION
Profile	

FindAll()

Declaration

```
public static Profile[] FindAll()
```

Returns

TYPE	DESCRIPTION
Profile[]	

IsSet(String, Boolean)

Gets if the scene is added in a dynamic collection.

Declaration

```
public bool IsSet(string scene, bool includeStandalone = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.Boolean	includeStandalone	

Returns

TYPE	DESCRIPTION
System.Boolean	

IsSet(String, String)

Gets if the scene is added in a dynamic collection.

Declaration

```
public bool IsSet(string key, string scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	key	
System.String	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

MarkAsDirty()

Mark scriptable object as dirty after modifying.

Declaration

```
public void MarkAsDirty()
```

Remarks

No effect in build.

Order(SceneCollection)

Returns the order of this collection.

Declaration

```
public int Order(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Int32	

Order(SceneCollection, Nullable<Int32>)

Returns and/or sets the order of this collection in the scene manager window.

Declaration

```
public int Order(SceneCollection collection, int? newIndex = null)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Nullable<System.Int32>	newIndex	

Returns

TYPE	DESCRIPTION
System.Int32	

Remarks

Cannot use in build.

Remove(Scene, String, Boolean)

Removes the scene from the specified dynamic collection.

Declaration

```
public void Remove(Scene scene, string collectionTitle = "", bool updateBuildSettings = true)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	collectionTitle	
System.Boolean	updateBuildSettings	

Remarks

Only available in editor.

Remove(SceneCollection)

Removes a collection from this profile.

Declaration

```
public void Remove(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Remarks

Only available in editor.

StartupCollections()

Gets the collections that will be opened on startup.

Declaration

```
public IEnumerable<SceneCollection> StartupCollections()
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<SceneCollection>	

Remarks

If no collection is explicitly defined to be opened during startup, then the first available collection in list will be returned.

Events

onProfileChanged

Declaration

```
public static event Action onProfileChanged
```

Event Type

TYPE	DESCRIPTION
System.Action	

PropertyChanged

Declaration

```
public event PropertyChangedEventHandler PropertyChanged
```

Event Type

TYPE	DESCRIPTION
System.ComponentModel.PropertyChangedEventHandler	

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class Scene

A Scene is a UnityEngine.ScriptableObject that represents a scene in Unity, and are automatically generated or updated when a scene is added, renamed, moved or removed.

The advantage of doing it this way is that we can actually create variables in script that refers to a scene rather than an arbitrary int or magic string. This also allows us to open scenes directly from an UnityEngine.Events.UnityEvent and not have to use a proxy script.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
Scene

Implements

ISceneObject

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class Scene : ScriptableObject, ISceneObject, IEquatable<Scene>, IEquatable<OpenSceneInfo>,
IEquatable<Scene>, IEquatable<Scene?>, IEquatable<SceneAsset>, INotifyPropertyChanged
```

Properties

assetID

The id of the asset in the asset database.

Automatically updated.

Declaration

```
public string assetID { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

isActive

Gets if this scene is currently active.

Declaration

```
public bool isActive { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isIncluded

Gets whatever this scene is included in build, this would be if the scene is contained in a collection, or if it is used as a splash

screen / loading screen.

Declaration

```
public bool isIncluded { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

name

Declaration

```
public string name { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Remarks

See also: assetManagement.Rename().

path

The path to the scene file, relative to the project folder.

Automatically updated.

Declaration

```
public string path { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Methods

Close()

Closes a scene regardless of whatever it is associated with a collection, or is was opened as stand-alone.

Declaration

```
public SceneOperation Close()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

CloseEvent()

Declaration

```
public void CloseEvent()
```

Equals(OpenSceneInfo)

Declaration

```
public bool Equals(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Scene)

Declaration

```
public bool Equals(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Nullable<Scene>)

Declaration

```
public bool Equals(Scene? scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Nullable<UnityEngine.SceneManagement.Scene>	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Object)

Declaration

```
public override bool Equals(object other)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	other	

Returns

TYPE	DESCRIPTION
System.Boolean	

Overrides

UnityEngine.Object.Equals(System.Object)

Equals(SceneAsset)

Declaration

```
public bool Equals(SceneAsset scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEditor.SceneAsset	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Scene)

Declaration

```
public bool Equals(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Find(String, SceneCollection, Profile)

Finds the scene with the specified name.

Declaration

```
public static Scene Find(string name, SceneCollection inCollection = null, Profile inProfile = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
SceneCollection	inCollection	
Profile	inProfile	

Returns

TYPE	DESCRIPTION
Scene	

FindAll(String, SceneCollection, Profile)

Finds the scenes with the specified name.

Declaration

```
public static IEnumerable<Scene> FindAll(string name, SceneCollection inCollection = null, Profile inProfile =
null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
SceneCollection	inCollection	
Profile	inProfile	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	

FindCollections(Profile)

Finds which collections that this scene is a part of.

Declaration

```
public (SceneCollection collection, bool asLoadingScreen)[] FindCollections(Profile profile)
```

Parameters

TYPE	NAME	DESCRIPTION
Profile	profile	

Returns

TYPE	DESCRIPTION
System.ValueTuple<SceneCollection, System.Boolean>[]	

FindCollections(Boolean)

Finds which collections that this scene is a part of.

Declaration

```
public (SceneCollection collection, bool asLoadingScreen)[] FindCollections(bool allProfiles = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	allProfiles	

Returns

TYPE	DESCRIPTION
System.ValueTuple<SceneCollection, System.Boolean>[]	

FindObject<T>()

Finds the object in the heirarchy of this Scene.

Declaration

```
public T FindObject<T>()
    where T : Component
```

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

Remarks

Only works if scene is loaded.

FindObjects<T>()

Finds the objects in the heirarchy of this Scene.

Declaration

```
public IEnumerable<T> FindObjects<T>()
    where T : Component
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	

Type Parameters

NAME	DESCRIPTION
T	

Remarks

Only works if scene is loaded.

GetHashCode()

Declaration

```
public override int GetHashCode()
```

Returns

TYPE	DESCRIPTION
System.Int32	

Overrides

UnityEngine.Object.GetHashCode()

GetOpenSceneInfo()

Finds the first open instance of this Scene, if it is open.

Declaration

```
public OpenSceneInfo GetOpenSceneInfo()
```

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

GetRootGameObjects()

Gets the root game objects in this Scene, only works if scene is loaded.

Declaration

```
public IEnumerable<GameObject> GetRootGameObjects()
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<UnityEngine.GameObject>	

IsOpen()

Gets whatever the scene is open, either as part of a collection, or as stand-alone.

Declaration

```
public IsOpenReturnValue IsOpen()
```

Returns

TYPE	DESCRIPTION
IsOpenReturnValue	

OnOpen(Int32, Int32)

Declaration

```
[OnOpenAsset]
public static bool OnOpen(int instanceID, int _)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Int32	instanceID	
System.Int32	_	

Returns

TYPE	DESCRIPTION
System.Boolean	

OnPropertyChanged()

Declaration

```
public void OnPropertyChanged()
```

Open()

Declaration

```
public SceneOperation<OpenSceneInfo> Open()
```

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

OpenEvent()

Declaration

```
public void OpenEvent()
```

OpenSingle()

Close existing scenes and open the specified one.

Declaration

```
public SceneOperation<OpenSceneInfo> OpenSingle()
```

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

Remarks

This will close the current collection.

OpenSingleEvent()

Close existing scenes and open the specified one.

Declaration

```
public void OpenSingleEvent()
```

Remarks

This will close the current collection.

OpenWithLoadingScreenEvent(Scene)

Declaration

```
public void OpenWithLoadingScreenEvent(Scene loadingScene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	loadingScene	

Preload()

Preloads the scene.

Declaration

```
public SceneOperation<PreloadedSceneHelper> Preload()
```

Returns

TYPE	DESCRIPTION
SceneOperation<PreloadedSceneHelper>	

Remarks

Use FinishLoading() or AdvancedSceneManager.Core.SceneManagerBase.Open(AdvancedSceneManager.Models.Scene) to finish loading scene.

Reopen()

Reopen a scene regardless of whatever it is associated with a collection, or is was opened as stand-alone.

Declaration

```
public SceneOperation<OpenSceneInfo> Reopen()
```

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

ReopenEvent()

Declaration

```
public void ReopenEvent()
```

SetActiveScene()

Sets a scene as the activate scene.

Declaration

```
public void SetActiveScene()
```

Toggle()

Toggles the scene open or closed, if the scene is part of the current collection, then the scene will be toggled within the collection, otherwise, it will be toggled as a stand-alone scene.

Declaration

```
public SceneOperation Toggle()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

Toggle(Boolean)

Toggles the scene open or closed, if the scene is part of the current collection, then the scene will be toggled within the collection, otherwise, it will be toggled as a stand-alone scene.

Declaration

```
public SceneOperation Toggle(bool enabled)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	enabled	

Returns

TYPE	DESCRIPTION
SceneOperation	

ToggleEvent()

Declaration

```
public void ToggleEvent()
```

ToggleEvent(Boolean)

Declaration

```
public void ToggleEvent(bool enabled)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	enabled	

Events

PropertyChanged

Declaration

```
public event PropertyChangedEventHandler PropertyChanged
```

Event Type

TYPE	DESCRIPTION
System.ComponentModel.PropertyChangedEventHandler	

Implements

ISceneObject

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Enum SceneCloseBehavior

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum SceneCloseBehavior
```

Fields

NAME	DESCRIPTION
Close	
KeepOpenAlways	
KeepOpenIfNextCollectionAlsoContainsScene	

Class SceneCollection

A SceneCollection contains a list of Scene, all of which are opened when the SceneCollection is opened (except for scenes tagged DoNotOpen).

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
SceneCollection
SceneCollectionTemplate

Implements

ISceneObject

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneCollection : ScriptableObject, IReadOnlyList<Scene>, IReadOnlyCollection<Scene>,
IEnumerable<Scene>, IEnumerable, ISceneObject, INotifyPropertyChanged
```

Remarks

Only one SceneCollection can be open at a time.

Properties

activeScene

Specifies the scene that should be activated after collection is opened.

Declaration

```
public Scene activeScene { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

Count

Declaration

```
public int Count { get; }
```

Property Value

TYPE	DESCRIPTION
System.Int32	

extraData

The extra data that is associated with this collection.

Declaration

```
public ScriptableObject extraData { get; set; }
```

Property Value

TYPE	DESCRIPTION
UnityEngine.ScriptableObject	

Remarks

Use ExtraData<T>() to cast it to the desired type.

hasScenes

Gets if this collection has any scenes.

Declaration

```
public bool hasScenes { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isIncluded

Gets whatever this collection should be included in build.

Declaration

```
public bool isIncluded { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

IsReadOnly

Inherited from System.Collections.Generic.IReadOnlyList<T>, not to be confused with read-only functionality for collections (which does not currently exist, beyond locking plugin), this is just for enumeration.

Declaration

```
public bool IsReadOnly { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Item[Int32]

Declaration

```
public Scene this[int index] { get; }
```

Parameters

TYPE	NAME	DESCRIPTION
System.Int32	index	

Property Value

TYPE	DESCRIPTION
Scene	

loadingPriority

The thread priority to use when opening this collection.

Declaration

```
public CollectionThreadPriority loadingPriority { get; set; }
```

Property Value

TYPE	DESCRIPTION
CollectionThreadPriority	

loadingScreen

The loading screen that is associated with this collection.

Declaration

```
public Scene loadingScreen { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene	

loadingScreenUsage

Specifies what loading screen to use.

Declaration

```
public LoadingScreenUsage loadingScreenUsage { get; set; }
```

Property Value

TYPE	DESCRIPTION
LoadingScreenUsage	

name

Declaration

```
public string name { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Remarks

See also: assetManagement.Rename().

scenes

Gets the scenes in this collection, note that some might be null if no reference is added in scene manager window.

Declaration

```
public Scene[] scenes { get; set; }
```

Property Value

TYPE	DESCRIPTION
Scene[]	

startupOption

Specifies startup option.

Declaration

```
public CollectionStartupOption startupOption { get; set; }
```

Property Value

TYPE	DESCRIPTION
CollectionStartupOption	

title

The title of this collection.

Declaration

```
public string title { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Remarks

See also: assetManagement.Rename().

Methods

AllScenePaths()

Gets all scenes contained in this collection, including overriden loading screen, if set.

Declaration

```
public string[] AllScenePaths()
```

Returns

TYPE	DESCRIPTION
System.String[]	

AllScenes()

Gets all scenes contained in this collection, including overriden loading screen, if set.

Declaration

```
public Scene[] AllScenes()
```

Returns

TYPE	DESCRIPTION
Scene[]	

Close()

Closes the current collection.

Declaration

```
public SceneOperation Close()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

CloseEvent()

Closes the current collection.

Declaration

```
public void CloseEvent()
```

Contains(Scene)

Declaration

```
public bool Contains(Scene item)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	item	

Returns

TYPE	DESCRIPTION
System.Boolean	

ExtraData<T>()

Casts and returns extraData as the specified type. Returns null if invalid type.

Declaration

```
public T ExtraData<T>()
    where T : ScriptableObject
```

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

Find(String, Boolean)

Finds the SceneCollection with the specified name.

Declaration

```
public static SceneCollection Find(string title, bool onlyActiveProfile = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	title	
System.Boolean	onlyActiveProfile	

Returns

TYPE	DESCRIPTION
SceneCollection	

FindProfile()

Find the Profile that this collection is associated with.

Declaration

```
public Profile FindProfile()
```

Returns

TYPE	DESCRIPTION
Profile	

GetEnumerator()

Declaration

```
public IEnumerator<Scene> GetEnumerator()
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerator<Scene>	

IsOpen()

Gets whatever the collection is currently open.

Declaration

```
public bool IsOpen()
```

Returns

TYPE	DESCRIPTION
System.Boolean	

OnPropertyChanged()

Raises PropertyChanged.

Declaration

```
public void OnPropertyChanged()
```

Remarks

Only available in editor.

Open()

Opens the collection.

Declaration

```
public SceneOperation Open()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

OpenEvent()

Opens the collection.

Declaration

```
public void OpenEvent()
```

OpenOrReopen()

Opens or reopens the collection, depending on whatever it is open or not.

Declaration

```
public SceneOperation OpenOrReopen()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

Reopen()

Reopens the current collection.

Declaration

```
public SceneOperation Reopen()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

ReopenEvent()

Reopens the current collection.

Declaration

```
public void ReopenEvent()
```

Tag(Scene, SceneTag)

Declaration

```
public SceneTag Tag(Scene scene, SceneTag setTo = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
SceneTag	setTo	

Returns

TYPE	DESCRIPTION
SceneTag	

Toggle()

Toggles the collection.

Declaration

```
public SceneOperation Toggle()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

Toggle(Boolean)

Toggles the collection.

Declaration

```
public SceneOperation Toggle(bool enabled)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	enabled	

Returns

TYPE	DESCRIPTION
SceneOperation	

ToggleEvent()

Toggles the collection.

Declaration

```
public void ToggleEvent()
```

ToggleEvent(Boolean)

Toggles the collection.

Declaration

```
public void ToggleEvent(bool enabled)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	enabled	

Events

PropertyChanged

Occurs when a property changes on this collection.

Declaration

```
public event PropertyChangedEventHandler PropertyChanged
```

Event Type

TYPE	DESCRIPTION
System.ComponentModel.PropertyChangedEventHandler	

Remarks

Only available in editor.

Implements

ISceneObject

Extension Methods

SceneCollectionUtility.Find(IEnumerable<Scene>, String)
ScriptableObjectUtility.Save(ScriptableObject)
ListUtility.ForEach<T>(IEnumerable<T>, Action<T, Int32>)
ListUtility.Flatten<T>(IEnumerable<T>, Func<T, IEnumerable<T>>)
ListUtility.Except<T>(IEnumerable<T>, T)
ListUtility.GroupConsecutive<T>(IEnumerable<T>, Func<T, T, Boolean>)

Class SceneCollectionTemplate

Represents a template for a SceneCollection.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
SceneCollection
SceneCollectionTemplate

Implements

ISceneObject

Inherited Members

SceneCollection.PropertyChanged
SceneCollection.OnPropertyChanged()
SceneCollection.Count
SceneCollection.IsReadOnly
SceneCollection.Item[Int32]
SceneCollection.Contains(Scene)
SceneCollection.GetEnumerator()
SceneCollection.extraData
SceneCollection.isIncluded
SceneCollection.scenes
SceneCollection.loadingScreen
SceneCollection.loadingScreenUsage
SceneCollection.activeScene
SceneCollection.startupOption
SceneCollection.loadingPriority
SceneCollection.hasScenes
SceneCollection.Tag(Scene, SceneTag)
SceneCollection.Open()
SceneCollection.OpenOrReopen()
SceneCollection.Toggle()
SceneCollection.Toggle(Boolean)
SceneCollection.Reopen()
SceneCollection.Close()
SceneCollection.IsOpen()
SceneCollection.OpenEvent()
SceneCollection.ToggleEvent()
SceneCollection.ToggleEvent(Boolean)
SceneCollection.ReopenEvent()
SceneCollection.CloseEvent()
SceneCollection.AllScenes()
SceneCollection.AllScenePaths()
SceneCollection.FindProfile()
SceneCollection.Find(String, Boolean)
SceneCollection.ExtraData<T>()

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public class SceneCollectionTemplate : SceneCollection, IReadOnlyList<Scene>, IReadOnlyCollection<Scene>,
IEnumerable<Scene>, IEnumerable, ISceneObject, INotifyPropertyChanged
```

Properties

name

Declaration

```
public string name { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Remarks

See also: assetManagement.Rename().

title

The title of this collection.

Declaration

```
public string title { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Remarks

See also: assetManagement.Rename().

Methods

Apply(SceneCollection)

Applies a SceneCollectionTemplate on this SceneCollection.

Declaration

```
public void Apply(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Remarks

Only available in editor. Not reversible.

CreateCollection(Profile)

Creates a SceneCollection from this SceneCollectionTemplate.

Declaration

```
public SceneCollection CreateCollection(Profile profile)
```

Parameters

TYPE	NAME	DESCRIPTION
Profile	profile	

Returns

TYPE	DESCRIPTION
SceneCollection	

Remarks

Only available in editor.

CreateTemplate(String, SceneCollection)

Creates SceneCollectionTemplate from the specified SceneCollection.

Declaration

```
public static SceneCollectionTemplate CreateTemplate(string path, SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
SceneCollectionTemplate	

Remarks

Only available in editor.

CreateTemplateInCurrentFolder(SceneCollection)

Creates SceneCollectionTemplate from the specified SceneCollection, using the currently open folder in the project view.

Declaration

```
public static SceneCollectionTemplate CreateTemplateInCurrentFolder(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
SceneCollectionTemplate	

Remarks

Only available in editor.

Implements

ISceneObject

Extension Methods

SceneCollectionUtility.Find(IEnumerable<Scene>, String)
ScriptableObjectUtility.Save(ScriptableObject)
ListUtility.ForEach<T>(IEnumerable<T>, Action<T, Int32>)
ListUtility.Flatten<T>(IEnumerable<T>, Func<T, IEnumerable<T>>)
ListUtility.Except<T>(IEnumerable<T>, T)
ListUtility.GroupConsecutive<T>(IEnumerable<T>, Func<T, T, Boolean>)

Enum SceneOpenBehavior

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
public enum SceneOpenBehavior
```

Fields

NAME	DESCRIPTION
DoNotOpenInCollection	
OpenNormally	

Class SceneTag

A layer that makes it easier to identify certain scenes.

Inheritance

System.Object

SceneTag

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class SceneTag
```

Constructors

SceneTag()

Declaration

```
[Obsolete]
public SceneTag()
```

SceneTag(String, Nullable<Color>, String)

Declaration

```
public SceneTag(string title, Color? color = null, string id = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	title	
System.Nullable<UnityEngine.Color>	color	
System.String	id	

Fields

closeBehavior

Specifies how the scene should behave when a SceneCollection is closed.

Declaration

```
public SceneCloseBehavior closeBehavior
```

Field Value

TYPE	DESCRIPTION
SceneCloseBehavior	

color

Declaration

```
public Color color
```

Field Value

TYPE	DESCRIPTION
UnityEngine.Color	

Default

Declaration

```
public static readonly SceneTag Default
```

Field Value

TYPE	DESCRIPTION
SceneTag	

DoNotOpen

Declaration

```
public static readonly SceneTag DoNotOpen
```

Field Value

TYPE	DESCRIPTION
SceneTag	

id

Declaration

```
public string id
```

Field Value

TYPE	DESCRIPTION
System.String	

label

Declaration

```
public string label
```

Field Value

TYPE	DESCRIPTION
System.String	

name

Declaration

```
public string name
```

Field Value

TYPE	DESCRIPTION
System.String	

openBehavior

Specifies how the scene should behave when a SceneCollection is opened.

Declaration

```
public SceneOpenBehavior openBehavior
```

Field Value

TYPE	DESCRIPTION
SceneOpenBehavior	

Persistent

Declaration

```
public static readonly SceneTag Persistent
```

Field Value

TYPE	DESCRIPTION
SceneTag	

PersistIfRequired

Declaration

```
public static readonly SceneTag PersistIfRequired
```

Field Value

TYPE	DESCRIPTION
SceneTag	

Methods

Equals(SceneTag)

Declaration

```
public bool Equals(SceneTag layer)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneTag	layer	

Returns

TYPE	DESCRIPTION
System.Boolean	

Equals(Object)

Declaration

```
public override bool Equals(object other)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Object	other	

Returns

TYPE	DESCRIPTION
System.Boolean	

Overrides

System.Object.Equals(System.Object)

Find(String)

Declaration

```
public static SceneTag Find(string id)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	

Returns

TYPE	DESCRIPTION
SceneTag	

GetHashCode()

Declaration

```
public override int GetHashCode()
```

Returns

TYPE	DESCRIPTION
System.Int32	

Overrides

System.Object.GetHashCode()

ToString()

Declaration

```
public override string ToString()
```

Returns

TYPE	DESCRIPTION
System.String	

Overrides

System.Object.ToString()

Operators

Equality(SceneTag, SceneTag)

Declaration

```
public static bool operator ==(SceneTag left, SceneTag right)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneTag	left	
SceneTag	right	

Returns

TYPE	DESCRIPTION
System.Boolean	

Inequality(SceneTag, SceneTag)

Declaration

```
public static bool operator !=(SceneTag left, SceneTag right)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneTag	left	
SceneTag	right	

Returns

TYPE	DESCRIPTION
System.Boolean	

Class TagList

Inheritance

System.Object

System.Collections.Generic.Dictionary<System.String, System.String>

SerializableDictionary<System.String, System.String>

TagList

Inherited Members

SerializableDictionary<String, String>.throwOnDeserializeWhenKeyValueMismatch

SerializableDictionary<String, String>.keys

SerializableDictionary<String, String>.values

SerializableDictionary<String, String>.OnBeforeSerialize()

SerializableDictionary<String, String>.OnAfterDeserialize()

Namespace: AdvancedSceneManager.Models

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class TagList : SerializableDictionary<string, string>, IDictionary<string, string>,
ICollection<KeyValuePair<string, string>>, IReadOnlyDictionary<string, string>,
IReadOnlyCollection<KeyValuePair<string, string>>, IEnumerable<KeyValuePair<string, string>>, IDictionary,
ICollection, IEnumerable, IDeserializationCallback, ISerializable, ISerializationCallbackReceiver
```

Properties

Item[Scene]

Declaration

```
public SceneTag this[Scene scene] { get; set; }
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Property Value

TYPE	DESCRIPTION
SceneTag	

Item[String]

Declaration

```
public SceneTag this[string path] { get; set; }
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Property Value

TYPE	DESCRIPTION
SceneTag	

Methods

TryGetValue(Scene, out SceneTag)

Declaration

```
public bool TryGetValue(Scene scene, out SceneTag layer)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
SceneTag	layer	

Returns

TYPE	DESCRIPTION
System.Boolean	

TryGetValue(String, out SceneTag)

Declaration

```
public bool TryGetValue(string path, out SceneTag layer)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	
SceneTag	layer	

Returns

TYPE	DESCRIPTION
System.Boolean	

Extension Methods

DictionaryUtility.WithNullKey<TKey, TValue>(Dictionary<TKey, TValue>, TKey)
DictionaryUtility.Set<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)
DictionaryUtility.EnsureAdded<TKey, TValue>(Dictionary<TKey, TValue>, TKey)
DictionaryUtility.Add<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)
DictionaryUtility.GetValue<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)
ListUtility.ForEach<T>(IEnumerable<T>, Action<T, Int32>)
ListUtility.Flatten<T>(IEnumerable<T>, Func<T, IEnumerable<T>>)
ListUtility.Except<T>(IEnumerable<T>, T)
ListUtility.GroupConsecutive<T>(IEnumerable<T>, Func<T, T, Boolean>)

Namespace AdvancedSceneManager.Utility

Classes

ASM

This is SceneHelper, but we don't want the script to show up in object picker to avoid confusion, using a different name seems to be the only way?

CanvasGroupExtensions

CanvasSortOrderUtility

An utility class to manage sort order on canvases.

DefaultSceneUtility

An utility class that manages the default scene, called 'AdvancedSceneManager'.

The default scene allows us to more easily close all scenes when we need to, since unity requires at least one scene to be open at any time.

DictionaryUtility

GuidReference

GuidReference.Editor

GuidReferenceUtility

An utility for referencing objects globally.

InGameToolbarUtility

Provides an in-game toolbar that makes debugging scene management in build easier.

LerpUtility

Provides some convinience functions for lerping.

LoadingScreenUtility

Manager for loading screens.

PauseScreenUtility

Contains functions for interacting with the default pause screen.

PersistentUtility

Manages persistent scenes.

PreloadedSceneHelper

An helper class to make working with preloaded scenes easier, contains method for finish loading and discarding preloaded scene.

QueueUtility<T>

A utility that provides queuing.

SceneCollectionUtility

An utility class to perform actions on collections.

SceneDataUtility

A utility for storing scene related data. Data can only be saved to disk in editor.

SceneHelper

An helper for opening and closing scenes or scene collections.

Most common use case would be to open / close scenes or scene collections from UnityEngine.Events.UnityEvent.

SceneUtility

An utility class to perform actions on scenes.

ScriptableObjectUtility

Contains utility methods for UnityEngine.ScriptableObject.

SerializableDictionary<TKey, TValue>

A serializable dictionary.

SerializableStringBoolDict

TimeSpanUtility

Interfaces

IQueueable

Represents a queueable item.

Class ASM

This is SceneHelper, but we don't want the script to show up in object picker to avoid confusion, using a different name seems to be the only way?

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
ASM

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
[AddComponentMenu("")]
public class ASM : ScriptableObject
```

Properties

name

Declaration

```
public string name { get; }
```

Property Value

TYPE	DESCRIPTION
System.String	

Methods

Close(Scene)

Closes a scene regardless of whatever it is associated with a collection, or is was opened as stand-alone.

Declaration

```
public void Close(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

CloseCollection()

Closes the current collection.

Declaration

```
public void CloseCollection()
```

DiscardPreload()

Closes the scene.

Declaration

```
public void DiscardPreload()
```

FindCollections(Scene)

Finds the collections that are associated with this Scene.

Declaration

```
public (SceneCollection collection, bool asLoadingScreen)[] FindCollections(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.ValueTuple<SceneCollection, System.Boolean>[]	

FinishPreload()

Finishes loading scene.

Declaration

```
public void FinishPreload()
```

IsOpen(Scene)

Gets whatever the scene is open, either as part of a collection, or as stand-alone.

Declaration

```
public IsOpenReturnValue IsOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
IsOpenReturnValue	

IsOpen(SceneCollection)

Gets whatever the collection is currently open.

Declaration

```
public bool IsOpen(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
System.Boolean	

Open(Scene)

Opens a scene.

Declaration

```
public void Open(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Remarks

Throws a OpenSceneException if the scene cannot be opened by the current collection.

Open(SceneCollection)

Declaration

```
public void Open(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

OpenOrReopen(Scene)

Opens the scene if not open, otherwise it will be reopened.

Declaration

```
public void OpenOrReopen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

OpenOrReopenCollection(SceneCollection)

Opens or reopens the collection, depending on whatever it is open or not.

Declaration

```
public void OpenOrReopenCollection(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

OpenSingle(Scene)

Close existing scenes and open the specified one.

Declaration

```
public void OpenSingle(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Remarks

This will close the current collection.

OpenWhereNameStartsWith(String)

Open all scenes that starts with the specified name.

Declaration

```
public void OpenWhereNameStartsWith(string name)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	

Preload(Scene)

Declaration

```
public void Preload(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Quit()

Declaration

```
public void Quit()
```

Reopen(Scene)

Declaration

```
public void Reopen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

ReopenCollection()

Reopens the current collection.

Declaration

```
public void ReopenCollection()
```

Restart()

Declaration

```
public void Restart()
```

RestartCollection()

Reopens the current collection.

Declaration

```
public void RestartCollection()
```

SetActiveScene(Scene)

Sets a scene as the activate scene.

Declaration

```
public void SetActiveScene(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Toggle(Scene)

Declaration

```
public void Toggle(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Toggle(Scene, Boolean)

Declaration

```
public void Toggle(Scene scene, bool enabled)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Boolean	enabled	

Toggle(SceneCollection)

Toggles the collection.

Declaration

```
public void Toggle(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Toggle(SceneCollection, Boolean)

Toggles the collection.

Declaration

```
public void Toggle(SceneCollection collection, bool enabled)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Boolean	enabled	

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class CanvasGroupExtensions

Inheritance

System.Object

CanvasGroupExtensions

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class CanvasGroupExtensions
```

Methods

Fade(CanvasGroup, Single, Single, Boolean)

Animates the alpha of a UnityEngine.CanvasGroup.

Declaration

```
public static IEnumerator Fade(this CanvasGroup group, float to, float duration, bool setBlocksRaycasts =
true)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.CanvasGroup	group	
System.Single	to	
System.Single	duration	
System.Boolean	setBlocksRaycasts	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Class CanvasSortOrderUtility

An utility class to manage sort order on canvases.

Inheritance

System.Object
CanvasSortOrderUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class CanvasSortOrderUtility
```

Methods

MakeSure(Canvas, Canvas, Canvas)

Adds a contraint on the sort order of this UnityEngine.Canvas based on one or two other canvases.

See parameter comments for more info.

Declaration

```
public static void MakeSure(this Canvas canvas, Canvas above = null, Canvas below = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Canvas	canvas	
UnityEngine.Canvas	above	Makes sure that this canvas is always above this one.
UnityEngine.Canvas	below	Makes sure that this canvas is always below this one.

PutAtBottom(Canvas)

Sets the sort order on this canvas to be on bottom of all other canvases managed by CanvasSortOrderUtility.

Declaration

```
public static void PutAtBottom(this Canvas canvas)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Canvas	canvas	

PutOnTop(Canvas)

Sets the sort order on this canvas to be on top of all other canvases managed by CanvasSortOrderUtility.

Declaration

```
public static void PutOnTop(this Canvas canvas)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Canvas	canvas	

Remove(Canvas)

Removes this canvas from the managed list.

Declaration

```
public static void Remove(Canvas canvas)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Canvas	canvas	

Class DefaultSceneUtility

An utility class that manages the default scene, called 'AdvancedSceneManager'.

The default scene allows us to more easily close all scenes when we need to, since unity requires at least one scene to be open at any time.

Inheritance

System.Object
DefaultSceneUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class DefaultSceneUtility
```

Fields

Name

Declaration

```
public const string Name = "AdvancedSceneManager"
```

Field Value

TYPE	DESCRIPTION
System.String	

Methods

IsDefaultScene(Scene)

Gets whatever the specified scene is the default scene.

Declaration

```
public static bool IsDefaultScene(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

Class DictionaryUtility

Inheritance

System.Object
DictionaryUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class DictionaryUtility
```

Methods

Add<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)

Adds or sets the value of a key.

Declaration

```
public static void Add<TKey, TValue>(this Dictionary<TKey, TValue> d, TKey key, TValue value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TValue>	d	
TKey	key	
TValue	value	

Type Parameters

NAME	DESCRIPTION
TKey	
TValue	

Add<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, TItem)

Adds the value to the list with the specified key. Creates list automatically if null and adds key if necessary.

Declaration

```
public static void Add<TKey, TList, TItem>(this Dictionary<TKey, TList> d, TKey key, TItem item)
    where TList : IList<TItem>, new()
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TList>	d	
TKey	key	
TItem	item	

Type Parameters

NAME	DESCRIPTION
TKey	
TList	
TItem	

AddRange<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, TItem[])

Adds the values to the list with the specified key. Creates list automatically if null and adds key if necessary.

Declaration

```
public static void AddRange<TKey, TList, TItem>(this Dictionary<TKey, TList> d, TKey key, params TItem[]
items)
    where TList : IList<TItem>, new()
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TList>	d	
TKey	key	
TItem[]	items	

Type Parameters

NAME	DESCRIPTION
TKey	
TList	
TItem	

AddRange<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, IEnumerable<TItem>)

Adds the values to the list with the specified key. Creates list automatically if null and adds key if necessary.

Declaration

```
public static void AddRange<TKey, TList, TItem>(this Dictionary<TKey, TList> d, TKey key, IEnumerable<TItem>
items)
    where TList : IList<TItem>, new()
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TList>	d	
TKey	key	

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<TItem>	items	

Type Parameters

NAME	DESCRIPTION
TKey	
TList	
TItem	

EnsureAdded<TKey, TValue>(Dictionary<TKey, TValue>, TKey)

Declaration

```
public static void EnsureAdded<TKey, TValue>(this Dictionary<TKey, TValue> d, TKey key)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TValue>	d	
TKey	key	

Type Parameters

NAME	DESCRIPTION
TKey	
TValue	

GetValue<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)

Gets the value of the specified key, returns default if it does not exist.

Declaration

```
public static TValue GetValue<TKey, TValue>(this Dictionary<TKey, TValue> d, TKey key, TValue defaultValue =
null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TValue>	d	
TKey	key	
TValue	defaultValue	

Returns

TYPE	DESCRIPTION
TValue	

Type Parameters

NAME	DESCRIPTION
TKey	
TValue	

Remove<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, TItem)

Removes the value to the list with the specified key.

Declaration

```
public static void Remove<TKey, TList, TItem>(this Dictionary<TKey, TList> d, TKey key, TItem value)
    where TList : IList<TItem>, new()
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TList>	d	
TKey	key	
TItem	value	

Type Parameters

NAME	DESCRIPTION
TKey	
TList	
TItem	

Set<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)

Declaration

```
public static TValue Set<TKey, TValue>(this Dictionary<TKey, TValue> d, TKey key, TValue value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TValue>	d	
TKey	key	
TValue	value	

Returns

TYPE	DESCRIPTION
TValue	

Type Parameters

NAME	DESCRIPTION
TKey	
TValue	

WithNullKey<TKey, TValue>(Dictionary<TKey, TValue>, TKey)

Declaration

```
public static Dictionary<TKey, TValue> WithNullKey<TKey, TValue>(this Dictionary<TKey, TValue> d, TKey
nullKey)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TValue>	d	
TKey	nullKey	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.Dictionary<TKey, TValue>	

Type Parameters

NAME	DESCRIPTION
TKey	
TValue	

Class GuidReference

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
GuidReference

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
[ExecuteAlways]
[ExecuteInEditMode]
public class GuidReference : MonoBehaviour
```

Fields

guid

Declaration

```
public string guid
```

Field Value

TYPE	DESCRIPTION
System.String	

Extension Methods

SceneUtility.Scene(Component)

Class GuidReference.Editor

Inheritance

System.Object
UnityEngine.Object
UnityEngine.ScriptableObject
UnityEditor.Editor
GuidReference.Editor

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
[CustomEditor(typeof(GuidReference))]
public class Editor : Editor
```

Methods

OnInspectorGUI()

Declaration

```
public override void OnInspectorGUI()
```

Overrides

UnityEditor.Editor.OnInspectorGUI()

UseDefaultMargins()

Declaration

```
public override bool UseDefaultMargins()
```

Returns

TYPE	DESCRIPTION
System.Boolean	

Overrides

UnityEditor.Editor.UseDefaultMargins()

Extension Methods

ScriptableObjectUtility.Save(ScriptableObject)

Class GuidReferenceUtility

An utility for referencing objects globally.

Inheritance

System.Object
GuidReferenceUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class GuidReferenceUtility
```

Methods

AddPersistent(GameObject)

Adds a persistent reference to this UnityEngine.GameObject.

Declaration

```
public static string AddPersistent(GameObject obj)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	obj	

Returns

TYPE	DESCRIPTION
System.String	

Remarks

Only usable in editor.

AddRuntime(Object)

Adds a reference to the object, returns the id that will be used to find it again.

Declaration

```
public static string AddRuntime(Object obj)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Object	obj	

Returns

TYPE	DESCRIPTION
System.String	

Find(String)

Finds a reference if it exists.

Declaration

```
public static Object Find(string id)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	

Returns

TYPE	DESCRIPTION
UnityEngine.Object	

Find(String, Action<Object>)

Finds a reference if it exists.

Declaration

```
public static IEnumerator Find(string id, Action<Object> callback)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	
System.Action<UnityEngine.Object>	callback	

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Find<T>(String)

Finds a reference if it exists.

Declaration

```
public static T Find<T>(string id)
    where T : Object
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

FindPersistent(String)

Finds the persistent reference in the currently open scenes.

Declaration

```
public static GameObject FindPersistent(string guid)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	guid	

Returns

TYPE	DESCRIPTION
UnityEngine.GameObject	

GenerateID()

Generates an id.

Declaration

```
public static string GenerateID()
```

Returns

TYPE	DESCRIPTION
System.String	

Remarks

Uses https://blog.codinghorror.com/equipping-our-ascii-armor.

GetOrAddPersistent(GameObject)

Adds a persistent reference to this UnityEngine.GameObject.

Declaration

```
public static string GetOrAddPersistent(GameObject obj)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	obj	

Returns

TYPE	DESCRIPTION
System.String	

Remarks

Can only add in editor, returns null otherwise.

HasReference(String)

Gets if reference exists.

Declaration

```
public static bool HasReference(string id)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	

Returns

TYPE	DESCRIPTION
System.Boolean	

RemovePersistent(GameObject, Boolean)

Removes a persistent reference to this UnityEngine.GameObject.

Declaration

```
public static void RemovePersistent(GameObject obj, bool saveScene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	obj	
System.Boolean	saveScene	

Remarks

Only usable in editor.

RemoveRuntime(String)

Removes the reference to the object with this id.

Declaration

```
public static void RemoveRuntime(string id)
```

Parameters

TYPE	NAME	DESCRIPTION

TYPE	NAME	DESCRIPTION
System.String	id	

RemoveRuntime(Object)

Removes the reference to this object.

Declaration

```
public static void RemoveRuntime(Object obj)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Object	obj	

TryFind(String, out Object)

Tries to find the reference.

Declaration

```
public static Object TryFind(string id, out Object obj)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	
UnityEngine.Object	obj	

Returns

TYPE	DESCRIPTION
UnityEngine.Object	

TryFind<T>(String, out T)

Gets if reference exists.

Declaration

```
public static bool TryFind<T>(string id, out T obj)
    where T : Object
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	id	
T	obj	

Returns

TYPE	DESCRIPTION
System.Boolean	

Type Parameters

NAME	DESCRIPTION
T	

TryFindPersistent(String, out GameObject)

Finds the persistent reference in the currently open scenes.

Declaration

```
public static bool TryFindPersistent(string guid, out GameObject obj)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	guid	
UnityEngine.GameObject	obj	

Returns

TYPE	DESCRIPTION
System.Boolean	

Class InGameToolbarUtility

Provides an in-game toolbar that makes debugging scene management in build easier.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
InGameToolbarUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class InGameToolbarUtility : MonoBehaviour
```

Remarks

Only activates in editor and developer builds, and is disabled in non dev build.

Properties

expandedByDefault

Declaration

```
public static bool expandedByDefault { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isEnabled

Declaration

```
public static bool isEnabled { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isEnabledInEditor

Enables or disables InGameToolbarUtility in editor.

Declaration

```
public static bool isEnabledInEditor { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Extension Methods

SceneUtility.Scene(Component)

Interface IQueueable

Represents a queueable item.

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public interface IQueueable
```

Remarks

See also QueueUtility<T>.

Methods

OnCancel()

Called when queueable is cancelled.

Declaration

```
void OnCancel()
```

OnTurn(Action)

Called when it is this queueables turn.

Declaration

```
void OnTurn(Action onComplete)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	onComplete	Must be called when operation is done, otherwise queue will be stuck.

Class LerpUtility

Provides some convinience functions for lerping.

Inheritance

System.Object
LerpUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class LerpUtility
```

Methods

Lerp(Single, Single, Single, Action<Single>, Action)

Lerp from `start` to `end` over `duration` seconds.

Declaration

```
public static IEnumerator Lerp(float start, float end, float duration, Action<float> callback, Action
onComplete = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Single	start	The start value.
System.Single	end	The end value.
System.Single	duration	The duration in seconds to lerp for.
System.Action<System.Single>	callback	The callback each lerp interval.
System.Action	onComplete	Callback when complete.

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Lerp(Vector2, Vector2, Single, Action<Vector2>, Action)

Lerp from `start` to `end` over `duration` seconds.

Declaration

```
public static IEnumerator Lerp(Vector2 start, Vector2 end, float duration, Action<Vector2> callback, Action
onComplete = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Vector2	start	
UnityEngine.Vector2	end	
System.Single	duration	The duration in seconds to lerp for.
System.Action<UnityEngine.Vector2>	callback	
System.Action	onComplete	Callback when complete.

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Lerp(Vector3, Vector3, Single, Action<Vector3>, Action)

Lerp from `start` to `end` over `duration` seconds.

Declaration

```
public static IEnumerator Lerp(Vector3 start, Vector3 end, float duration, Action<Vector3> callback, Action
onComplete = null)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Vector3	start	
UnityEngine.Vector3	end	
System.Single	duration	The duration in seconds to lerp for.
System.Action<UnityEngine.Vector3>	callback	
System.Action	onComplete	Callback when complete.

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

Class LoadingScreenUtility

Manager for loading screens.

Inheritance

System.Object
LoadingScreenUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class LoadingScreenUtility
```

Properties

fade

Finds the default fade loading screen. Will be null if not included in build.

Declaration

```
public static Scene fade { get; }
```

Property Value

TYPE	DESCRIPTION
Scene	

IsAnyLoadingScreenOpen

Gets if any loading screens are open.

Declaration

```
public static bool IsAnyLoadingScreenOpen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

loadingScreens

The currently open loading screens.

Declaration

```
public static ReadOnlyCollection<LoadingScreen> loadingScreens { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<LoadingScreen>	

Methods

CloseAll()

Hide all loading screens.

Declaration

```
public static SceneOperation CloseAll()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

CloseLoadingScreen(LoadingScreen)

Hide the loading screen.

Declaration

```
public static SceneOperation CloseLoadingScreen(LoadingScreen loadingScreen)
```

Parameters

TYPE	NAME	DESCRIPTION
LoadingScreen	loadingScreen	

Returns

TYPE	DESCRIPTION
SceneOperation	

CloseLoadingScreen(Scene)

Hide the loading screen.

Declaration

```
public static SceneOperation CloseLoadingScreen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
SceneOperation	

DoAction(Scene, Action, Action<LoadingScreen>)

Opens loading screen, performs action and hides loading screen again.

Declaration

```
public static SceneOperation DoAction(Scene scene, Action action, Action<LoadingScreen> loadingScreenCallback)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Action	action	
System.Action<LoadingScreen>	loadingScreenCallback	

Returns

TYPE	DESCRIPTION
SceneOperation	

DoAction(Scene, Func<IEnumerator>, Action<LoadingScreen>)

Opens loading screen, performs action and hides loading screen again.

Declaration

```
public static SceneOperation DoAction(Scene scene, Func<IEnumerator> coroutine, Action<LoadingScreen>
loadingScreenCallback)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.Func<System.Collections.IEnumerator>	coroutine	
System.Action<LoadingScreen>	loadingScreenCallback	

Returns

TYPE	DESCRIPTION
SceneOperation	

Remarks

Throws OpenSceneException if `scene` is null.

DoActionWithFade(Action, Single, Nullable<Color>)

Fades screen out, performs action and fades in again.

Declaration

```
public static SceneOperation DoActionWithFade(Action action, float duration = 1F, Color? color = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action	action	
System.Single	duration	

TYPE	NAME	DESCRIPTION
System.Nullable<UnityEngine.Color>	color	

Returns

TYPE	DESCRIPTION
SceneOperation	

DoActionWithFade(Func<IEnumerator>, Single, Nullable<Color>)

Fades screen out, performs action and fades in again.

Declaration

```
public static SceneOperation DoActionWithFade(Func<IEnumerator> action, float duration = 1F, Color? color =
null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<System.Collections.IEnumerator>	action	
System.Single	duration	
System.Nullable<UnityEngine.Color>	color	

Returns

TYPE	DESCRIPTION
SceneOperation	

FadeIn(LoadingScreen, Single, Nullable<Color>)

Fades in the screen.

Declaration

```
public static SceneOperation FadeIn(LoadingScreen loadingScreen, float duration = 1F, Color? color = null)
```

Parameters

TYPE	NAME	DESCRIPTION
LoadingScreen	loadingScreen	
System.Single	duration	
System.Nullable<UnityEngine.Color>	color	

Returns

TYPE	DESCRIPTION
SceneOperation	

FadeOut(Single, Nullable<Color>)

Fades out the screen.

Declaration

```
public static SceneOperation<LoadingScreen> FadeOut(float duration = 1F, Color? color = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Single	duration	
System.Nullable<UnityEngine.Color>	color	

Returns

TYPE	DESCRIPTION
SceneOperation<LoadingScreen>	

FindLoadingScreen(SceneCollection)

Find the loading screen that is associated with this collection.

Declaration

```
public static Scene FindLoadingScreen(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Returns

TYPE	DESCRIPTION
Scene	

IsLoadingScreenOpen(OpenSceneInfo)

Gets if this scene is a loading screen.

Declaration

```
public static bool IsLoadingScreenOpen(OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

IsLoadingScreenOpen(Scene)

Gets if this scene is a loading screen.

Declaration

```
public static bool IsLoadingScreenOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

IsLoadingScreenOpen(Scene)

Gets if this scene is a loading screen.

Declaration

```
public static bool IsLoadingScreenOpen(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
System.Boolean	

OpenLoadingScreen(Scene, Nullable<Single>, Action<LoadingScreen>, String)

Shows a loading screen.

Declaration

```
public static SceneOperation<LoadingScreen> OpenLoadingScreen(Scene scene, float? timeout = null,
Action<LoadingScreen> callbackBeforeBegin = null, string typeName = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

TYPE	NAME	DESCRIPTION
System.Nullable<System.Single>	timeout	The time to wait after opening scene before cancelling load screen.
System.Action<LoadingScreen>	callbackBeforeBegin	
System.String	typeName	Overrides the name of the type in the missing script message.

Returns

TYPE	DESCRIPTION
SceneOperation<LoadingScreen>	

OpenLoadingScreen(SceneCollection, Nullable<Single>, Action<LoadingScreen>)

Shows the loading screen associated with this collection.

Declaration

```
public static SceneOperation<LoadingScreen> OpenLoadingScreen(SceneCollection collection, float? timeout =
null, Action<LoadingScreen> callbackBeforeBegin = null)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	
System.Nullable<System.Single>	timeout	The time to wait after opening scene before cancelling load screen.
System.Action<LoadingScreen>	callbackBeforeBegin	

Returns

TYPE	DESCRIPTION
SceneOperation<LoadingScreen>	

OpenLoadingScreen<T>(Scene, Nullable<Single>, Action<T>, String)

Shows a loading screen.

Declaration

```
public static SceneOperation<T> OpenLoadingScreen<T>(Scene scene, float? timeout = null, Action<T>
callbackBeforeBegin = null, string typeName = null)
    where T : LoadingScreen
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

TYPE	NAME	DESCRIPTION
System.Nullable<System.Single>	timeout	The time to wait after opening scene before cancelling load screen.
System.Action<T>	callbackBeforeBegin	
System.String	typeName	Overrides the name of the type in the missing script message.

Returns

TYPE	DESCRIPTION
SceneOperation<T>	

Type Parameters

NAME	DESCRIPTION
T	

Class PauseScreenUtility

Contains functions for interacting with the default pause screen.

Inheritance

System.Object
UnityEngine.Object
UnityEngine.Component
UnityEngine.Behaviour
UnityEngine.MonoBehaviour
PauseScreenUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
[AddComponentMenu("")]
public class PauseScreenUtility : MonoBehaviour
```

Fields

canvasGroup

Declaration

```
public CanvasGroup canvasGroup
```

Field Value

TYPE	DESCRIPTION
UnityEngine.CanvasGroup	

quit

Declaration

```
public Button quit
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Button	

restartCollection

Declaration

```
public Button restartCollection
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Button	

restartGame

Declaration

```
public Button restartGame
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Button	

resume

Declaration

```
public Button resume
```

Field Value

TYPE	DESCRIPTION
UnityEngine.UI.Button	

Properties

isOpen

Gets if the pause screen is currently open.

Declaration

```
public static bool isOpen { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

Begin()

Declaration

```
public IEnumerator Begin()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

End()

Declaration

```
public IEnumerator End()
```

Returns

TYPE	DESCRIPTION
System.Collections.IEnumerator	

TYPE	DESCRIPTION

Hide(Boolean)

Hides the pause screen.

Declaration

```
public static void Hide(bool ignoreAnimations = false)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Boolean	ignoreAnimations	

ListenForKey()

Starts listening keys and opens pause screen when keys pressed.

Declaration

```
public static void ListenForKey()
```

Quit()

Declaration

```
public void Quit()
```

RestartCollection()

Declaration

```
public void RestartCollection()
```

RestartGame()

Declaration

```
public void RestartGame()
```

Resume()

Declaration

```
public void Resume()
```

Show()

Shows the pause screen.

Declaration

```
public static void Show()
```

StopListening()

Stops listening for keys, this will disable pause screen. (Manually calling Show() will still work though)

Declaration

```
public static void StopListening()
```

Toggle()

Toggles the pause screen on / off.

Declaration

```
public static void Toggle()
```

Extension Methods

SceneUtility.Scene(Component)

Class PersistentUtility

Manages persistent scenes.

Inheritance

System.Object
PersistentUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class PersistentUtility
```

Methods

GetPersistentOption(Scene)

Gets the persistent option that is set for this UnityEngine.SceneManagement.Scene.

Declaration

```
public static SceneCloseBehavior GetPersistentOption(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
SceneCloseBehavior	

Set(Scene, SceneCloseBehavior)

Set SceneCloseBehavior for this scene.

Declaration

```
public static void Set(Scene scene, SceneCloseBehavior behavior = SceneCloseBehavior.KeepOpenAlways)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	
SceneCloseBehavior	behavior	

Unset(Scene)

Unset and revert to default SceneCloseBehavior for this scene.

Declaration

```
public static void Unset(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

UnsetAll()

Unsets SceneCloseBehavior for all scenes.

Declaration

```
public static void UnsetAll()
```

Class PreloadedSceneHelper

An helper class to make working with preloaded scenes easier, contains method for finish loading and discarding preloaded scene.

Inheritance

System.Object
PreloadedSceneHelper

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public class PreloadedSceneHelper
```

Constructors

PreloadedSceneHelper(OpenSceneInfo, Boolean)

Declaration

```
public PreloadedSceneHelper(OpenSceneInfo scene, bool hasRunCallbacks)
```

Parameters

TYPE	NAME	DESCRIPTION
OpenSceneInfo	scene	
System.Boolean	hasRunCallbacks	

Properties

hasRunCallbacks

Gets whatever OnSceneOpen() callbacks has been called.

Declaration

```
public bool hasRunCallbacks { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isStillPreloaded

Gets whatever the scene is still preloaded.

Declaration

```
public bool isStillPreloaded { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

scene

The scene that was preloaded.

Declaration

```
public OpenSceneInfo scene { get; }
```

Property Value

TYPE	DESCRIPTION
OpenSceneInfo	

Methods

Discard()

Closes the scene.

Declaration

```
public SceneOperation Discard()
```

Returns

TYPE	DESCRIPTION
SceneOperation	

FinishLoading()

Finishes loading scene.

Declaration

```
public SceneOperation<OpenSceneInfo> FinishLoading()
```

Returns

TYPE	DESCRIPTION
SceneOperation<OpenSceneInfo>	

Class QueueUtility<T>

A utility that provides queuing.

Inheritance

System.Object

QueueUtility<T>

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class QueueUtility<T>
    where T : IQueueable
```

Type Parameters

NAME	DESCRIPTION
T	

Properties

isBusy

Gets whatever any items in the queue are running.

Declaration

```
public static bool isBusy { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

queue

Gets the items currently in queue.

Declaration

```
public static ReadOnlyCollection<T> queue { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<T>	

running

Gets the items that are currently running.

Declaration

```
public static ReadOnlyCollection<T> running { get; }
```

Property Value

TYPE	DESCRIPTION
System.Collections.ObjectModel.ReadOnlyCollection<T>	

Methods

IsQueued(T)

Get if the item is queued.

Declaration

```
public static bool IsQueued(T queueable)
```

Parameters

TYPE	NAME	DESCRIPTION
T	queueable	

Returns

TYPE	DESCRIPTION
System.Boolean	

IsRunning(T)

Gets if the item is running.

Declaration

```
public static bool IsRunning(T queueable)
```

Parameters

TYPE	NAME	DESCRIPTION
T	queueable	

Returns

TYPE	DESCRIPTION
System.Boolean	

StopAll()

Cancels all queued and running items.

Declaration

```
public static void StopAll()
```

Events

queueEmpty

Occurs when an queued item finishes and queue is empty.

Declaration

```
public static event Action queueEmpty
```

Event Type

TYPE	DESCRIPTION
System.Action	

Class SceneCollectionUtility

An utility class to perform actions on collections.

Inheritance

System.Object
SceneCollectionUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class SceneCollectionUtility
```

Methods

Create(String, Profile)

Creates a SceneCollection.

Declaration

```
public static SceneCollection Create(string title, Profile profile = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	title	The name of the collection.
Profile	profile	The profile to add this collection to. Defaults to current.

Returns

TYPE	DESCRIPTION
SceneCollection	

Find(IEnumerable<OpenSceneInfo>, Scene)

Finds the scene with the specified path.

Declaration

```
public static OpenSceneInfo Find(this IEnumerable<OpenSceneInfo> list, Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	list	
Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

Find(IEnumerable<OpenSceneInfo>, Scene)

Finds the scene with the specified path.

Declaration

```
public static OpenSceneInfo Find(this IEnumerable<OpenSceneInfo> list, Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<OpenSceneInfo>	list	
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

Find(IEnumerable<Scene>, String)

Finds the scene with the specified path.

Declaration

```
public static Scene Find(this IEnumerable<Scene> list, string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	list	The list of scenes to query.
System.String	path	The path of the scene to find.

Returns

TYPE	DESCRIPTION
Scene	

Find(IEnumerable<Scene>, String)

Finds the scene with the specified path.

Declaration

```
public static Scene Find(this IEnumerable<Scene> list, string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Collections.Generic.IEnumerable<UnityEngine.SceneManagement.Scene>	list	
System.String	path	The path of the scene to find.

Returns

TYPE	DESCRIPTION
UnityEngine.SceneManagement.Scene	

Remove(SceneCollection)

Removes a SceneCollection.

Declaration

```
public static void Remove(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

RemoveNullScenes(SceneCollection)

Removes all null scenes in the collection.

Declaration

```
public static void RemoveNullScenes(SceneCollection collection)
```

Parameters

TYPE	NAME	DESCRIPTION
SceneCollection	collection	

Class SceneDataUtility

A utility for storing scene related data. Data can only be saved to disk in editor.

Inheritance

System.Object
SceneDataUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class SceneDataUtility
```

Methods

Enumerate<T>(String)

Enumerates all `T` on all scenes.

Declaration

```
public static IEnumerable<T> Enumerate<T>(string key)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	key	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<T>	

Type Parameters

NAME	DESCRIPTION
T	

Get<T>(Scene, String, T)

Gets the value with the specified key, for the specified scene.

Declaration

```
public static T Get<T>(Scene scene, string key, T defaultValue = null)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	key	
T	defaultValue	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

Get<T>(String, String, T)

Gets the value with the specified key, for the specified scene.

Declaration

```
public static T Get<T>(string scene, string key, T defaultValue = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.String	key	
T	defaultValue	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

GetDirect(Scene, String)

Gets the value with the specified key, for the specified scene. This is the direct version, all values are stores as string, which means Get<T>(String, String, T) must convert value beforehand, this method doesn't.

Declaration

```
public static string GetDirect(Scene scene, string key)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	key	

Returns

TYPE	DESCRIPTION
System.String	

GetDirect(String, String)

Gets the value with the specified key, for the specified scene. This is the direct version, all values are stores as string, which means Get<T>(String, String, T) must convert value beforehand, this method doesn't.

Declaration

```
public static string GetDirect(string scene, string key)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.String	key	

Returns

TYPE	DESCRIPTION
System.String	

Set<T>(Scene, String, T)

Sets the value with the specified key, for the specified scene.

Declaration

```
public static void Set<T>(Scene scene, string key, T value)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	key	
T	value	

Type Parameters

NAME	DESCRIPTION
T	

Set<T>(String, String, T)

Sets the value with the specified key, for the specified scene.

Declaration

```
public static void Set<T>(string scene, string key, T value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.String	key	
T	value	

Type Parameters

NAME	DESCRIPTION
T	

SetDirect(Scene, String, String)

Sets the value with the specified key, for the specified scene. This is the direct version, all values are stores as string, which means Get<T>(String, String, T) must convert value beforehand, this method doesn't.

Declaration

```
public static void SetDirect(Scene scene, string key, string value)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	key	
System.String	value	

SetDirect(String, String, String)

Sets the value with the specified key, for the specified scene. This is the direct version, all values are stores as string, which means Get<T>(String, String, T) must convert value beforehand, this method doesn't.

Declaration

```
public static void SetDirect(string scene, string key, string value)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.String	key	
System.String	value	

Unset(Scene, String)

Unsets the value with the specified key, for the specified scene.

Declaration

```
public static void Unset(Scene scene, string key)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	
System.String	key	

Unset(String, String)

Unsets the value with the specified key, for the specified scene.

Declaration

```
public static void Unset(string scene, string key)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	scene	
System.String	key	

Class SceneHelper

An helper for opening and closing scenes or scene collections.

Most common use case would be to open / close scenes or scene collections from UnityEngine.Events.UnityEvent.

Inheritance

System.Object
SceneHelper

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class SceneHelper
```

Remarks

See also: ASM.

Properties

current

Declaration

```
public static ASM current { get; }
```

Property Value

TYPE	DESCRIPTION
ASM	

Class SceneUtility

An utility class to perform actions on scenes.

Inheritance

System.Object
SceneUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class SceneUtility
```

Properties

hasAnyScenes

Gets if there are any scenes open that are not dynamically created, and not yet saved to disk.

Declaration

```
public static bool hasAnyScenes { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

isStartupScene

Gets if current, and only, scene is the startup scene.

Declaration

```
public static bool isStartupScene { get; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

sceneCount

Declaration

```
public static int sceneCount { get; }
```

Property Value

TYPE	DESCRIPTION
System.Int32	

Methods

Create(Action<Scene>, SceneCollection, Nullable<Int32>, Boolean, Boolean)

Creates a scene, using save prompt for path. Returns null if save dialog cancelled.

Declaration

```
public static void Create(Action<Scene> onCreated, SceneCollection collection = null, int? index = null, bool replaceIndex = false, bool save = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<Scene>	onCreated	
SceneCollection	collection	The collection to add the scene to.
System.Nullable<System.Int32>	index	The index of the scene in `collection`, no effect if `collection` is null.
System.Boolean	replaceIndex	Replaces the scene at the specified index, rather than insert it.
System.Boolean	save	Save collection to disk.

Remarks

Only usable in editor

Create(String, SceneCollection, Nullable<Int32>, Boolean, Boolean, Boolean)

Creates a scene at the specified path.

Declaration

```
public static Scene Create(string path, SceneCollection collection = null, int? index = null, bool replaceIndex = false, bool save = true, bool createSceneScriptableObject = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	The path that the scene should be saved to.
SceneCollection	collection	The collection to add the scene to.
System.Nullable<System.Int32>	index	The index of the scene in `collection`, no effect if `collection` is null.
System.Boolean	replaceIndex	Replaces the scene at the specified index, rather than insert it.
System.Boolean	save	Save collection to disk.

TYPE	NAME	DESCRIPTION
System.Boolean	createSceneScriptableObject	If false, no Scene(GameObject) UnityEngine.ScriptableObject will be created, scene also won't be added to `collection`. Returns null.

Returns

TYPE	DESCRIPTION
Scene	

Remarks

Only usable in editor

CreateDynamic(String, LocalPhysicsMode)

Creates a scene at runtime, that is not saved to disk.

Declaration

```
public static OpenSceneInfo CreateDynamic(string name, LocalPhysicsMode localPhysicsMode =
LocalPhysicsMode.None)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
UnityEngine.SceneManagement.LocalPhysicsMode	localPhysicsMode	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

CreateInCurrentFolder(Action<Scene>, SceneCollection, Nullable<Int32>, Boolean, Boolean)

Creates a scene, using UnityEditor.ProjectWindowUtil.CreateScene().

Declaration

```
public static void CreateInCurrentFolder(Action<Scene> onCreated, SceneCollection collection = null, int?
index = null, bool replaceIndex = false, bool save = true)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Action<Scene>	onCreated	
SceneCollection	collection	The collection to add the scene to.

TYPE	NAME	DESCRIPTION
System.Nullable<System.Int32>	index	The index of the scene in `collection`, no effect if `collection` is null.
System.Boolean	replaceIndex	Replaces the scene at the specified index, rather than insert it.
System.Boolean	save	Save collection to disk.

Remarks

Only usable in editor

Find(Func<Scene, Boolean>, SceneCollection, Profile)

Find scenes by predicate, in the specified collection or profile, if defined.

Declaration

```
public static IEnumerable<Scene> Find(Func<Scene, bool> predicate, SceneCollection inCollection = null,
Profile inProfile = null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<Scene, System.Boolean>	predicate	
SceneCollection	inCollection	
Profile	inProfile	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	

Find(String, SceneCollection, Profile)

Find scenes by name, in the specified collection or profile, if defined.

Declaration

```
public static IEnumerable<Scene> Find(string name, SceneCollection inCollection = null, Profile inProfile =
null)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	
SceneCollection	inCollection	

TYPE	NAME	DESCRIPTION
Profile	inProfile	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	

FindASMScene(SceneAsset)

Finds the asm representation of this UnityEditor.SceneAsset.

Declaration

```
public static Scene FindASMScene(this SceneAsset scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEditor.SceneAsset	scene	

Returns

TYPE	DESCRIPTION
Scene	

Remarks

Only available in editor.

FindOpen(Func<Scene, Boolean>)

Find open scenes by predicate.

Declaration

```
public static IEnumerable<Scene> FindOpen(Func<Scene, bool> predicate)
```

Parameters

TYPE	NAME	DESCRIPTION
System.Func<Scene, System.Boolean>	predicate	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	

FindOpen(String)

Find open scenes by name.

Declaration

```
public static IEnumerable<Scene> FindOpen(string name)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	name	

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<Scene>	

GetAllOpenUnityScenes()

Get all open unity scenes.

Declaration

```
public static IEnumerable<Scene> GetAllOpenUnityScenes()
```

Returns

TYPE	DESCRIPTION
System.Collections.Generic.IEnumerable<UnityEngine.SceneManagement.Scene>	

MergeScenes(String[])

Merges the scenes together, the first scene in the list will be the output scene.

Declaration

```
public static void MergeScenes(params string[] scenes)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String[]	scenes	

Remarks

Only available in editor.

Move(GameObject, OpenSceneInfo)

Declaration

```
public static void Move(this GameObject obj, OpenSceneInfo scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	obj	
OpenSceneInfo	scene	

Move(GameObject, Scene)

Declaration

```
public static void Move(this GameObject obj, Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	obj	
UnityEngine.SceneManagement.Scene	scene	

MoveToNewScene(GameObject[])

Moves the object to a new scene.

Only available in editor.

Declaration

```
public static void MoveToNewScene(params GameObject[] objects)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject[]	objects	

Remove(Scene)

Removes the `scene` and its associated UnityEditor.SceneAsset, and removes any references to it from any SceneCollection.

Declaration

```
public static void Remove(Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
Scene	scene	

Remove(String)

Removes the UnityEditor.SceneAsset at the specified path and its associated Scene(GameObject), and removes any references to it from any SceneCollection.

Declaration

```
public static void Remove(string path)
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	path	

Scene(Component)

Gets the runtime info of the associated scene to this UnityEngine.Component.

Declaration

```
public static OpenSceneInfo Scene(this Component component)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.Component	component	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

Scene(GameObject)

Gets the runtime info of the associated scene to this UnityEngine.GameObject.

Declaration

```
public static OpenSceneInfo Scene(this GameObject gameObject)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.GameObject	gameObject	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

Scene(Scene)

Gets the ASM runtime info of this UnityEngine.SceneManagement.Scene.

Declaration

```
public static OpenSceneInfo Scene(this Scene scene)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.SceneManagement.Scene	scene	

Returns

TYPE	DESCRIPTION
OpenSceneInfo	

Class ScriptableObjectUtility

Contains utility methods for UnityEngine.ScriptableObject.

Inheritance

System.Object
ScriptableObjectUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class ScriptableObjectUtility
```

Methods

GetSingleton<T>(String, String)

Gets a singleton instance of the specified type.

Declaration

```
public static T GetSingleton<T>(string assetPath, string resourcesPath)
    where T : ScriptableObject
```

Parameters

TYPE	NAME	DESCRIPTION
System.String	assetPath	
System.String	resourcesPath	

Returns

TYPE	DESCRIPTION
T	

Type Parameters

NAME	DESCRIPTION
T	

Save(ScriptableObject)

Saves the UnityEngine.ScriptableObject.

Declaration

```
public static void Save(this ScriptableObject obj)
```

Parameters

TYPE	NAME	DESCRIPTION
UnityEngine.ScriptableObject	obj	

Remarks

Safe to call from outside editor, but has no effect.

Class SerializableDictionary<TKey, TValue>

A serializable dictionary.

Inheritance

System.Object
System.Collections.Generic.Dictionary<TKey, TValue>
SerializableDictionary<TKey, TValue>
TagList
SerializableStringBoolDict

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class SerializableDictionary<TKey, TValue> : Dictionary<TKey, TValue>, IDictionary<TKey, TValue>,
ICollection<KeyValuePair<TKey, TValue>>, IReadOnlyDictionary<TKey, TValue>,
IReadOnlyCollection<KeyValuePair<TKey, TValue>>, IEnumerable<KeyValuePair<TKey, TValue>>, IDictionary,
ICollection, IEnumerable, IDeserializationCallback, ISerializable, ISerializationCallbackReceiver
```

Type Parameters

NAME	DESCRIPTION
TKey	
TValue	

Remarks

Older unity versions might need a wrapper class, since unity does not support serializing generic types. Don't forget System.SerializableAttribute on wrapper!

Fields

keys

Declaration

```
[SerializeField]
protected List<TKey> keys
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<TKey>	

values

Declaration

```
[SerializeField]
protected List<TValue> values
```

Field Value

TYPE	DESCRIPTION
System.Collections.Generic.List<TValue>	

Properties

throwOnDeserializeWhenKeyValueMismatch

Declaration

```
public bool throwOnDeserializeWhenKeyValueMismatch { get; set; }
```

Property Value

TYPE	DESCRIPTION
System.Boolean	

Methods

OnAfterDeserialize()

Declaration

```
public virtual void OnAfterDeserialize()
```

OnBeforeSerialize()

Declaration

```
public virtual void OnBeforeSerialize()
```

Extension Methods

DictionaryUtility.WithNullKey<TKey, TValue>(Dictionary<TKey, TValue>, TKey)
DictionaryUtility.Set<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)
DictionaryUtility.EnsureAdded<TKey, TValue>(Dictionary<TKey, TValue>, TKey)
DictionaryUtility.Add<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)
DictionaryUtility.GetValue<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)
ListUtility.ForEach<T>(IEnumerable<T>, Action<T, Int32>)
ListUtility.Flatten<T>(IEnumerable<T>, Func<T, IEnumerable<T>>)
ListUtility.Except<T>(IEnumerable<T>, T)
ListUtility.GroupConsecutive<T>(IEnumerable<T>, Func<T, T, Boolean>)

Class SerializableStringBoolDict

Inheritance

System.Object

System.Collections.Generic.Dictionary<System.String, System.Boolean>

SerializableDictionary<System.String, System.Boolean>

SerializableStringBoolDict

Inherited Members

SerializableDictionary<String, Boolean>.throwOnDeserializeWhenKeyValueMismatch

SerializableDictionary<String, Boolean>.keys

SerializableDictionary<String, Boolean>.values

SerializableDictionary<String, Boolean>.OnBeforeSerialize()

SerializableDictionary<String, Boolean>.OnAfterDeserialize()

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
[Serializable]
public class SerializableStringBoolDict : SerializableDictionary<string, bool>, IDictionary<string, bool>,
ICollection<KeyValuePair<string, bool>>, IReadOnlyDictionary<string, bool>,
IReadOnlyCollection<KeyValuePair<string, bool>>, IEnumerable<KeyValuePair<string, bool>>, IDictionary,
ICollection, IEnumerable, IDeserializationCallback, ISerializable, ISerializationCallbackReceiver
```

Extension Methods

DictionaryUtility.WithNullKey<TKey, TValue>(Dictionary<TKey, TValue>, TKey)

DictionaryUtility.Set<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)

DictionaryUtility.EnsureAdded<TKey, TValue>(Dictionary<TKey, TValue>, TKey)

DictionaryUtility.Add<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)

DictionaryUtility.Add<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, TItem)

DictionaryUtility.AddRange<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, IEnumerable<TItem>)

DictionaryUtility.AddRange<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, TItem[])

DictionaryUtility.Remove<TKey, TList, TItem>(Dictionary<TKey, TList>, TKey, TItem)

DictionaryUtility.GetValue<TKey, TValue>(Dictionary<TKey, TValue>, TKey, TValue)

ListUtility.ForEach<T>(IEnumerable<T>, Action<T, Int32>)

ListUtility.Flatten<T>(IEnumerable<T>, Func<T, IEnumerable<T>>)

ListUtility.Except<T>(IEnumerable<T>, T)

ListUtility.GroupConsecutive<T>(IEnumerable<T>, Func<T, T, Boolean>)

Class TimeSpanUtility

Inheritance

System.Object

TimeSpanUtility

Namespace: AdvancedSceneManager.Utility

Assembly: AdvancedSceneManager.dll

Syntax

```
public static class TimeSpanUtility
```

Methods

FormatUnits_Components(Single, String)

Formats the time as a readable string.

Declaration

```
public static (string time, string unit) FormatUnits_Components(float milliseconds, string format = "#.###")
```

Parameters

TYPE	NAME	DESCRIPTION
System.Single	milliseconds	
System.String	format	

Returns

TYPE	DESCRIPTION
System.ValueTuple<System.String, System.String>	

ToDisplayString(Single, String)

Declaration

```
public static string ToDisplayString(this float milliseconds, string format = "#.###")
```

Parameters

TYPE	NAME	DESCRIPTION
System.Single	milliseconds	
System.String	format	

Returns

TYPE	DESCRIPTION
System.String	

ToDisplayString(TimeSpan, String)

Declaration

```
public static string ToDisplayString(this TimeSpan timeSpan, string format = "#.###")
```

Parameters

TYPE	NAME	DESCRIPTION
System.TimeSpan	timeSpan	
System.String	format	

Returns

TYPE	DESCRIPTION
System.String	

ToDisplayString_Components(TimeSpan, String)

Declaration

```
public static (string time, string unit) ToDisplayString_Components(this TimeSpan timeSpan, string format =
"#.###")
```

Parameters

TYPE	NAME	DESCRIPTION
System.TimeSpan	timeSpan	
System.String	format	

Returns

TYPE	DESCRIPTION
System.ValueTuple<System.String, System.String>	